Från monolit till mikrotjänster

Erland Lindmark

Innehåll

Inledning **3**

1. Det välbekanta IT-stödet **6**

2. Vad händer när systemet delas upp? **14**

3. Vem ansvarar för vad när systemet blir distribuerat? **21**

4. Hur ska systemet delas upp? **28**

5. Var ska tjänsterna köras? **35**

6. Hur kommunicerar tjänsterna? **40**

7. Vem ansvarar för informationen? **47**

8. Hur förändras utvecklingsmiljöerna? **54**

9. Hur förändras testmiljöer och kvalitetssäkring? **60**

10. Hur levererar vi förändringar utan att störa helheten? **67**

11. Vem tillhandahåller miljöer och gemensamma förmågor? **74**

12. Vad händer när en tjänst slutar fungera? **80**

13. Hur vet vi att hela IT-stödet fungerar? **86**

14. Hur felsöker och hanterar vi incidenter? **91**

15. Hur skyddar vi systemet och dess information? 98

16. Hur hanterar vi livscykler, underhåll och teknisk skuld? 105

17. Hur säkerställer vi återställning, kontinuitet och kostnadskontroll? 112

18. Hur organiserar vi ansvar och styrning över tid? 119

19. Hur lever vi med gamla och nya arkitekturer samtidigt? 126

20. När är mikrotjänster rätt väg? 133

Inledning

Ett IT-stöd kan se enkelt ut från användarens perspektiv. En handläggare öppnar ett ärende, läser dokument, fattar beslut och skickar ett meddelande. Bakom detta finns emellertid en lång rad tekniska och organisatoriska förutsättningar: applikationer måste samverka, data måste vara tillförlitliga, nätverket måste fungera och någon måste kunna upptäcka och avhjälpa fel.

I ett traditionellt system är mycket av detta samlat. Frontend, backend och databas ingår ofta i en gemensam installation eller åtminstone i en samordnad livscykel. Det finns kända servrar och miljöer, etablerade driftinstruktioner och ofta en tydlig väg från ändringsbeställning till driftsättning. Även när många personer och leverantörer är inblandade finns det ett system att peka på.

När samma verksamhetsstöd delas upp i mikrotjänster förändras förutsättningarna. Funktioner som tidigare anropade varandra inom samma applikation kan behöva kommunicera över nätverk. Information som tidigare uppdaterades i en gemensam databas kan ligga under olika tjänsters ansvar. Tjänster kan driftsättas, uppgraderas och gå sönder oberoende av varandra. Men användaren förväntar sig fortfarande att *ärendehanteringen* fungerar som en helhet.

Det är denna förskjutning boken handlar om: **från en samordnad applikation till en distribuerad lösning som fortfarande måste leverera ett sammanhållet IT-stöd**.

Två utgångspunkter, samma helhet

Boken vänder sig till två grupper av erfarna läsare. Den ena kommer från applikationssidan: systemutveckling, applikationsarkitektur, test, integrationer eller applikationsförvaltning. Den andra kommer från driftsidan: server- och virtualiseringsdrift, nätverk, databasadministration, IT-säkerhet, plattformar, övervakning eller teknisk förvaltning. De två grupperna kan redan arbeta nära varandra, och i vissa organisationer tillhör de samma team.

Du behöver inte behärska den andra gruppens hela yrkesområde. Däremot behöver du förstå varför dina egna beslut påverkar andra delar av organisationen. En ny tjänstegräns kan skapa nya kommunikationsvägar och nätverksregler. En ny driftplattform kan förändra hur applikationer måste hantera konfiguration, fel och övervakning. Ingen av dessa förändringar kan beskrivas rättvisande som enbart en utvecklings- eller driftfråga.

Vad boken inte förutsätter

Du förväntas känna till hur ett traditionellt IT-stöd utvecklas eller förvaltas, men inte ha byggt mikrotjänster tidigare. Centrala begrepp introduceras när de behövs. Boken är inte en programmeringshandledning och inte heller en manual för Kubernetes, molntjänster eller en viss leverantörs produkter.

Mikrotjänster kräver varken containrar, en viss molnplattform eller en bestämd teamorganisation. Sådana lösningar förekommer ofta eftersom de kan underlätta automatisering och oberoende leveranser. De är ändå val, inte definitionen av arkitekturen. Boken skiljer därför på konsekvenserna av ett distribuerat system och på lösningar som en organisation väljer för att hantera konsekvenserna.

Ett ärendehanteringssystem som följer med

Genom boken följer vi ett fiktivt ärendehanteringsstöd. I utgångsläget har det en frontend, en backend och en gemensam relationsdatabas. Miljöerna är relativt förutsägbara: utveckling, test och produktion, med en uppsättning virtuella maskiner och tydliga driftkontakter.

Verksamheten växer. Dokumenthantering, notifieringar, beslut och parts-

information får nya krav och olika förändringstakt. Organisationen överväger att bryta ut funktioner till självständiga tjänster. För varje steg undersöker vi vad som förändras i programvaran, infrastrukturen och ansvarsfördelningen. Exemplet är avsiktligt förenklat; inga detaljer ska uppfattas som ett påstående att alla organisationer bör välja samma arkitektur.

Hur boken är upplagd

Först etableras vad vi menar med ett traditionellt IT-stöd och med mikrotjänster. Därefter behandlas hur funktioner delas upp, var de körs, hur de kommunicerar och hur dataansvar fördelas. Vi följer förändringen genom utvecklings- och testmiljöer, leverans, övervakning, incidenthantering och långsiktig förvaltning. Till sist samlas frågorna om styrning, ansvar och valet mellan olika arkitekturer.

En återkommande jämförelse visar vad som var bekant tidigare, varför förutsättningarna förändras, vilka alternativa lösningar som finns och vem som behöver ansvara för vad. Ansvar delas upp i att ställa krav, fatta beslut, genomföra, sköta driften och följa upp resultat; det är inte alltid samma funktion som gör allt. Den som kommer från applikationssidan får se driftens konsekvenser av arkitekturbesluten. Den som kommer från driftsidan får se hur applikationens egenskaper avgör vad som faktiskt går att drifta säkert och effektivt.

Syftet är inte att övertyga dig om att mikrotjänster alltid är bättre. En välstrukturerad monolit kan vara både enklare, billigare och säkrare att förvalta. Syftet är att ge underlag för bättre beslut när självständiga tjänster är aktuella – och för bättre samarbete oavsett vilken arkitektur som väljs. R1

Källor och vidare läsning

- **R1** Martin Fowler, *Microservice Trade-Offs* (2015).

1. Det välbekanta IT-stödet

Ett ärende kommer in på måndagsmorgonen. En handläggare öppnar det i webbläsaren, kompletterar uppgifter, lägger till ett dokument och skickar ärendet vidare. För användaren är allt detta delar av ett och samma IT-stöd. För organisationen bakom tjänsten är det också ofta ett sammanhållet objekt att utveckla, driftsätta, skydda och förvalta.

Innan vi undersöker vad mikrotjänster förändrar behöver vi förstå varför den modellen har fungerat så väl – och vilka antaganden som lätt följer med när man lämnar den.

Ett system, flera tekniska delar

Vårt fiktiva ärendehanteringsstöd har tre tydliga huvuddelar: frontend, backend och en relationsdatabas. Frontend visar ärenden och tar emot användarens inmatning. Backend hanterar regler för till exempel ärendestatus, behörigheter och beslutsfattande. Databasen lagrar ärenden, dokumentmetadata, parter, beslut och historik.

Det betyder inte att systemet är enkelt. Backend kan rymma hundratals moduler, integrationer och affärsregler. Frontend och backend kan driftsättas separat, och databasen kan ligga på andra servrar. Det avgörande i detta exempel är snarare att flera funktioner förvaltas och förändras inom en gemensamt samordnad applikationshelhet.

En *monolit* betyder i arkitektursammanhang vanligen att en större del av applikationens funktioner levereras som en gemensam driftsättningsenhet. Frontend, backend och databas är däremot en beskrivning av

tekniska lager, inte i sig ett bevis för att lösningen är monolitisk. Det är inte samma sak som dålig kodstruktur. En modulär monolit kan ha tydliga interna gränser och en välordnad domänmodell. Omvänt kan tre tekniska lager ha omfattande inbördes beroenden även när de körs på skilda servrar.

Vi använder fortsättningsvis det traditionella IT-stödet som jämförelsepunkt, inte som en karikatyr av föråldrad teknik.

Från användarens funktion till driftens installation

På applikationssidan talar man ofta om funktioner: registrera ärende, fatta beslut, arkivera dokument. På driftsidan talar man ofta om installationer: applikationsserver, databasserver, lastbalanserare och anslutningar genom nätverket. I ett traditionellt system kan dessa två kartor relativt enkelt knytas till samma systemnamn.

En möjlig produktionsmiljö består av två virtuella maskiner för webb och applikation, en separat databasmiljö samt en lastbalanserare framför webbgränssnittet. Databasmiljön kan i sin tur bestå av flera instanser eller en hanterad databastjänst. I praktiken kan det finnas ytterligare noder för hög tillgänglighet, driftövervakning och integrationer. Det är inte antalet maskiner som gör miljön traditionell. Det är att installationen ofta planeras och förändras som en sammanhållen systemmiljö.

Applikationens arkitektur och driftens topologi är två olika bilder av samma lösning. Ett viktigt grundarbete är att kunna översätta mellan dem. En funktion kan köras på flera maskiner; en maskin kan bära flera funktioner. Den relationen är sällan exakt ett till ett, inte ens i en monolit.

Miljöerna skapar förutsägbarhet

De flesta etablerade organisationer skiljer mellan utveckling, test och produktion. Ibland finns dessutom integrations-, acceptans- och utbildningsmiljöer. Varje miljö har sitt syfte, sina åtkomsträttigheter och sin driftmässiga förvaltning.

Applikationsgruppen kan arbeta i utvecklingsmiljön, sammanställa en release och lämna över den för test. Driftorganisationen kan etablera

servrar, öppna nätverksförbindelser, installera certifikat, säkra databaser och planera driftsättningen. Testorganisationen kan verifiera den nya systemversionen mot ett definierat urval av testdata.

Detta behöver inte ske manuellt. Även traditionella system kan använda automatiserade bygg- och leveransflöden, infrastruktur som kod och självbetjäning. Skillnaden som blir viktig senare är att **systemets delar ofta har en gemensam leveransrytm och att miljön ses som en installation av just detta system**.

Ansvar som löper genom flera organisationer

En vanlig ansvarsfördelning kan se ut så här:

Område	Typiskt ansvar i traditionell organisation	Det som måste samordnas
Funktion och applikationskod	Utvecklings- och applikationsförvaltning	Krav, kvalitet, felrättningar och release
Servrar och virtualisering	Infrastrukturdrift	Kapacitet, operativsystem, tillgänglighet och patchning
Nätverk och brandväggar	Nätverksdrift och säkerhetsfunktion	Tillåtna trafikflöden, zoner och felsökning
Databasteknik	DBA eller databasdrift	Installation, prestanda, backup och återställning
Behörighet och autentisering	Applikationsansvariga tillsammans med identitetsfunktion	Inloggning, roller, åtkomstkontroll och spårbarhet
Verksamhetsfunktion och prioritering	Systemägare eller verksamhetsnära förvaltning	Nytta, finansiering och förändringsbeslut

Tabellen beskriver ett vanligt organisationsmönster, inte en universell ansvarsfördelning. Flera roller kan ligga i samma team, och molntjänster eller externa leverantörer kan flytta det praktiska utförandet. Men

ansvar måste ändå vara uttalat. För varje område behöver man skilja på verksamhetens krav, tekniska beslut, den dagliga driften och uppföljningen av att tjänsten fungerar.

Det traditionella systemnamnet fungerar ofta som en gemensam referenspunkt. Systemägaren kan samtidigt ansvara för verksamhetens samlade nytta utan att själv ansvara för exempelvis nätverkskomponenternas tekniska drift. När drift får larm om *ärendehanteringen* finns kända kontaktvägar, systemdokumentation och en avgränsad uppsättning komponenter att undersöka.

Den gemensamma databasen är både styrka och beroende

En gemensam relationsdatabas gör det ofta enkelt att läsa och uppdatera sammanhängande information i en transaktion. Om ett beslut registreras och ärendets status ska uppdateras kan båda förändringarna i många fall genomföras atomärt: antingen lyckas båda eller också rullas de tillbaka.

Det ger en begriplig konsistensmodell. Databasadministratörer kan arbeta med index, behörigheter, replikering, backup och återställning med kännedom om en gemensam databasinstallation. Utvecklare kan göra sammanställningar över flera tabeller utan att nödvändigtvis använda ett nätverksanrop mellan varje informationsområde.

Samtidigt uppstår kopplingar. Om många funktioner läser direkt ur samma tabeller kan en schemaändring få stora konsekvenser. En långvarig transaktion kan påverka andra delar av systemet. Ett växande databasbehov kan tvinga fram gemensamma prioriteringar. Den gemensamma databasen är därför både en integrationsfördel och en potentiell begränsning för oberoende förändringar. Att samla data i samma databasteknik behöver dock inte innebära att alla moduler tillåts läsa och skriva varandras tabeller.

Här finns en lärdom som blir central senare: att dela upp dataansvaret kan minska vissa kopplingar, men det tar också bort möjligheten att utan vidare förlita sig på en enda databastransaktion för alla verksamhetssteg.

Nätverk, brandväggar och lastbalansering har tydliga platser

I vår traditionella miljö kan nätverket beskrivas med ett relativt begränsat antal flöden: användare till webbgränssnitt, webb/applikation till databas och anslutningar från applikationen till ett antal externa system. Brandväggsregler och säkerhetszoner kan dokumenteras per systemmiljö och tekniskt lager. Lastbalanseraren styr trafik till tillgängliga applikationsinstanser.

Det betyder inte att nätverket alltid är statiskt eller enkelt. Men antalet kommunikationsparter är ofta hanterbart och förändringar kan kopplas till en samordnad release. Det blir relativt tydligt vilken grupp som hanterar nätverkskomponenten och vilken applikationsgrupp som behöver beskriva sitt trafikbehov. Även i traditionella miljöer kan trafikregler vara automatiserade och åtkomsten styras med identiteter snarare än enbart IP-adresser.

Autentisering kan vara integrerad med organisationens identitetsplattform och auktorisation kan genomföras i backend. Redan här behöver ansvar delas mellan identitetsfunktion, applikation och säkerhetsorganisation. Ett traditionellt system har alltså redan tvärfunktionella beroenden; mikrotjänster skapar inte behovet av samarbete från ingenting.

Att driftsätta en förändring

Anta att verksamheten vill införa en ny regel: ett ärende får bara avslutas när vissa dokument har granskats. Utvecklare ändrar affärslogiken, kanske databasens schema, och anpassar användargränssnittet. Testarna verifierar att ärendet inte kan avslutas i fel tillstånd. En release förbereds och driftansvariga säkerställer att version, databasförändring och instruktioner passar ihop.

Vinsten med samordningen är att den förväntade systemversionen kan definieras som en helhet. Nackdelen är att även en begränsad ändring ibland måste passera samma bygg-, test- och driftsättningsflöde som större förändringar. Om många team vill leverera i olika takt blir den gemensamma leveransrytmen ett organisatoriskt beroende.

Det går att minska detta med modulär design och god automatisering, utan att omedelbart införa mikrotjänster. Frågan är hur långt behovet av självständighet faktiskt sträcker sig.

Drift när något går fel

Föreställ dig att användarna inte längre kan öppna ärenden. Driftövervakningen visar kanske hög svarstid i backend och ökad belastning på databasen. En driftteknikер kontrollerar CPU, minne, nätverksförbindelser och databashälsa. En applikationsutvecklare söker i applikationsloggar och jämför med den senaste releasen. DBA granskar frågor och låsningar. Tillsammans avgränsar man felet.

Här finns en viktig styrka: antalet steg mellan användarens begäran och databasen kan vara relativt få. Men inte heller i en traditionell applikation räcker serverstatus som bevis på att verksamhetsfunktionen fungerar. Servern kan vara igång medan ett affärsflöde är trasigt. Det är därför missvisande att framställa verksamhetsnära övervakning som något som först uppfanns med mikrotjänster.

Skillnaden framöver är att ett fel kan uppstå i en tjänst, bero på en annan, förstärkas av en tredje och ändå upplevas av användaren som ett enda fel i ärendehanteringen. Därför blir spårbarhet och tydlig incidentledning ännu viktigare.

Förvaltning över lång tid

Ett IT-stöd är mer än sin nästa release. Operativsystem ska uppdateras, databasmotorn behöver underhållas, certifikat löper ut och beroenden kan bli osäkra. Kapacitet måste planeras, dokumentation hållas aktuell, backup provas och återställningsrutiner fungera även när ordinarie personal inte är tillgänglig.

I en sammanhållen applikation kan mycket av detta planeras runt systemets gemensamma livscykel. Driftfönster, uppgraderingsprojekt och förvaltningsbudget knyts till ett tydligt objekt. Det gör kostnaderna synliga men kan också innebära att hela systemet måste samordnas för en uppgradering som bara berör en del.

Den löpande förvaltningen visar därmed samma dubbla egenskap som arkitekturen: central samordning gör vissa saker enklare och andra långsammare.

Det som är värt att behålla

När organisationer moderniserar IT-stöd finns en risk att det invanda beskrivs som felaktigt. Det vore en olycklig utgångspunkt. Tydlig systemdokumentation, kontrollerade förändringar, ansvarig drift, etablerade säkerhetskrav och prövad återställningsförmåga är värdefulla oavsett arkitektur. Mikrotjänster förändrar hur detta genomförs, inte varför det behövs.

Tre styrkor hos en välbyggd monolit förtjänar att lyftas fram. Den kan ha lägre komplexitet i drift eftersom färre självständiga komponenter måste övervakas. Den kan underlätta konsekvent datahantering inom samma transaktionsgräns. Och den kan ge enklare felsökning när ett användarflöde huvudsakligen sker inom en process. Dessa fördelar kan väga tungt i verksamheter där förändringstakten är måttlig eller teamen få.

Det finns samtidigt situationer där en gemensam livscykel blir begränsande: när delområden behöver förändras oberoende, skala på olika sätt eller ha särskilda krav på tillgänglighet och säkerhet. Då kan en uppdelning i självständiga tjänster vara motiverad. Men de nya friheterna kommer med nya beroenden.

Frågan vi tar med oss

Vårt ärendehanteringsstöd fungerar. Handläggaren kan registrera, bedöma och avsluta ärenden, och organisationen vet i stora drag hur systemet ska förvaltas. Ändå har verksamheten börjat kräva snabbare utveckling av dokumenthantering och notifieringar än resten av systemet kan leverera.

Vad händer om dokumentfunktionen bryts ut till en egen tjänst? Då uppstår frågor som tidigare kunde hanteras inom samma applikation: Hur kommunicerar ärendedelen med dokumentdelen? Vem äger dokumentinformationen? Var körs den nya tjänsten? Hur upptäcks ett avbrott? Vem ansvarar när den samlade ärendehanteringen inte fungerar?

Nästa kapitel börjar med just den förändringen. Poängen är inte att den gamla arkitekturen var fel, utan att en ny arkitektur ändrar vad organisationen måste kunna kontrollera och samordna. R1

Källor och vidare läsning

- **R1** Martin Fowler, *Microservice Trade-Offs* (2015).

2. Vad händer när systemet delas upp?

Det är måndag morgon. En handläggare öppnar ett ärende och väljer ett dokument. Tidigare hämtade samma backend både ärendeuppgifter och dokumentinformation ur den gemensamma databasen. Nu har dokumenthanteringen fått en egen tjänst. På skärmen ser arbetet likadant ut. Under ytan har en ny gräns uppstått: ärendehanteringen måste be en annan tjänst om hjälp.

Den skillnaden kan verka liten. Kanske är den nya dokumenttjänsten utvecklad i samma programmeringsspråk som den gamla backendmodulen. Kanske körs båda fortfarande på VM:ar i organisationens datacenter. Ändå förändras flera förutsättningar samtidigt. Två delar kan uppdateras vid olika tidpunkter. Ett anrop kan misslyckas trots att båda programmen var för sig fungerar. Och ett fel som användaren uppfattar som ett enda systemfel kan kräva insatser från flera team.

Detta kapitel handlar om vad uppdelningen *innebär*. Senare kapitel behandlar hur vi väljer tjänstegränser, utformar nätverket, fördelar data och organiserar driften i detalj.

En uppdelning som börjar med ett verksamhetsbehov

Vårt fiktiva ärendehanteringssystem har vuxit. Dokumenthanteringens förändringstakt skiljer sig från resten av systemet. Verksamheten vill lägga till nya dokumentformat, förbättra sökningen och utveckla externa gränssnitt utan att varje gång behöva samordna en release av hela

ärendehanteringen.

Det finns flera möjliga lösningar. Dokumentdelen kan förbättras som en tydligare modul inuti monoliten, eller så kan den skiljas ut som en egen distribuerad tjänst. Vi antar här att organisationen väljer det senare och att dokumenttjänsten får ett separat leveransflöde och en uttalad ägare. Detta är ett *fiktivt arkitekturbeslut*, inte ett påstående om att uppdelningen alltid är bättre.

Förut kunde backend anropa en intern funktion, exempelvis ”hämta dokumentmetadata”. Ett sådant anrop sker i regel inom samma körande program. Efter uppdelningen behöver ärendedelen begära motsvarande information via ett definierat gränssnitt. Förbindelsen kan gå över ett nätverk och beror på att dokumenttjänsten är nåbar, har kapacitet och förstår anropets version.

Den nya gränsen syns i arkitekturen, men även i organisationens vardag. Någon måste avgöra hur länge ärendedelen ska vänta på svar, vad användaren ska få se vid ett avbrott och vem som ska larmas när dokumenttjänsten inte kan leverera. Det är inte längre möjligt att betrakta dokumenthantering enbart som en intern kodstruktur.

Från gemensam livscykel till flera livscykler

Ett sammanhållet system kan utvecklas modulärt, ha automatiserade tester och skalas med flera instanser. Det som särskiljer vårt nya exempel är inte att det plötsligt blir modernt, utan att dokumenttjänsten har en egen kör- och leveranslivscykel. Den kan i princip byggas, driftsättas och förändras utan att ärendedelen ändras, förutsatt att deras kontrakt förblir kompatibla.

Det ger en möjlig fördel: dokumentteamet kan rätta ett isolerat fel utan att driftsätta resten av ärendehanteringen. Men självständigheten är inte gratis. Om dokumenttjänsten ändrar svarets struktur så att ärendedelen inte längre förstår den, är förändringen i praktiken fortfarande samordnad. Oberoende driftsättning förutsätter alltså gränssnitt som utvecklas med hänsyn till sina användare.

När flera tjänster har skilda versioner finns inte alltid en enda versionsbeteckning som entydigt beskriver hela IT-stödet. Drift, support och felsökning kan behöva veta vilken version av varje berörd tjänst som faktiskt kördes när ett problem uppstod. Leveransinformation och spår-

barhet behöver då knyta samman tjänsteversioner, konfiguration och tidpunkt, snarare än enbart hänvisa till en gemensam systemrelease.

För applikationssidan blir kontrakt, kompatibilitet och beroenden centrala designfrågor. För driftsidan blir det viktigt att kunna se den faktiska kombinationen av tjänster och förstå vilka förändringar som genomförts utan att hela systemet har tagits ur drift.

En ny sorts fel: delar kan fungera samtidigt som helheten inte gör det

Anta att ärendetjänsten fungerar och dokumenttjänstens process är igång, men att nätverksförbindelsen mellan dem bryts. Båda kan svara på var sin enkel kontroll av att processen lever. En sådan kontroll visar inte nödvändigtvis om tjänsten kan utföra hela sin uppgift eller nå sina beroenden. Ändå kan handläggaren inte öppna sina dokument. Det är ett typiskt exempel på ett *partiellt fel*: en del av den distribuerade helheten fungerar inte, medan andra delar fortfarande gör det.

Andra fel kan vara mer svårtolkade. Dokumenttjänsten kanske svarar, men långsamt. Ärendedelen väntar och förbrukar resurser. Om många handläggare samtidigt försöker öppna dokument kan väntande anrop tränga undan annan verksamhetsfunktion. Ett begränsat problem i en tjänst riskerar därmed att sprida sig till andra delar av systemet.

En sådan felkedja är inte oundviklig. Men den behöver motverkas med medveten utformning: rimliga tidsgränser för anrop, begränsad resursförbrukning, försiktigt utformade återförsök (som annars kan förvärra överbelastning) och tydliga beslut om vad som kan fortsätta fungera när dokumenttjänsten är otillgänglig. Dessa tekniker utforskas närmare i kapitel 12.

Det intressanta för verksamheten är oftast inte vilken process som har fallerat. Frågan är om ett ärende går att handlägga. Därför behöver tekniska hälsomätningar kompletteras med förståelse för hela användarflödet. Övervakningen av enskilda komponenter och övervakningen av den samlade tjänsten besvarar olika frågor.

Fler körande delar betyder fler saker att hålla reda på

En distribuerad arkitektur kan fortfarande drivas på traditionella VM:ar. En mikrotjänst behöver inte ha en egen VM, en egen fysisk server eller ens en egen databasserver. Samma VM kan köra flera tjänster, och en containerplattform kan placera flera instanser på gemensam infrastruktur. Valet påverkar isolering, kostnad och driftsättningssätt, men avgör inte ensamt om arkitekturen är en mikrotjänstarkitektur.

Det som förändras i vårt fall är antalet självständigt hanterade tjänster. Varje tjänst behöver en definierad väg till produktion, konfiguration, övervakning och rutiner för underhåll. Om den kan köras i flera instanser måste man kunna skilja på den logiska tjänsten och de faktiska instanser som för tillfället utför arbetet.

En traditionell inventering av tre VM:ar kan därför vara korrekt som infrastrukturförteckning men otillräcklig som karta över verksamhetsstödets beroenden. Vi behöver också veta vilken tjänst som talar med vilken, vilka gränssnitt de använder och hur beroendena påverkar användarnas arbetsflöden. Det betyder inte att traditionell konfigurationshantering försvinner. Den behöver kompletteras med tjänste- och beroendeinformation, i vissa miljöer automatiskt insamlad.

Här blir samspelet mellan utveckling och drift konkret. Applikationsteamet kan beskriva tjänstens krav på resurser, hälsa och kommunikation. Ett drift- eller plattformsteam kan erbjuda en säker och automatiserad exekveringsmiljö. Vilken grupp som konfigurerar en viss resurs beror på organisationens valda arbetsmodell, inte på ordet mikrotjänst.

Ett beroende blir ett avtal mellan parter

När dokumenthanteringen låg i samma applikation kunde utvecklare i många fall ändra både anropande och anropad kod i samma ändring. Efter uppdelningen behöver de förhålla sig till ett kontrakt. Det måste vara begripligt för andra team, kunna provas och förändras utan onödiga störningar.

Kontraktet handlar inte bara om vilka fält ett API skickar tillbaka. Det omfattar även förväntat beteende: vad händer om ett dokument saknas, om användaren saknar behörighet eller om tjänsten är tillfälligt överbelastad? Vilka svar ska konsumenten kunna hantera? Och hur länge stöds

en äldre variant av gränssnittet?

Dessutom finns ett driftskontrakt i praktisk mening. Vilken tillgänglighet förväntas? Hur snabbt måste fel upptäckas? Vem får förändra de nätverksförbindelser som krävs? Vem kan kontaktas vid problem? Inte allt behöver vara ett formellt avtal. Men utan gemensamma förväntningar uppstår lätt ett glapp mellan teknikens faktiska beteende och verksamhetens krav.

Självständighet behöver avvägas mot samordning

Det kan vara lockande att anta att fler tjänster automatiskt leder till snabbare leveranser. Så behöver det inte bli. Om varje ny verksamhetsfunktion kräver samtidiga ändringar i fem tjänster, och inget team kan testa dem utan de andra, har uppdelningen inte skapat särskilt mycket självständighet. Organisationen kan då ha fått kostnaden för distribution utan motsvarande nytta.

Ett viktigt mått är därför inte antalet tjänster utan hur väl deras gränser stöder oberoende förändring. Vi behöver även räkna in kostnaden för nätverk, identifiering, övervakning, driftkompetens och incidenthantering. En modulär monolit kan i vissa situationer vara ett bättre svar på samma verksamhetsbehov.

Det betyder inte att varje tjänst måste vara fullständigt oberoende. Gemensamma krav på identitet, säkerhet, loggformat och leveransplattform kan ge bättre förutsättningar för självständighet, eftersom varje team slipper uppfinna samma lösningar. Skillnaden är mellan samordning som möjliggör självständighet och koppling som tvingar fram gemensamma ändringar.

Ansvar följer inte automatiskt med när koden flyttas

I det gamla systemet kunde en förvaltningsansvarig samordna ändringar i dokumentdelen och driftorganisationen hantera den gemensamma miljön. När dokumentdelen blir en egen tjänst behöver det uttalas vem som ansvarar för dess funktion, säkerhetskrav, leveranser och driftbarhet. Men det betyder inte att ett utvecklingsteam automatiskt tar över

brandväggar, databasserver, backup eller beredskap.

Ansvar behöver delas upp efter vad som faktiskt ska göras. Verksamhetsansvariga anger vilka funktioner och kvaliteter som krävs. Applikationsansvariga utformar tjänstens beteende och gränssnitt. Infrastruktur- och plattformsansvariga tillhandahåller överenskomna förmågor för körning, nätverk och drift. Säkerhetsfunktioner sätter och följer upp relevanta kontrollkrav. Organisationen måste dessutom utse ett tydligt ansvar för att den *samlade* ärendehanteringen fungerar.

Tänk på ett avbrott då handläggaren inte kan öppna ett dokument. Det räcker inte att ärendeteamet säger att dess tjänst fungerar och dokumentteamet säger att dess process är igång. Någon behöver leda felsökningen över gränserna och ta ansvar för användarpåverkan. Vem som gör vad fördjupas i kapitel 3 och senare i kapitlen om övervakning och incidenthantering.

Två perspektiv på samma förändring

För den som kommer från applikationsutveckling innebär uppdelningen ett större ansvar att beskriva beroenden och utforma hur tjänsten beter sig när omvärlden inte svarar som väntat. Ett internt metodanrop har blivit en kommunikationsrelation som måste fungera under verkliga driftsförhållanden. Det ger möjlighet till självständigare leveranser men ställer också krav på kompatibilitet, tydliga gränssnitt och en överenskommen hantering av behörighetsbeslut över tjänstegränser.

För den som kommer från drift och infrastruktur är själva verksamhetsstödet fortfarande en helhet, men dess tekniska delar kan förändras oberoende och ha olika egenskaper. Driftsättningsobjekt, larm och samband blir fler. Samtidigt kan standardiserad plattformsteknik och automatisering minska det manuella arbetet per tjänst – om ansvar och verktyg är genomtänkta.

Båda perspektiven möts i samma fråga: Vilka delar måste vara självständiga, vilka gemensamma funktioner behövs och hur ser vi till att ansvar inte faller mellan teamen?

Vad förändras för vem?

Fråga	Applikationsperspektiv	Drift- och infrastrukturperspektiv	Gemensam utmaning
Förändringar	Hantera kontrakt och kompatibilitet	Se faktisk version och konfiguration i drift	Leverera självständigt utan att bryta helheten
Fel	Hantera väntan, avbrott och degraderat beteende	Knyta tekniska signaler till tjänsteberoenden	Förstå användarpåverkan vid partiella fel
Miljöer	Beskriva resurs- och kommunikationsbehov	Erbjuda säker placering, åtkomst och övervakning	Skapa fungerande gränssnitt för självbetjäning och kontroll
Ansvar	Äga överenskommet tjänstebeteende	Äga överenskomna drifts- och plattformsförmågor	Utse ansvar för verksamhetsstödets funktion

När vi fortsätter

Utbrytningen av dokumenthanteringen har synliggjort något som förut låg inuti en sammanhållen applikation: beroenden, förändringar och fel passerar nu gränser mellan självständiga tekniska delar. Därför behöver ansvarsfördelningen bli tydligare, inte otydligare.

Nästa kapitel undersöker vem som faktiskt äger olika delar av den distribuerade lösningen – och varför ansvaret för användarens fungerande ärendehantering inte kan delas upp på samma sätt som koden. R2 R1

Källor och vidare läsning

- **R2** James Lewis och Martin Fowler, *Microservices* (2014).
- **R1** Martin Fowler, *Microservice Trade-Offs* (2015).

3. Vem ansvarar för vad när systemet blir distribuerat?

En användare försöker öppna ett dokument i ett ärende. Ärendet visas, men dokumentet går inte att hämta. I går fungerade allt. Användaren anmäler ett fel i ärendehanteringssystemet. För verksamheten finns fortfarande ett IT-stöd som ska fungera. För den tekniska organisationen kan felet nu finnas i ärendetjänsten, dokumenttjänsten, en nätverkspolicy, ett identitetssystem eller en gemensam plattform.

Det som tidigare kunde hanteras som ett fel inom en sammanhållen applikation har blivit en fråga om samverkan mellan flera självständiga delar. Men vem ska egentligen lösa problemet? Det korta svaret är att ansvaret behöver beskrivas på flera nivåer. Ingen av dessa nivåer kan ensam ersätta helhetsansvaret.

Helheten finns kvar även när applikationen delas upp

I vårt fiktiva ärendehanteringssystem fanns från början en frontend, en backend och en gemensam databas. Driftorganisationen förvaltade exempelvis VM:ar, nätverk och databasmiljöer, medan applikationsgruppen utvecklade och förändrade funktionerna. Systemägaren kunde begära åtgärder för hela IT-stödet och leveranserna samordnades vanligtvis inom en gemensam release.

Organisationen behövde redan då samarbeta över kompetensgränserna. En monolit innebär inte att alla beslut fattas av en person, och en central driftfunktion har aldrig ensam garanterat att en applikation fungerar. Men när många funktioner ingår i samma körbara applikation och samma release finns ofta färre tekniska gränssnitt mellan de delar som ska ändras och felsökas.

När dokumenthanteringen bryts ut uppstår två tekniklivscykler. Dokumenttjänsten kan uppdateras utan att ärendetjänsten uppdateras. Den kan ha egna data, driftsättningsrutiner, resurskrav och övervakningssignaler. Detta kan minska behovet av gemensamma releaser. Samtidigt måste tjänsterna fortfarande samverka för att användaren ska kunna arbeta.

En teknisk ansvarsfördelning kan därför bli mer detaljerad utan att ansvaret för verksamhetsresultatet får bli otydligare. Även om tjänsterna förvaltas av olika team behöver en utsedd funktion kunna samordna prioriteringar och återställning för IT-stödet som helhet.

Fyra nivåer av ansvar

Ett användbart första steg är att skilja på fyra ansvarsnivåer. De kan organiseras på olika sätt och behöver inte motsvara fyra separata team. Ansvarsnivåerna är inte en rangordning: verksamhetsansvar, tekniskt genomförande och oberoende kontroll kan ligga i olika organisatoriska delar.

Ansvar för verksamhetens IT-stöd

Någon behöver företräda verksamhetens krav på den samlade funktionen: att ärenden kan registreras, dokument öppnas, beslut fattas och information bevaras enligt tillämpliga krav. Den rollen behöver kunna prioritera risker och åtgärder även när problemen sträcker sig över flera tjänster.

Helhetsansvaret omfattar inte nödvändigtvis drift av varje komponent. Det innebär däremot att ingen användare ska behöva förstå mikrotjänstgränserna för att få hjälp eller veta var ett verksamhetskritiskt fel ska anmälas.

Ansvar för en mikrotjänst

Ett tjänsteteam kan få ansvar för dokumenttjänstens funktion och livscykel: kod, gränssnitt, datamodell, beroenden, releaser, driftbarhet och felrättning. För att ägarskapet ska fungera behöver teamet känna till tjänstens prestandakrav, åtkomstbehov och hur den påverkar andra tjänster.

Det betyder inte nödvändigtvis att utvecklarna ska administrera servrar, databasmotorer eller brandväggar, eller ens att tjänsteteamet bemannar jour dygnet runt. Sådant måste avgöras genom en uttrycklig drift- och beredskapsmodell. Men de kan inte betrakta sådana frågor som helt frikopplade från tjänstens funktion. Om tjänsten måste kunna anropa ett dokumentlager måste behovet beskrivas så att infrastrukturen kan tillhandahålla och verifiera kommunikationen.

Ansvar för plattform och gemensam drift

Plattforms- och driftfunktioner kan tillhandahålla exekveringsmiljöer, nätverk, databastjänster, certifikathantering, loggplattformar, backupmekanismer och automatiserade leveransvägar. Plattformen bör erbjuda tydliga tjänster med dokumenterade gränssnitt och förutsättningar, snarare än enbart en lista över maskiner som någon annan får använda.

Även här finns flera möjliga modeller. En organisation kan ha en central driftavdelning som också är plattformsteam, eller en separat plattformsfunktion som samverkar med nätverks-, databas- och säkerhetsspecialister. Det väsentliga är att ansvar och eskalering framgår.

Ansvar för gemensamma säkerhets- och informationskrav

Säkerhet, regelefterlevnad, informationsklassning och identitetshantering korsar tjänstegränserna. En central funktion kan sätta krav på exempelvis nätverkssegmentering och autentisering, medan plattformen genomdriver tekniska kontroller och tjänsteteamet implementerar tillämpningsspecifika behörighetsregler. Ingen bör anta att ett godkänt nätverksflöde automatiskt innebär att en användare har rätt att läsa ett dokument.

Fem frågor som avslöjar otydligt ansvar

Det är lätt att säga att ett team ”äger” en tjänst. Svårare är att veta vad ägarskapet innebär när något ska ändras eller slutar fungera. För varje viktig förmåga behöver man kunna besvara fem olika frågor:

1. **Vem ställer kraven?** Exempelvis vem avgör tillgänglighetsnivå och skyddsbehov för dokumenthanteringen?
2. **Vem fattar beslut?** Vem har mandat att godkänna en ny integrationsväg eller en ändring som påverkar andra tjänster?
3. **Vem genomför förändringen?** Vem ändrar kod, konfiguration, brandväggsregel eller databasinställning?
4. **Vem driver och återställer?** Vem reagerar på larm, leder åtgärder och återställer funktionen utanför kontorstid?
5. **Vem verifierar och följer upp?** Vem kontrollerar att kraven faktiskt uppfylls efter ändringen?

I praktiken behövs även en tydlig väg för eskalering när flera team berörs eller ingen uppenbart äger orsaken till ett fel. Under en incident bör det vara klart vem som leder arbetet och vem som beslutar om riskfyllda tillfälliga åtgärder.

Samma grupp kan besvara flera frågor, men inte alltid alla. Att skilja dem åt hjälper oss att undvika två vanliga misstag: att ansvaret blir oklart mellan teamen, eller att ett team förväntas bära ansvar utan att ha tillräckliga mandat och verktyg.

Fallet: Dokumenttjänsten kan inte nås

Anta att dokumenttjänsten har fått en ny adress efter en driftsättning och att trafikregler eller tjänsteupptäckt inte har uppdaterats korrekt. Ärendetjänstens anrop misslyckas. I loggarna syns timeout, medan nätverksplattformen visar nekad trafik. Det är ett möjligt felscenario, inte en generell följd av att en tjänst byter adress. Användaren ser bara att dokumentet inte öppnas.

Den som tar emot användarens felanmälan bör kunna skapa ett sammanhållet ärende och inleda felsökning. Applikationsteamet behöver kunna avgöra vilket anrop som misslyckas och vilka krav som finns på destination och behörighet. Plattformen eller nätverksfunktionen behöver kunna visa hur trafiken dirigeras och vilka regler som gäller. Om en regel behöver ändras måste någon ha mandat att beställa, granska och genom-

föra ändringen. Till sist behöver någon verifiera att hela användarflödet fungerar igen – inte bara att båda tjänsternas processer är igång.

Om övervakningen endast säger att ärendetjänsten och dokumenttjänsten var för sig är ”gröna” kan felet ändå ligga kvar. Därför behöver ansvar för verksamhetsflöden och tjänstebaserad observerbarhet mötas. Senare kapitel fördjupar de tekniska metoderna för detta.

Fråga i händelsen	Exempel på primärt ansvar	Samverkan som krävs
Upptäcka att dokument inte går att öppna	Helhetsövervakning eller support	Tjänsteteam och drift
Identifiera misslyckade anrop	Berörda tjänsteteam	Gemensam logg- och spårningsplattform
Granska trafikregler	Nätverks-/plattformsteam	Tjänsteteam och säkerhet
Bedöma konsekvensen för verksamheten	Ansvarig för IT-stödet	Incidentledning och verksamhet
Verifiera återställd funktion	Utsedd incident- och tjänsteansvarig	Alla berörda team

Tabellen är ett exempel, inte en generell organisationsmall. I en annan organisation kan vissa av uppgifterna utföras av samma team eller en extern leverantör.

Ansvar som förflyttas – och ansvar som delas

Med automatisering kan en del arbetsmoment flyttas närmare tjänsteteamet. Ett team kan exempelvis själv beställa en testmiljö genom ett godkänt gränssnitt, konfigurera resursgränser eller deklarera vilka tjänster det behöver kommunicera med. Plattformen kan verkställa önskemålen inom centralt beslutade ramar.

Det betyder inte att ansvaret för säkerhet och drift alltid ”flyttar till utvecklarna”. Ofta förändras snarare fördelningen mellan att **ange ett behov**, **tillhandahålla en gemensam förmåga** och **säkerställa att behovet uppfylls**. Automatisering ändrar hur arbetet utförs, men den tar inte bort behovet av ägarskap eller kontroll.

Samma resonemang gäller databaser. Ett tjänsteteam kan ansvara för informationsmodell och schemaändringar medan DBA- eller plattformsteam ansvarar för databasmotor, prestandakapacitet, backupmekanismer och grundläggande säkerhetskonfiguration. Återställning måste sedan provas mot tjänstens faktiska beteende. Ett lyckat databasjobb är inte automatiskt en återställd verksamhetsfunktion.

Två perspektiv på samma gränssnitt

För läsaren från applikationssidan blir den viktigaste förändringen att ansvaret för en funktion inte slutar där koden slutar. Hur tjänsten kommunicerar, kan övervakas och kan återhämta sig är också en del av dess utformning. Det innebär ett tätare samarbete med infrastrukturen, men inte nödvändigtvis att varje utvecklare behöver vara specialist på nätverk eller operativsystem.

För läsaren från driftsidan blir den viktiga förändringen att driftens objekt kan bli fler, mer föränderliga och mer beroende av applikationens logik. Man behöver förstå tjänsternas kritiska relationer, inte bara deras VM:ar och databaser. Samtidigt kan välbyggda plattformsförmågor ge större kontroll genom gemensamma policyer, standardiserade driftsättningar och bättre spårbarhet.

Båda sidor behöver ett gemensamt språk för beroenden, servicekrav, incidenter och förändringar. En överenskommelse om vem som gör vad är användbar först när den också omfattar mandat, tillgång till relevant information och praktiska vägar till hjälp.

Helhetsansvar får inte bli ett tomrum

Ett distribuerat IT-stöd kan ha tydliga ägare för samtliga mikrotjänster och ändå sakna någon som kan prioritera ett fel i den samlade användarupplevelsen. Det kan också ha stark central styrning men så många godkännandeled att varje liten förändring tar onödigt lång tid. Oberoende leveranser förutsätter därför inte att all styrning avskaffas, utan att beslut kan tas inom tydliga mandat och verifierbara ramar. Båda ytterligheterna är problematiska.

En fungerande ansvarsfördelning behöver därför förena lokal handlingsförmåga med ett uttryckligt ansvar för helheten. Tjänsteteam ska kunna fatta beslut inom sina gränser. Gemensamma funktioner ska kunna

ange och genomdriva de ramar som behövs. Verksamheten ska fortfarande kunna kräva en fungerande tjänst, oavsett hur många tekniska delar som samverkar.

Här handlar ansvar främst om **vem som gör vad när ett distribuerat IT-stöd byggs, ändras eller får problem**. I kapitel 18 flyttar vi perspektivet till **hur organisationen håller ansvar och mandat fungerande över flera år** – genom styrning, finansiering, uppföljning och överlämningar. Först behöver vi förstå hur valet av tjänstegränser faktiskt påverkar både applikation och drift.

Vad förändras för vem?

För applikationssidan: Tjänsteansvar omfattar också driftbarhet, integrationskontrakt, beroenden och möjligheten att felsöka ett användarflöde.

För driftsidan: Det blir viktigare att tillhandahålla begripliga plattformsförmågor och insyn i dynamiska beroenden än att enbart administrera en fast uppsättning maskiner.

För helheten: Någon måste fortfarande kunna prioritera, samordna och följa upp verksamhetsfunktioner som spänner över flera tjänster. Ansvar delas upp, men får inte lämnas odefinierat. R2

Källor och vidare läsning

- **R2** James Lewis och Martin Fowler, *Microservices* (2014).

4. Hur ska systemet delas upp?

När ett välbekant IT-stöd ska delas upp är det frestande att börja med det som går att se: en backend, några databastabeller eller en lista med befintliga moduler. Men det svåraste beslutet handlar inte om hur koden ska flyttas. Det handlar om **var gränserna ska gå för självständiga förändringar, tydligt ansvar och rimliga beroenden**.

Det valet påverkar både den som utvecklar funktionerna och den som ska hålla dem tillgängliga. För varje ny tjänst uppstår ett eget logiskt ansvar för version, driftsättning, observerbarhet, skydd och återhämtning. Gemensamma plattformar kan automatisera mycket av arbetet, men de tar inte bort behovet av en identifierbar tjänsteägare. Uppdelningen måste därför motiveras av en verklig nytta som överstiger kostnaden för fler gränser.

Börja med verksamhetens behov, inte med tekniken

Tänk på det gemensamma ärendehanteringssystemet i tidigare kapitel. En handläggare registrerar ett ärende, bifogar handlingar, fattar beslut och skickar besked till en part. I monoliten kan allt detta genomföras av funktioner i samma backend mot en gemensam databas. Det innebär inte att funktionerna redan utgör lämpliga mikrotjänster.

Ett bättre första steg är att fråga vilken verksamhetsförmåga varje del representerar, vilka regler den ansvarar för och vilken information den behöver kunna förvalta. Ärendehantering och dokumenthantering kan

till exempel ha olika regler för förändring, åtkomst, volym och lagring. Det talar för en möjlig gräns. Men om dokumenten endast är bilagor utan egen livscykel eller egna regler kan en separat dokumenttjänst innebära mer arbete än nytta.

En verksamhetsförmåga är inte automatiskt lika med en mikrotjänst. Den ger en utgångspunkt för analysen, inte en formel för antalet tjänster. Verksamhetsprocesser kan dessutom korsa flera sådana förmågor. Att handläggaren upplever ett sammanhängande arbetsflöde betyder alltså inte att alla aktiviteter behöver ligga i samma tjänst – men inte heller att varje aktivitet bör brytas ut.

Gränser som följer regler och ansvar

En användbar tjänstegräns håller samman sådant som behöver förändras tillsammans. Om flera funktioner delar samma verksamhetsregler, ändras av samma skäl och måste upprätthålla gemensamma invarianta villkor finns det ett starkt argument för att hålla dem nära varandra.

En *invariant* är ett villkor som alltid måste vara sant, till exempel att ett ärende inte får markeras som slutligt avgjort utan ett giltigt beslutsunderlag. Om underlaget kommer från en annan tjänst behöver man avgöra om regeln måste kontrolleras synkront eller kan säkras genom en verksamhetsprocess med definierade mellanlägen. Om ett sådant villkor kräver tätt samordnade ändringar i flera tjänster kan gränsen vara olämplig – eller så måste processen uttryckligen utformas för att hantera flera steg och mellanlägen. Detta är ett arkitekturval, inte bara en implementationsteknik.

Domändriven design erbjuder användbara begrepp som avgränsade kontexter (*bounded contexts*). En avgränsad kontext är ett område där begrepp och regler har en konsekvent betydelse. Den kan hjälpa till att identifiera tjänstegränser, men en kontext kan realiseras av en eller flera tjänster; den behöver inte vara ett driftsatt program i sig.

Tjänstegränsen behöver också ha en rimlig koppling till ett team eller en tydlig ägare. En funktion som ständigt kräver samtidiga ändringar från flera ansvariga team blir inte särskilt självständig bara för att den har ett eget API.

Tre tänkbara uppdelningar av dokumenthanteringen

Anta att verksamheten vill kunna vidareutveckla dokumentstödet oberoende av ärendehanteringen. Det finns minst tre rimliga lösningar:

Alternativ A: Behåll dokumenthanteringen som modul i monoliten. Ärende- och dokumentfunktionerna ligger kvar i samma driftsatta applikation men får tydligare interna gränssnitt och ägarskap. En förändring behöver inte passera ett nätverk och transaktioner kan i vissa fall hållas lokala. Däremot är leveranscykeln fortfarande gemensam om applikationen driftsätts som en enhet. Detta är ofta en bra start när oberoende driftsättning ännu inte ger tillräckligt värde.

Alternativ B: Bryt ut en sammanhållen dokumenttjänst. En tjänst får ansvar för dokumentmetadata, dokumentregler och ett väldefinierat kontrakt mot ärendehanteringen. Själva filinnehållet kan lagras i en gemensamt tillhandahållen objektlagring utan att dokumenttjänsten förlorar sitt informationsansvar. Den kan utvecklas och driftsättas separat, förutsatt att kontrakt och dataägarskap tillåter det. Men en ny nätverksgräns införs, liksom nya krav på hantering av otillgänglighet, loggning och behörighet.

Alternativ C: Dela upp dokumentområdet ytterligare. Lagring, metadata, konvertering och sökning blir separata tjänster. Det kan vara rimligt om delarna har tydligt skilda livscykler, belastningsmönster och ägare. Delade tekniska stödfunktioner som sökindex eller konvertering behöver dock inte automatiskt vara separata mikrotjänster med egna produktteam. Men för ett mindre ärendehanteringsstöd kan det skapa fyra driftobjekt och flera nya beroenden utan motsvarande verksamhetsnytta.

Jämförelsen måste även omfatta säkerhetsgränser, kostnader för jour och felsökning samt vilka verksamhetsflöden som försämras om dokumentstödet är otillgängligt. Dessa faktorer väger ibland tyngre än möjligheten till separat driftsättning.

Det avgörande är inte att välja den minsta möjliga tjänsten. Det är att välja **den minsta ändamålsenliga uppdelningen** för den aktuella organisationens behov.

Hur beroenden avslöjar dåliga gränser

Ett första test är att föreställa sig en vanlig förändring. Verksamheten vill exempelvis införa en ny dokumentkategori med särskilda gallringsregler. Om ändringen kräver samtidiga kod- och databasändringar i ärendetjänsten, dokumenttjänsten och ytterligare en metadatatjänst är det värt att undersöka om ansvaret verkligen har lagts på rätt plats.

Samtidiga ändringar är inte alltid fel. Vissa verksamhetsförändringar berör flera områden och behöver samordnas. Varningssignalen är **återkommande och ofrånkomlig samordning även för små lokala förändringar**.

Ett andra test är att analysera samtalsmönstret mellan tjänsterna. Om en enda användaroperation kräver en lång kedja av synkrona anrop ökar beroendet av att många delar fungerar samtidigt. Om tjänster dessutom frågar direkt efter varandras interna datamodeller är gränserna sannolikt för svaga. Valet mellan synkron kommunikation, händelser och köer fördjupas i kapitel 6; här räcker slutsatsen att **kommunikationsbehovet måste påverka gränsdragningen**.

Ett tredje test handlar om data: vem har rätt att ändra informationen och definiera dess innebörd? En separat tjänst som förutsätter direkt skrivåtkomst till en annan tjänsts interna tabeller blir starkt kopplad till dess förändringar. Hur dataägarskap och konsistens kan lösas förklaras i kapitel 7.

För driftorganisationen är en tjänstegräns också en driftgräns

Applikationsarkitekten ser ofta tjänstegränsen som en möjlighet att isolera förändringar. Driftorganisationen ser också ett nytt objekt att ta hand om: en komponent med egna versioner, konfigurationer, mätvärden, incidenter, säkerhetsbehov och beroenden.

Det betyder inte att varje mikrotjänst behöver en egen VM eller ett eget driftteam. Flera tjänster kan dela körplattform, automatiserad leveranskedja och övervakningsinfrastruktur. Skillnaden mellan logisk tjänst och exekveringsmiljö behandlas i nästa kapitel. Men även med en gemensam plattform måste det gå att veta **vilken tjänst som påverkar vilken verksamhetsfunktion**, vad den beror på och vart en incident ska eskaleras.

Anta att den nya dokumenttjänsten slutar svara. Ärendehanteringen kan kanske fortfarande visa befintliga ärenden men inte öppna bilagor. Om verksamheten uppfattar hela systemet som felaktigt räcker det inte att driften visar att ärendetjänstens process körs. Tjänstegränserna måste därför kombineras med en bild av användarflöden och beroenden. Metoderna för övervakning och incidenthantering utvecklas i kapitel 13 och 14.

En alltför fin uppdelning kan också skapa många certifikat, nätverksflöden, konfigurationer och larm. Det är legitimt att driftsidan invänder mot komplexiteten, precis som det är legitimt att applikationssidan vill möjliggöra oberoende utveckling. Dessa perspektiv bör vägas samman innan gränserna fastställs.

Vem fattar beslutet?

Verksamhetsföreträdare kan beskriva vilka förmågor som behöver utvecklas oberoende och vilka krav som är kritiska. Applikationsarkitekter och utvecklingsteam analyserar verksamhetsregler, data, samspel och förändringsmönster. Drift- och plattformsfunktioner bidrar med konsekvenser för kapacitet, isolering, observabilitet, nätverk och driftkostnad. Säkerhetsfunktionen granskar skyddsbehov och tillitsgränser.

Beslutsmandatet behöver vara uttalat. Det kan ligga hos en ansvarig arkitekt, en produktorganisation eller ett gemensamt arkitekturforum, beroende på styrmodellen. Oavsett modell bör berörda parter förstå avvägningarna och någon ha mandat att väga helhetens kostnader mot de enskilda teamens frihet.

Ett konkret beslutsunderlag för dokumenttjänsten behöver inte vara omfattande. Det bör åtminstone dokumentera varför uppdelningen övervägs, vilka funktioner och data som ingår, vilka beroenden som uppstår, vilka driftskrav som följer, vem som äger tjänsten och under vilka förutsättningar beslutet ska omprövas.

När uppdelningen blir en distribuerad monolit

En *distribuerad monolit* är en lösning där komponenter är tekniskt separerade men i praktiken så hårt beroende av varandra att de måste

förändras, driftsättas eller hanteras som en enda enhet. Då får organisationen kostnaderna för distribution – nätverk, felhantering och fler driftobjekt – utan att få den eftersträvade självständigheten.

Symptom kan vara att många tjänster alltid måste släppas samtidigt, att interna databastabeller delas utan tydliga gränser, att varje enkel användarhandling kräver många tätt kopplade anrop eller att ändringar i ett team ofta blockerar flera andra. Inget enskilt symptom är ensamt avgörande, men mönstret bör granskas.

Den motsatta ytterligheten finns också: en tjänst som blivit så stor att många orelaterade förändringar konkurrerar om samma leveranscykel och team. Målet är inte maximal uppdelning utan **balans mellan sammanhållning inom tjänster och rimliga beroenden mellan dem**.

Ett praktiskt beslutsprov

Innan dokumenthanteringen bryts ut kan projektgruppen ställa fem frågor:

1. Vilka verksamhetsregler och vilken information behöver hållas samman?
2. Vilka förändringar vill verksamheten och teamen verkligen kunna göra oberoende?
3. Vilka nya beroenden, felmöjligheter och driftobjekt skapar uppdelningen?
4. Vem tar långsiktigt ansvar för tjänsten och dess driftbarhet?
5. Skulle en tydligare modulär monolit uppfylla behoven med lägre komplexitet?

Beslutet blir mer användbart om man även beskriver hur det ska utvärderas: exempelvis minskade samordningsbehov vid ändringar, acceptabel felpåverkan och hanterbar driftkostnad. Sådana mått är exempel och ska väljas utifrån organisationens mål.

Om frågorna inte går att besvara är gränsen sannolikt för tidigt bestämd. En gradvis utbrytning med väl avgränsad modul och mätbara problem kan då vara klokare än ett stort engångsbeslut.

Vad förändras för vem?

För **applikationssidan** blir tjänstegränsen ett beslut om vilka verksamhetsregler, data och förändringar som ska kunna utvecklas självständigt. Teamet måste samtidigt göra beroenden och felbeteenden begripliga för dem som driver helheten.

För **driftsidan** innebär samma beslut nya logiska driftobjekt, trafikflöden och beroenden att hantera. Det är därför viktigt att bidra till gränsdragningen innan en uppdelning har låsts i kod och leveransstruktur.

Det **gemensamma ansvaret** är att värdera faktisk verksamhetsnytta mot livscykel-, säkerhets- och driftkostnader, och att bestämma vem som får ompröva en olämplig gräns.

Att ta med sig

En välvald tjänstegräns förenar verksamhetens behov av förändring med ett realistiskt tekniskt och organisatoriskt ansvar. För applikationssidan handlar den om tydliga regler, data och leveransoberoende. För driftsidan handlar den om hanterbara beroenden, diagnos, tillgänglighet och långsiktiga driftobjekt. Båda måste bidra för att gränsen ska ge självständighet i praktiken. R3 R1

I nästa kapitel följer den naturliga följdfrågan: när dokumenthanteringen väl har fått en egen logisk gräns, **var ska den då köras?**

Källor och vidare läsning

- **R3** Microsoft Learn, Azure Architecture Center, *Use domain analysis to model microservices* (åtkomst 2026-10-17).
- **R1** Martin Fowler, *Microservice Trade-Offs* (2015).

5. Var ska tjänsterna köras?

När dokumenthanteringen i vårt ärendehanteringssystem bryts ut uppstår en fråga som tidigare ofta hade ett enkelt svar: vilken server eller virtuell maskin ska köra den nya delen? En separat tjänst behöver inte en separat VM. Däremot behöver den en tydligt beskriven körmiljö, resurser och ansvar för att fungera säkert och tillförlitligt.

Den här skillnaden är central. Mikrotjänstarkitektur syftar till att göra avgränsade verksamhetsfunktioner möjliga att utveckla och leverera relativt självständigt. Det är en möjlighet som måste understödjas av tjänstegränser, kontrakt och leveransförmåga, inte en automatisk garanti. Det avgör däremot inte automatiskt om de ska köras på virtuella maskiner, i containrar, på en orkestreringsplattform eller som hanterade tjänster. Valet måste följa behovet, inte tvärtom.

Tjänst, instans och körmiljö är tre olika saker

En *tjänst* är en logisk förmåga med definierade funktioner och kontrakt. En *instans* är en körande kopia av tjänstens program. En *körmiljö* är den tekniska resurs som bär instansen, till exempel en VM, en container på en värd eller en hanterad exekveringstjänst. Samma tjänst kan ha flera instanser; en VM kan samtidigt bära flera tjänsters instanser. En container är en paketerad och isolerad körform, men container, instans och fysisk eller virtuell värd är inte utbytbara begrepp.

I det traditionella ärendehanteringssystemet kan frontend ligga på en VM, backend på två VM:ar bakom en lastbalanserare och databasen på

en särskild databasplattform. VM-indelningen följer då ungefär de tekniska lagren. När dokumenttjänsten bryts ut behöver man i stället fråga vilka krav den har på kapacitet, isolering, tillgänglighet och förvaltning. Antalet mikrotjänster bestämmer inte antalet VM:ar.

Fyra vanliga alternativ

En VM per tjänst. Detta liknar en välbekant driftmodell. Operativsystem, behörigheter och underhåll kan avgränsas tydligt, men tjugo tjänster kan innebära många operativsystem, uppdateringar och övervakningsobjekt. Hög tillgänglighet kan dessutom kräva mer än en VM för samma tjänst.

Flera tjänster på en VM. Det kan vara resurssnålt, särskilt vid liten belastning. Samtidigt delar tjänsterna kapacitet och felorsaker: ett värdproblem eller en tung process kan påverka flera tjänster. Det måste finnas tydliga regler för resursanvändning och ansvar för värden.

Containrar på virtuella eller fysiska värdar. Containrar gör det möjligt att paketera programmet och dess beroenden på ett mer enhetligt sätt. Det kan underlätta driftsättning och automatisering, men det behövs fortfarande värdar, nätverk, säkerhetsuppdateringar och lagring. En container är inte automatiskt samma säkerhetsgräns som en separat VM.

Orkestrerad eller hanterad exekvering. Beroende på plattform kan den placera instanser, starta om dem vid vissa fel och styra resurser. Automatiska omstarter är inte detsamma som att tjänsten eller verksamhetsflödet har återhämtats. Med hanterade tjänster kan den egna organisationen slippa delar av värd- eller klusteradministrationen. Det innebär inte att ansvar för tjänstens funktion, konfiguration, information, kostnad och säkerhet försvinner. Dessutom behöver driftorganisationen förstå vilken funktion plattformen faktiskt garanterar.

En monolit kan köras i containrar. Mikrotjänster kan köras på VM:ar. Teknikvalet i sig säger alltså inte om arkitekturen är bra eller dålig.

Placering och isolering är arkitekturbeslut

Anta att dokumenttjänsten både tar emot handlingar och framställer förhandsvisningar av stora dokument. Förhandsvisningen kan använda

mycket CPU och minne och därmed störa handläggarens uppladdning. Det behöver inte innebära två mikrotjänster, men det kräver kontroll över belastning och resurser.

Placeringen bör bedömas utifrån processor, minne, lagring och nätverksbehov, men också utifrån gemensamma felorsaker. Två instanser på samma värd skyddar inte mot ett värdfel. Två värdar som delar databas, lagringssystem eller nätverk kan fortfarande falla samtidigt. Redundans måste därför bedömas utifrån verkliga felområden och verksamhetens krav på avbrott och dataförlust.

Säkerhetskrav kan också kräva olika grader av fysisk eller logisk separation. Dessa skydd måste bedömas tillsammans med risker för delad värdkärna, administrativa behörigheter och gemensamma plattformskomponenter. Namnrymder och containerskiljelinjer skapar inte automatiskt tillräcklig isolering. Nätverkspolicy, identiteter, behörigheter och ibland separata kluster eller värdar kan behövas. Sådana beslut måste bygga på riskbedömning, inte bara på önskemål om effektiv resursanvändning.

Skalning är mer än fler instanser

Den gamla backendservern kunde få mer minne eller kompletteras med en till server. En separat dokumenttjänst kan på motsvarande sätt få fler instanser utan att ärendetjänsten behöver skalas. Det förutsätter att flaskhalsen faktiskt finns i den del som får fler instanser. Men vinsten uppstår först när tjänsten är byggd för det. Om sessionsinformation bara finns i en instans eller om alla anrop stoppas av en gemensam databas blir det svårt att tillgodogöra sig extra kapacitet.

Automatisk skalning behöver rimliga resurströsklar, marginaler och mätvärden. Skalar man för sent kan köer och svarstider växa; skalar man för aggressivt kan kostnaderna bli höga och systemet instabilt. En driftplattform kan utföra skalningen, men applikationsteamet måste kunna beskriva tjänstens belastningsmönster och hur den reagerar vid överbelastning.

Hälsokontroller, omstarter och tillstånd

Att en process körs betyder inte att den kan ta emot trafik. En tjänst kan exempelvis vara igång men sakna anslutning till ett nödvändigt beroende. Plattformen behöver därför signaler som skiljer mellan att instansen

lever och att den är redo att hantera nya anrop. Samtidigt måste kontroller utformas så att problem i en annan tjänst inte leder till onödiga omstarter i hela kedjan.

Kortlivade instanser gör det särskilt viktigt att inte lagra oersättlig information enbart på deras lokala filsystem. Dokumenttjänstens filer behöver till exempel lagras i en beständig lagringstjänst med eget dataansvar, återställningskrav och åtkomstskydd. Detta gäller även om applikationsteamet inte självt administrerar lagringen. Beständig lagring är dock inte liktydig med backup eller verifierad återställningsförmåga; dessa behandlas senare i boken. R4 R5

Ansvar mellan applikation och drift

Anta att dokumenttjänsten plötsligt börjar svara långsamt. Plattformsteamet ser att instanserna når minnesgränsen; applikationsteamet upptäcker att senaste versionen håller kvar stora dokument i minnet. Åtgärden kräver båda perspektiven, även om ägarskapet för felsökning och ändring måste vara bestämt i förväg.

En möjlig ansvarsfördelning är att applikationsteamet beskriver kapacitetsbehov, hälsokontroller, start- och stoppbeteende samt programberoenden. Plattformsteamet erbjuder tillförlitlig exekvering, placering, automatisering, resurskontroll och gemensam övervakningsinfrastruktur. Infrastruktur- och säkerhetsfunktioner beslutar om de övergripande regler som måste gälla. Vem som får förändra resursgränser, vem som godkänner avvikelser från isoleringskrav och vem som leder en incident ska vara uttryckligt överenskommet, inte bero på organisationsnamn. Det behöver även vara tydligt vem som följer upp att vald körmiljö levererar överenskomna förutsättningar.

För **applikationsläsaren** är den nya insikten att driftsättning inte bara är att lämna över ett program: tjänstens resurs- och felbeteende är en del av leveransen. För **driftläsaren** är insikten att traditionell kompetens om servrar, virtualisering och kapacitet fortfarande behövs, men i kombination med dynamisk placering, automatisering och förståelse för tjänsteberoenden. R4

Vi väljer körmiljö för dokumenttjänsten

I vårt fortsatta exempel låter vi dokumenttjänsten köras som två containerinstanser på en gemensamt förvaltad plattform, fördelade på skilda noder. Dokumenten lagras utanför instanserna. Det är ett illustrativt val, inte ett påstående om den enda rätta lösningen. Teamen testar hur tjänsten påverkas av omstart, otillräckligt minne och fördröjd lagring. De kontrollerar också att instanserna verkligen placeras på skilda noder och att trafik kan ledas bort från en instans som inte är redo. Om två noder ändå delar ett kritiskt beroende dokumenteras den kvarstående risken.

Härifrån återstår en avgörande fråga: hur når ärendetjänsten dokumenttjänsten och hur tillåts, säkras och styrs trafiken mellan dem? Det blir nästa kapitels ämne.

Det vi behöver tänka annorlunda kring

Övergången till mikrotjänster tar inte bort servrar, lagring eller nätverk. Den gör **tjänstens logiska ansvar** mindre bundet till **en bestämd teknisk körplats**. Det kan ge snabbare förändringar, flexiblare resursutnyttjande och bättre felisolering, men endast om tjänster och plattformar utformas för det och deras ansvarsfördelning fungerar.

Källor och vidare läsning

- **R4** Kubernetes, Liveness, Readiness, and Startup Probes (åtkomst 2026-10-17).
- **R5** Kubernetes, ReplicaSet (åtkomst 2026-10-17).

6. Hur kommunicerar tjänsterna?

När ärendehanteringen och dokumenthanteringen låg i samma backend kunde de dela programkod, anropa varandras funktioner och använda samma databas. När dokumenthanteringen blir en självständig tjänst förändras något grundläggande: även en till synes enkel fråga om ett dokument måste nu passera en gräns där nätverk, kontrakt, säkerhet och driftförutsättningar spelar roll.

Föreställ dig att en handläggare öppnar ett ärende. Ärendetjänsten behöver visa en lista över dokument. I testmiljön fungerar allt, men i produktionen visas inga dokument. Ett tomt resultat får inte förväxlas med ett lyckat API-svar: gränssnittet behöver tydligt skilja mellan att ärendet saknar dokument och att hämtningen har misslyckats. Loggarna visar att ärendetjänsten försöker nå dokumenttjänsten – och får timeout. Är felet i programmet, DNS, lastbalanseringen, en brandvägg, tjänstens hälsa eller i en behörighetskontroll? Och vem ska avgöra det?

Det här kapitlet handlar om hur **tjänsternas kommunikation blir både en arkitekturfråga och en driftfråga**. Val av API eller händelseflöde kan inte göras isolerat från trafikväg, säkerhetsgränser och möjligheten att felsöka helheten.

Från interna anrop till kontrakt över nätverk

Ett internt metodanrop i en monolit sker ofta inom samma process och kan dela minne och datatyper. Ett anrop till en annan tjänst behöver däremot ett uttryckligt kontrakt: adress, protokoll, meddelandeformat,

identitet och hur olika svar ska tolkas. Även om tjänsterna körs på samma fysiska värd måste man tänka på att mottagaren kan startas om, bytas ut eller vara otillgänglig när anropet görs.

Kontraktet är inte bara en teknisk specifikation. Det är en överenskommelse mellan parter som kan utvecklas och driftsättas vid olika tidpunkter. Dokumenttjänsten måste exempelvis kunna ändra sin interna implementation utan att ärendetjänsten går sönder. Därför behöver kontrakten vara stabila, dokumenterade, testbara och utvecklas med medvetenhet om befintliga konsumenter.

Det är lätt att göra misstaget att ersätta interna funktionsanrop med många täta fjärranrop. Om en enda skärmbild kräver en lång kedja av små anrop blir användarupplevelsen känslig för fördröjning och partiella fel. Tjänstegränser bör därför också bedömas utifrån hur ofta och på vilket sätt tjänsterna behöver kommunicera.

Synkrona anrop när svar behövs direkt

Ett synkront API-anrop passar när ärendetjänsten behöver ett svar för att kunna visa aktuella dokumentmetadata. Ofta används HTTP med ett tydligt definierat gränssnitt. Avsändaren måste veta när den ska sluta vänta och vad som ska hända om svaret uteblir.

Här behöver vi skilja mellan olika fel. Ett nekande svar på grund av saknad behörighet betyder något annat än att tjänsten är överbelastad. Ett timeout-fel betyder inte nödvändigtvis att mottagaren inte utförde åtgärden; svaret kan ha försvunnit på vägen tillbaka. Därför är det riskabelt att automatiskt försöka igen, särskilt för operationer som skapar eller ändrar information. Säker återupprepning kan kräva idempotens, till exempel ett unikt operations-ID som mottagaren kan känna igen.

Även läsanrop behöver välavvägda tidsgränser. Om ärendetjänsten väntar länge på dokumenttjänsten kan dess egna resurser bindas upp och antalet väntande förfrågningar växa. Överbelastning kan då spridas mellan tjänster. Hur sådana fel begränsas fördjupas i kapitel 12; här är poängen att kommunikationsmönstret måste ge utrymme för dem.

Asynkron kommunikation när arbetet kan ske senare

Alla behov kräver inte ett omedelbart svar. När ett dokument har registrerats kanske andra delar av systemet ska bygga ett sökindex eller skicka en notifiering. Ett meddelande eller en händelse kan göra det möjligt att utföra arbetet senare, utan att den som registrerar dokumentet måste vänta.

Asynkron kommunikation tar inte bort beroenden. Den förändrar dem. Nu måste någon ansvara för köer eller meddelandetjänster, kvarvarande meddelanden, omleverans, felhantering och hur konsumenter hanterar äldre meddelandeformat. En avsändare behöver också förstå vilken garanti transporten erbjuder: ett meddelande kan exempelvis levereras mer än en gång, och ordningen kan variera mellan mottagare eller partitioner. Mottagaren måste kunna hantera sådana situationer.

En händelse om att ett dokument har registrerats säger inte automatiskt att alla mottagare har hunnit uppdatera sina vyer. Fördröjningen kan vara acceptabel, men den ska vara ett medvetet verksamhetsbeslut. Informationskonsistens och transaktionsgränser behandlas vidare i kapitel 7. R6

Vägen från tjänst till tjänst

I en traditionell miljö har applikationen ofta ett begränsat antal kända nätverksförbindelser: användare till frontend, frontend till backend och backend till databasen. Dessa kan avspeglas i relativt statiska brandväggsregler och lastbalanseringskonfigurationer.

När fler tjänster samverkar tillkommer fler trafikflöden. De är inte alltid synliga genom att man bara studerar vilka VM:ar som finns. Tjänster kan ha flera instanser som byts ut eller flyttas, samtidigt som deras logiska adresser behöver vara stabila. Därför behöver man skilja mellan **tjänstens namn**, **dess aktuella nätverksändpunkter** och **de nätverksregler som tillåter kommunikationen**.

DNS och tjänsteupptäckt kan hjälpa en klient att hitta rätt mottagare. En lastbalanserare eller annan trafikfunktion kan fördela förfrågningar mellan flera instanser. Vid inkommande användartrafik kan det finnas en ingress eller API-gateway som är en gemensam ingång. Intern kommunikation mellan tjänster kan följa andra vägar. Det finns inte ett enda

obligatoriskt mönster; valet beror på plattform, trafik, säkerhet och komplexitet.

Lastbalansering ger inte automatiskt hög tillgänglighet. Om samtliga instanser är beroende av samma trasiga databas eller om trafikstyrningen själv blir en felpunkt kan tjänsten ändå vara nere. Det krävs också rimliga hälsosignaler och kapacitet för att återstående instanser ska klara belastningen. R6

Brandväggar och zoner finns kvar – men gränserna kan förändras

Ett vanligt missförstånd är att brandväggar blir irrelevanta när tjänster körs i containrar. Tvärtom måste organisationen fortfarande bestämma vilka flöden som är tillåtna. Men reglerna kan behöva knytas till logiska tjänster och deras identiteter eller etiketter, snarare än enbart till långlivade serveradresser. Identitetsbaserad åtkomstkontroll kompletterar, men ersätter inte alltid, nätverkssegmentering.

Tänk att ärendetjänsten behöver nå dokumenttjänstens API, men inte direkt dess datalager. En princip om minsta nödvändiga åtkomst innebär då att endast den avsedda trafikvägen ska tillåtas. Det kan uppnås med traditionella brandväggar, nätverkspolicyer på en plattform eller en kombination. Hur reglerna realiseras är inte samma sak som vem som fastställer säkerhetskraven.

Nätverkszoner kan fortsatt vara viktiga, exempelvis för att skilja produktion från test eller särskilt skyddsvärda system från andra resurser. En del organisationer väljer dessutom finmaskig segmentering mellan tjänster inom en zon. Att en tjänst körs i en separat container eller namnrymd bevisar dock inte i sig att nätverket är säkert isolerat.

En praktisk förändring uppstår när applikationsteamet behöver ändra trafikflödet ofta. Det är sällan hållbart att för varje ny tjänsteinstans beställa en manuell regel för en ny IP-adress. I stället kan applikationsteamet beskriva vilket flöde som behövs, en gemensam plattform omsätta beskrivningen till regler och nätverks- eller säkerhetsfunktionen fastställa vilka förbindelser som över huvud taget får öppnas. Denna ansvarsfördelning är ett alternativ, inte ett krav som följer av mikrotjänster.

Kryptering och identitet längs trafikvägen

Trafik mellan tjänster kan behöva skyddas med TLS, och i vissa miljöer används ömsesidig TLS för att också styrka den anropande tjänstens identitet. Certifikat behöver utfärdas, förnyas och spärras på ett kontrollerat sätt. Automatisering kan minska manuella fel, men skapar samtidigt krav på säkra rutiner för nycklar och tillit.

En krypterad förbindelse säger inte att den anropande tjänsten får utföra varje åtgärd. Nätverksåtkomst, autentisering och auktorisation är olika kontroller som samverkar. Vem användaren är, hur en tjänst styrker sin identitet och hur beslut om rättigheter fattas behandlas mer utförligt i kapitel 15.

När kommunikationen inte fungerar – vem felsöker?

Återvänd till produktionsproblemet: dokumentlistan visas inte. Applikationsteamet kan kontrollera vilket API som anropades, vilken tidsgräns som användes och hur tjänsten hanterade felet. Plattformsteamet kan undersöka tjänsteupptäckt, lastbalansering, instansstatus och eventuella plattformspolicyer. Nätverksfunktionen kan verifiera om trafiken passerar avsedda zoner och brandväggar. Säkerhetsfunktionen kan utreda om en policy ska ändras – eller om blockeringen faktiskt är korrekt.

För att denna samverkan ska fungera behövs gemensamma spår: korrelations-ID i anrop, begripliga felkoder och relevanta loggar från både applikation och trafikkomponenter. Felsökning får inte kräva att någon manuellt gissar vilken VM ett anrop råkade passera. Hur signalerna används i övervakning och incidentarbete fördjupas i kapitel 13 och 14.

Ett viktigt beslut måste vara fattat i förväg: **vem leder felsökningen när flera ansvariga funktioner berörs?** Den som äger nätverket behöver inte äga den verksamhetsincident som användaren upplever. Och den som äger ärendetjänsten behöver inte ha rätt att ändra brandväggspolicyn.

En möjlig ansvarsfördelning

Beslut eller uppgift	Applikationsteam	Plattform/drift	Nätverk/säkerhet
Beskriva nödvändiga anrop och API-kontrakt	Huvudansvar	Bidrar med driftskrav	Bidrar med säkerhetskrav
Fastställa tillåtna trafikflöden	Föreslår behov	Implementerar enligt modell	Fastställer övergripande policy
Driva lastbalanseringsförmåga	Anger tjänstens behov	Huvudansvar i denna modell	Samverkar vid exponeringskrav
Hantera timeout och fel i klientkod	Huvudansvar	Tillhandahåller mätvärden	—
Felsöka kommunikationsproblem	Undersöker anrop	Undersöker plattform och trafik	Undersöker nätverk och policy

Tabellen är ett exempel på ansvar för genomförande och samverkan, inte en fullständig RACI-matris. Beslutsmandat för avsteg, driftberedskap och incidentledning behöver dokumenteras separat. En liten organisation kan samla flera roller i samma team, och en större organisation kan separera nätverk, säkerhet, drift och plattform. Det viktiga är att ingen förutsätter att någon annan äger helheten.

Vårt ärendehanteringssystem – från fel till förbättring

I vårt exempel visar det sig att ärendetjänsten försöker nå dokumenttjänsten via en adress som fungerar i utvecklingsmiljön men inte är avsedd för produktion. Produktionsmiljön har en korrekt nätverkspolicy som blockerar denna väg. Felet är alltså inte att brandväggen behöver öppnas utan att en miljöspecifik adress har hamnat i applikationens konfiguration.

Lösningen är att dokumenttjänsten får en stabil, godkänd tjänsteadress i respektive miljö och att applikationens konfiguration styrs via en kontrollerad leveransprocess. Applikationsteamet ändrar klientkonfigurationen

och kompletterar med automatiserade konfigurationskontroller. Ett separat integrationstest i en representativ miljö verifierar adressupplösning, faktisk trafikväg och att blockeringar ger synliga fel i stället för en missvisande tom dokumentlista. Plattformsteamet ser till att den avsedda tjänsteadressen och de dokumenterade trafikreglerna finns. Säkerhetsfunktionen behöver inte godkänna en generell brandväggsöppning för att kompensera för felet.

Detta visar varför arkitektur och drift inte kan behandlas var för sig. Ett litet konfigurationsfel blir ett problem över flera ansvarsgränser, men en fungerande samarbetsmodell gör det möjligt att både åtgärda det och minska risken för upprepning.

Det som faktiskt förändras

I en monolit var en stor del av kommunikationen ett internt programmeringsproblem. I en mikrotjänstarkitektur blir flera sådana beroenden uttryckliga och går över process- eller nätverksgränser. Det ger möjlighet till självständighet, men ökar behovet av genomtänkta kontrakt, trafikvägar och ansvar.

För **applikationssidan** betyder förändringen att nätverkets beteende och tekniska fel måste påverka hur API:er och klienter byggs. För **driftsidan** betyder den att tjänsternas logiska beroenden behöver synas i nätverkspolicy, trafikstyrning och felsökning – även när instanserna flyttar på sig. För **båda** blir kommunikation en gemensam arkitekturfråga snarare än enbart ett anrop i kod eller en port i en brandvägg.

Källor och vidare läsning

- **R6** Microsoft Learn, Design interservice communication for microservices (åtkomst 2026-10-17).

7. Vem ansvarar för informationen?

När ärendehanteringssystemet bestod av en gemensam backend och databas kunde utvecklarna ofta lösa ett nytt informationsbehov genom att lägga till en kolumn, skapa en relation och skriva en SQL-fråga. Det betydde inte att informationsägarskapet var enkelt, men tekniskt låg informationen nära till hands. När dokumenthanteringen blir en separat tjänst förändras förutsättningarna. Ärendetjänsten behöver fortfarande visa vilka dokument som hör till ärendet, men kan inte längre utgå från att den själv kan läsa och ändra dokumenttjänstens tabeller.

Frågan är inte bara var data ska lagras. Den handlar om vem som får definiera dess innebörd, vem som får ändra dess struktur, hur flera tjänster kan ge användaren en korrekt bild och vem som ser till att informationen finns kvar och går att hantera över tid.

En databas gjorde inte informationen till ett gemensamt ansvar

I vårt traditionella ärendehanteringssystem finns kanske tabeller för ärenden, parter, beslut, dokument och händelser i samma databas. En transaktion kan då uppdatera ett ärende och registrera en dokumentkoppling som en sammanhållen operation. En rapport kan använda en JOIN över tabellerna. Ett DBA-team kan sköta databasplattform, index, kapacitet, behörigheter och backup, medan applikationsteamet arbetar med tabeller och frågor.

Men att flera tabeller råkar ligga i samma databas betyder inte att de

bör ha samma verksamhetsansvar. Även i en monolit behöver organisationen veta vem som beslutar vad ett dokument, ett ärende eller ett beslut betyder, vilken information som får lagras och vilka kvalitetskrav som gäller. Mikrotjänster gör bara ansvarsskillnaderna mer synliga – och tekniskt svårare att kringgå.

Tre ansvar som måste hållas isär

Vi behöver skilja mellan minst tre nivåer.

Informationsansvar gäller begrepp, regler, klassificering, gallring, kvalitet och tillåten användning. Verksamheten eller en utsedd informationsansvarig måste kunna avgöra vad ett ärende och dess dokument betyder och vilka krav som gäller. Detta ansvar upphör inte för att en tjänst har en egen databas.

Tjänstens dataansvar gäller den datamodell och de gränssnitt som tjänsten använder för att uppfylla sitt uppdrag. Dokumenttjänstens team bör till exempel styra sin interna representation av dokumentmetadata och publicera överenskomna funktioner för att registrera eller läsa dem. Andra tjänster bör normalt inte koppla sin logik direkt till dokumenttjänstens tabellstruktur.

Databasdrift gäller den tekniska plattformens tillgänglighet, säkerhetskonfiguration, lagring, övervakning, kapacitet, patchning och mekanismer för backup och återställning. Det kan vara ett centralt DBA-team, ett plattformsteam eller en extern leverantör som utför mycket av detta. Tjänsteteamet måste ändå förstå hur databasen används och vilka risker dess ändringar medför.

Den som ansvarar för teknisk drift bestämmer alltså inte automatiskt informationsbegreppens innebörd. Den som äger datamodellen behöver inte själv administrera databasoperativsystemet. Och den som ansvarar för verksamhetsinformationen behöver inte godkänna varje teknisk schemaändring. Gränserna behöver definieras – inte antas.

Vad betyder det att en tjänst äger sina data?

Principen om tjänsteägd data innebär att andra tjänster i normalfallet inte ska ändra en tjänsts information genom direkt åtkomst till dess interna lagring. Ägandet handlar framför allt om kontroll över datarepresen-

tation och förändringstakt. Det kräver inte en fysisk databasserver per tjänst. Logisk isolering genom separata scheman och behörigheter kan vara tillräcklig i vissa fall, medan högre säkerhets- eller tillgänglighetskrav kan motivera starkare separation. Separata databaser, scheman eller åtkomstområden kan användas, beroende på risker och plattformsval. En delad databasmotor kan fortfarande drivas centralt, men en gemensam motor får inte bli en genväg till täta kopplingar mellan tjänsternas tabeller.

Tänk att ärendetjänsten behöver visa filnamn, dokumenttyp och skapandedatum. Det behöver också vara tydligt vilken tjänst som är auktoritativ för respektive uppgift, särskilt om en läsmodell innehåller kopior. Den kan hämta uppgifterna från dokumenttjänstens API eller använda en egen läsmodell som uppdateras av publicerade händelser. Den ska däremot inte i hemlighet bygga sin funktion på att en viss kolumn alltid finns i dokumenttjänstens databas. Om dokumenttjänsten då ändrar sitt schema kan den annars slå sönder ärendehanteringen trots att dess publika gränssnitt är oförändrat.

Tjänsteägarskap betyder heller inte att all information ska dupliceras överallt. Kopior kan vara motiverade för snabb läsning eller rapportering, men de innebär frågor om aktualitet, lagringsskydd, rättelse, gallring och vilka uppgifter som är auktoritativa.

När en SQL-transaktion inte längre räcker

Anta att en handläggare bifogar ett dokument till ett ärende. Tidigare kunde backend skapa en dokumentpost och en kopplingspost inom samma databastransaktion. Efter uppdelningen utförs minst två logiska handlingar: dokumenttjänsten tar emot dokumentet, och ärendetjänsten registrerar kopplingen till ärendet. Om den ena handlingen lyckas och den andra misslyckas uppstår ett mellanläge.

En distribuerad arkitektur kan sällan förutsätta en gemensam, snabb och tillgänglig transaktion över alla berörda tjänster. Det är möjligt att använda samordnade transaktioner i vissa miljöer, men det medför egna beroenden och är inte en generell lösning. Ofta behöver man i stället utforma verksamhetsflödet så att delstegen kan slutföras, upprepas, kompenseras eller markeras för manuell hantering.

En möjlig lösning är att dokumenttjänsten först tar emot filen och ger den ett identifierbart tillstånd, exempelvis *mottagen men ännu inte knu-*

ten till ärende. Ärendetjänsten registrerar därefter kopplingen. Om kopplingen misslyckas kan flödet försöka igen, eller låta dokumentet ligga kvar i väntan på kontroll och eventuell borttagning enligt fastställda regler. Vilken ordning som är rätt beror på verksamhetens krav och risken för förlorade respektive felaktigt kopplade dokument.

Kompensation innebär inte att tiden vrids tillbaka. Ett redan skickat meddelande eller en gjord extern åtgärd kanske inte kan göras ogjord. Då behövs en ny, spårbar åtgärd som korrigerar resultatet. Systemet bör också kunna förklara för användaren om registreringen är klar, pågår eller kräver åtgärd. R7 R8

Eventuell konsistens – ett medvetet avsteg från omedelbar samtidighet

Om ärendetjänsten har en egen lista över dokument kan den uppdateras genom en händelse från dokumenttjänsten. Listan blir då ibland tillfälligt inaktuell. Detta kallas ofta *eventuell konsistens*: förutsatt att uppdateringarna fungerar och inga nya förändringar sker kan kopiorna så småningom komma överens.

Detta kan vara acceptabelt för en söklista eller en översikt där en kort, överenskommen fördröjning saknar betydelse. Fördröjningen måste mätas och hanteras: eventuell konsistens anger inte i sig någon övre tidsgräns. Det kan vara oacceptabelt om handläggaren måste veta säkert att en handling har registrerats före en tidsfrist, eller om ett beslut får fattas endast när viss information har mottagits. Där krävs en tydlig källa till sanning och en verifierbar bekräftelse på det steg som är avgörande.

Här ligger en viktig arkitekturfråga: *vilka uppgifter måste vara korrekta just nu, och vilka får bli korrekta efter en viss fördröjning?* Svaret måste komma från verksamhetsreglerna. Teknikvalet får inte tyst ändra vad användaren kan lita på.

Beroende på meddelandesystemets och konsumenternas garantier kan meddelanden komma fram flera gånger eller behandlas i annan ordning än de skapades. Begreppet *idempotens* används för operationer där en upprepad begäran inte får skapa ytterligare affärseffekter. Konsumenter behöver därför kunna känna igen upprepade händelser och i relevanta fall hantera versioner eller ordning. Händelsebaserad synkronisering är inte en garanti för att systemet alltid konvergerar; mekanismer för upptäckt,

ombearbetning och avstämning måste finnas. R7

Schemaändringar blir en del av tjänstens löpande leverans

När flera funktioner läser samma tabeller kräver en schemaändring ofta samordning av alla användare. Tjänsteägd data kan ge en mer självständig förändringstakt, men bara om den publika informationen hanteras stabilt. Ett internt fältnamn kan bytas utan att andra märker det; ett API-fält som konsumenter använder kan däremot behöva stödjas under en övergångsperiod.

Även inom en tjänst kan nya och gamla applikationsinstanser köra samtidigt vid driftsättning. Databasmigreringar behöver därför ofta ske stegvis: först läggs en kompatibel struktur till, sedan börjar tjänsten använda den och först senare tas den gamla bort. Man behöver också tänka igenom om en misslyckad release kan rullas tillbaka när datamodellen redan förändrats. En återgång av programkod är inte samma sak som en säker återgång av databasen.

Det innebär att applikationsteam och databasansvariga behöver ett gemensamt arbetssätt för riskbedömning, migrering, kapacitet och verifiering. Även om en automatisk databasbackup finns måste ansvar för återläsning och verksamhetsmässig avstämning vara uttryckligt; samordnad katastrofåterställning behandlas närmare i kapitel 17. Centrala regler kan fortfarande vara nödvändiga, men manuella databasköer för varje rutinändring kan bli ett hinder om många tjänster levereras självständigt. Automatisering, tydliga godkännanden och spårbara migreringar är ofta bättre än osynliga genvägar.

Hur påverkas drift och förvaltning?

För driftläsaren är den viktigaste förändringen att *en verksamhetsinformation inte längre nödvändigtvis finns i en enda databas med ett enda återställningsögonblick*. Flera datalager, händelseströmmar och kopior kan behöva samverka. En databas kan vara tekniskt frisk medan användaren ser en gammal dokumentlista eller saknar ett registrerat dokument. Övervakningen behöver därför följa både tekniska resurser och informationsflöden – exempelvis växande köer eller uteblivna uppdateringar.

Det betyder inte att DBA-kompetensen blir mindre värdefull. Tvärtom behövs den för dimensionering, tillförlitlig databasdrift, indexering, säkerhetskonfiguration och vägledning om databasernas beteende. Skillnaden är att databastekniskt ansvar ofta behöver paketeras som en gemensam plattformsförmåga med tydliga tjänstegränser och självbetjäning där det är lämpligt.

För applikationsläsaren blir konsekvensen att informationsflödet över tjänstegränser är en del av funktionens korrekta beteende. Det räcker inte att spara data i den egna tjänsten. Teamet måste även veta vad som händer om ett meddelande fastnar, hur en dubblerad begäran hanteras och hur ett påbörjat arbetsflöde slutförs eller synliggörs.

Verksamhetens informationsansvariga måste samtidigt kunna bestämma bevarande, skydd, spårbarhet och kvalitet för informationen oavsett hur många tjänster som behandlar den. Skyldigheterna försvinner inte för att data kopieras till en ny läsmodell.

Ett gemensamt beslut om dokumentinformationen

I vårt exempel fastställer verksamheten vilka uppgifter som utgör en registrerad handling, när registreringen räknas som klar och hur länge dokumentinformation ska bevaras. Dokumenttjänstens team ansvarar för sin datamodell, de publicerade kontrakten och hur registreringens olika tillstånd hanteras. Ärendetjänstens team ansvarar för att användaren får en begriplig status och för att en möjlig läskopia inte framställs som om den alltid vore aktuell. Databas- eller plattformsteamet tillhandahåller de tekniska mekanismer som lösningen behöver och följer upp deras drift.

Teamens ansvar överlappar i frågor som kräver gemensamma beslut. Ett exempel är hur länge en utebliven uppdatering får pågå innan en incident ska utlösas. Verksamheten anger konsekvenserna, applikationsteamen definierar informationens synliga tillstånd och driftfunktionen säkerställer att avvikelsen kan upptäckas och hanteras. Ansvar behöver dokumenteras på den nivån, inte bara med en rad som säger ”databasen – drift”.

Från delad lagring till uttryckligt informationsansvar

Det avgörande skiftet är inte att varje mikrotjänst ska få sin egen databasserver. Det är att **informationens betydelse, tekniska representation och drift måste ha uttalade ansvariga**. När vi inte längre kan förlita oss på en gemensam transaktion eller en direkt SQL-fråga blir det viktigare att synliggöra mellanlägen, dataflöden och krav på aktualitet.

Det är först då självständiga tjänster kan förändras utan att verksamheten förlorar kontrollen över den information som gör IT-stödet användbart.

Källor och vidare läsning

- **R7** Microsoft Learn, Saga distributed transactions pattern (åtkomst 2026-10-17).
- **R8** Microsoft Learn, Compensating Transaction pattern (åtkomst 2026-10-17).

8. Hur förändras utvecklingsmiljöerna?

När dokumenthanteringen var en del av ärendehanteringssystemets backend kunde utvecklarna ofta starta samma applikation på sina datorer, ansluta till en lokal eller gemensam databas och prova en ändring. Efter utbrytningen behöver ärendetjänsten kommunicera med dokumenttjänsten. Och dokumenttjänsten är i sin tur beroende av lagring, identitetstjänster och kanske en meddelandekö. Måste alla dessa delar vara igång för att en utvecklare ska kunna ändra en knapp eller en affärsregel?

Det korta svaret är nej. Men det kräver ett medvetet val av **vilka beroenden som ska finnas på riktigt under utvecklingen och vilka som kan ersättas med kontrollerade alternativ**. Den frågan berör både applikations- och driftorganisationen.

Vad vi brukade mena med utvecklingsmiljö

I det traditionella IT-stödet var utvecklingsmiljön ofta en relativt stabil installation: en eller flera virtuella maskiner, en applikationsserver, en databas och gemensamma anslutningar till andra system. I vissa organisationer arbetade utvecklarna främst lokalt och använde en central integrationsmiljö när förändringen var redo. I andra var en delad utvecklingsserver själva navet i arbetet.

Den modellen har fördelar. En gemensam miljö gör det lätt att visa ett helt system, och driftorganisationen kan känna till alla resurser. Samtidigt kan utvecklare störa varandras tester och behöva invänta gemensamma installationer. Det är alltså inte ett nytt problem som mikrotjänster

skapar, men antalet självständiga förändringar och externa beroenden gör problemet mer påtagligt.

I en mikrotjänstarkitektur är **utvecklingsmiljö** inte nödvändigtvis en kopia av produktionen. Det kan betyda flera kompletterande saker: en enskild utvecklares lokala körning, ett teamägt utrymme på en gemensam plattform, en miljö för samspel mellan tjänster eller en kortlivad miljö som skapas för en viss ändring. Skillnaderna behöver vara uttalade, annars talar teamen om olika saker när de säger att en ändring är ”testad i utveckling”.

En tjänst i taget – utan att tappa bort helheten

Anta att en utvecklare ska ändra hur dokumentens filnamn visas. För att prova visningsregeln behövs kanske inte en riktig dokumentlagring eller en fullständig ärendehantering. Dokumenttjänsten kan köras lokalt med kontrollerad testinformation och ett simulerat svar från de externa beroendena.

En **testdubbel** kan ersätta en annan tjänst eller ett externt API och ge förutsägbara svar. Det gör utvecklingen snabbare och ger möjlighet att prova fel som är svåra att återskapa i en gemensam miljö. Men en testdubbel bevisar bara hur den egna tjänsten reagerar på det beteende som simuleringen beskriver. Den garanterar inte att den riktiga ärendetjänsten eller identitetsleverantören har samma kontrakt och beteende. Ett simulerat beroende måste därför hållas i takt med verkliga kontrakt, och verkliga integrationer måste verifieras separat. Därför behöver simuleringar kompletteras med kontrakts- och integrationstester längre fram.

För andra ändringar krävs en riktig databas, en meddelandekö eller flera tjänster tillsammans. En lokal uppsättning containrar kan då vara praktisk, men är inte ett krav. Det är inte heller alltid rimligt att köra resurskrävande eller skyddsvärda beroenden på en utvecklares dator. Även lokala processer och gemensamma utvecklingsresurser kan vara rätt val. Avgörande är hur lätt utvecklaren kan återskapa förutsättningarna och förstå vilka versioner och inställningar som används. R9

Det svåra är inte att starta – utan att återskapa

En utvecklare upptäcker ett fel som uppträder efter att ett dokument laddats upp två gånger. För att återskapa felet behövs både testdata och rätt ordning på händelser från dokumenttjänsten. Om utvecklingsmiljön innehåller slumpmässiga gamla testdata eller en okänd version av en beroende tjänst blir det svårt att avgöra om felet ligger i den egna koden.

Därför blir reproducerbarhet viktigare än att samtliga utvecklare använder exakt samma installation. Det bör gå att få fram kända versioner av relevanta beroenden, kontrollerad konfiguration och testdata som inte röjer produktionsinformation. Hur detta görs kan variera: versionsstyrda miljödefinitioner, automatiserade installationsskript, byggda containeravbilder och kataloger över gemensamma utvecklingstjänster är olika medel för samma mål.

Konfiguration hör inte hemma som lokala hemligheter i källkoden. Anslutningsuppgifter, certifikat och nycklar ska hanteras med särskilda mekanismer och separeras mellan miljöer. Testkonton behöver tillräckliga men begränsade rättigheter. Att utvecklingsmiljön är tillfällig gör inte informationssäkerheten oviktig.

Delade miljöer löser ett problem men kan skapa ett annat

En gemensam utvecklingsmiljö är användbar när många tjänster ska demonstreras tillsammans. Men om flera team kontinuerligt ersätter sina tjänsteversioner i samma miljö kan en ändring i dokumenttjänsten plötsligt påverka ärendeteamets arbete. Felsökningen kräver då kännedom om vem som driftsatt vad och när.

Möjliga sätt att minska osäkerheten är att införa tydliga versionsmarkeringar, stabila referensmiljöer, överenskomna uppdateringsfönster för vissa gemensamma resurser och automatiserad återställning av testdata. För delar som kan isoleras kan varje team eller ändring få ett eget logiskt miljöutrymme. Det betyder inte automatiskt en egen VM eller ett eget kluster.

Tillfälliga utvecklingsmiljöer kan byggas när en ändring behöver granskas, och tas bort när den inte längre behövs. De behöver definierad

livslängd, ägare, resursgränser och rutin för att avveckla både miljön och dess testdata. De minskar vissa konflikter men förbrukar resurser och kräver hantering av åtkomst, kostnader och städning. De löser inte heller behovet av ett kontrollerat test av de verkliga tjänsternas samspel. Vi återkommer till det i kapitel 9.

Vilka miljögränser behöver finnas?

Miljöuppdelningen handlar om mer än namnen utveckling, test och produktion. Skillnaderna kan omfatta identiteter, nätverksåtkomst, datakällor, lagringsutrymmen, behörigheter, loggning och vilka externa integrationer som får anropas.

En utvecklingstjänst ska exempelvis inte av misstag kunna skicka verkliga notifieringar till användare eller skriva i produktionsdatabasen. Samtidigt kan vissa gemensamma integrationer vara svåra eller dyra att duplicera. Då måste det dokumenteras vad som är verkligt, vad som är simulerat och vilka begränsningar det innebär för resultatet. Isolering genom namnrymder eller separata konfigurationsvärden räcker inte som säkerhetsgaranti utan kompletterande kontroll av åtkomst och trafik.

En vanlig missuppfattning är att mikrotjänster kräver en fullständig kopia av hela systemet per utvecklare. I själva verket är en genomtänkt kombination av lokala tjänster, simulerade beroenden och delade komponenter ofta både snabbare och billigare. Motprestationen är att teamen måste veta när den lokala verifieringen inte räcker.

Vem bygger och vem sköter utvecklingsmiljön?

I det traditionella arbetssättet kan beställningen ha varit ”vi behöver en ny utvecklings-VM med en databas”. I en plattformsorienterad miljö kan begäran i stället vara ”vi behöver en isolerad utvecklingsinstans av dokumenttjänsten med tillgång till en testdatabas och en simulerad notifieringstjänst”. Det flyttar fokus från en viss maskin till en efterfrågad förmåga.

Applikationsteamet behöver beskriva vilka beroenden tjänsten har, hur den konfigureras, hur testdata skapas och hur den kan startas och kontrolleras. Teamet måste också kunna tolka om ett fel hör till den egna

tjänsten eller till ett beroende.

Plattforms- och driftfunktionerna kan tillhandahålla och förvalta automatiserad miljöetablering, nätverk, tillräcklig kapacitet, databastjänster, åtkomstkontroll, loggning och gemensamma mekanismer för hemligheter. De behöver insyn i vad teamen använder för att kunna hantera säkerhet, livscykel och kostnad.

Säkerhets- och informationsansvariga behöver ange vilka miljöer som får använda vilka data, hur identiteter ska hanteras och vilka anslutningar som är tillåtna. Det är möjligt att fördela uppgifter på andra sätt, men det får inte vara oklart vem som beslutar, genomför och kontrollerar dem.

För delade utvecklingsmiljöer behöver någon dessutom ansvara för publicerad versionsinformation, upptäckt av störningar och en tydlig kontaktväg vid fel. Annars kan teamen fastna i felsökning av andra teams ändringar utan möjlighet att avgöra vad som hänt.

Det viktiga är att en självbetjäningsplattform inte innebär att varje utvecklare fritt får kringgå nätverks- eller dataskyddskrav. Tvärtom kan den ge ett bättre skydd när godkända mönster erbjuds automatiskt, i stället för att varje nytt behov kräver en unik manuell installation.

Dokumenttjänsten i vår utvecklingsmiljö

För vårt fiktiva ärendehanteringssystem väljer vi att låta dokumentteamet utveckla lokalt mot en liten testdatabas och en simulerad ärendetjänst. När ett nytt dokumentflöde behöver verifieras används en gemensam miljö med riktiga instanser av båda tjänsterna. Ett särskilt testkonto och syntetiska dokument används för att undvika personuppgifter från produktionen.

Vi dokumenterar också hur man framkallar avbrott mot lagring och timeout från ärendetjänsten. Då kan utvecklarna prova det egna beteendet utan att behöva orsaka störningar för övriga team. Den gemensamma miljön används senare för att kontrollera att den verkliga kommunikationen fungerar. Här är den viktigaste frågan att miljön går att skapa, förstå och felsöka; om kommunikationen uppfyller sina kontrakt och verksamhetskrav prövas systematiskt i nästa kapitel.

Här ser vi den viktigaste ansvarsförändringen: driftorganisationen behöver inte längre manuellt installera varje utvecklarförändring på en ge-

mensam server, men måste fortfarande ge teamen pålitliga, säkra och begripliga miljöförutsättningar. Applikationsteamet får större ansvar för att miljöbehov och beroenden går att beskriva och återskapa.

En utvecklingsmiljö är en förutsättning – inte ett kvitto

En snabb och reproducerbar utvecklingsmiljö gör det lättare att skapa, förstå och rätta tjänster. Men reproducerbar miljö och snabb lokal återkoppling är **förutsättningar**, inte bevis på att hela systemet fungerar. Teststrategi, kompatibilitetskrav och verksamhetskritiska scenarier hör därför till nästa kapitel.

I nästa kapitel går vi därför vidare från miljöernas uppbyggnad till frågan om **hur en helhet av självständigt föränderliga tjänster ska kvalitetssäkras**.

Källor och vidare läsning

- **R9** Docker, Docker Compose documentation (kontrollerad 2026-10-17).

9. Hur förändras testmiljöer och kvalitetssäkring?

En ny version av dokumenttjänsten klarar alla sina egna tester. Den är driftsatt i integrationsmiljön och svarar snabbt. Ändå kan handläggarna inte längre öppna dokument från ett ärende: ärendetjänsten skickar en förfrågan enligt ett äldre gränssnitt, och dokumenttjänsten förväntar sig ett nytt obligatoriskt fält. Varje team kan med visst fog säga att den egna tjänsten fungerar. Men IT-stödet fungerar inte.

Detta är en grundläggande skillnad mellan att testa en traditionell applikation och att kvalitetssäkra ett distribuerat system. I en monolit kan en gemensam systemversion ofta byggas, installeras och testas som en helhet. Med självständigt utvecklade och driftsatta tjänster finns inte alltid en enda versionskombination som representerar allt som kan möta produktion. Testningen måste därför ge svar på två frågor samtidigt: fungerar tjänsten på egen hand, och fortsätter verksamhetens flöden att fungera när andra tjänster utvecklas oberoende?

Det välbekanta systemtestet räcker inte ensamt

I vårt traditionella ärendehanteringssystem finns utvecklings-, test- och produktionsmiljöer. En ny release innehåller typiskt sammanhållna ändringar i backend, frontend och databas. Den installeras i en gemensam testmiljö, där systemtest och acceptanstest genomförs innan driftsätt-

ning. Metoden har styrkor: en tydlig kandidat till produktionssättning, ett känt installationspaket och möjlighet att pröva användarflöden i en sammanhållen miljö.

När dokumenthanteringen blir en fristående tjänst ändras förutsättningarna. Ärendetjänsten och dokumenttjänsten får egna releaser. En tredje tjänst kanske publicerar händelser när ett beslut fattas. Integrationstestmiljön kan vid varje tidpunkt innehålla en annan kombination av versioner än produktion. Att vänta tills *alla* tjänster kan testas som en enda release skulle i praktiken återinföra ett centralt leveransberoende.

Det betyder inte att systemtester och acceptanstester ska överges. Det betyder att de behöver kompletteras med snabbare och mer avgränsade kontroller, tydliga kontrakt och verifiering av de risker som uppstår i samspelet mellan tjänster. R10

Flera testnivåer besvarar olika frågor

Ett vanligt misstag är att välja antingen många isolerade tjänstetester eller omfattande end-to-end-tester. Det behövs normalt flera testnivåer med olika syften.

Tester inom en tjänst prövar affärsregler, validering och felhantering utan att alla beroenden behöver startas. Dokumenttjänsten kan till exempel verifiera att filformat och dokumentstatus hanteras korrekt. Dessa tester kan köras snabbt och ger ofta tydliga felmeddelanden. Men de visar inte att två tjänster tolkar ett gränssnitt likadant.

Kontraktstester prövar överenskommelsen mellan en tjänst som anropar ett gränssnitt och en tjänst som tillhandahåller det. Kontraktet kan gälla HTTP-anrop, meddelanden eller händelser: fältnamn, format, tillåtna värden, felkoder och ibland beteenden. Om ärendetjänsten behöver ett dokument-ID och en status ska en ändring hos dokumenttjänsten inte oväntat ta bort eller omdefiniera dessa uppgifter. Tester bör omfatta de konsumentkrav som faktiskt finns; ett godkänt kontraktstest är inte en garanti för hela verksamhetsflödet.

Integrationstester använder verkliga beroenden där det är motiverat: databas, kö, identitetsfunktion eller en annan tjänst. De fångar till exempel fel i serialisering, nätverkskonfiguration eller transaktionshantering. Sådana tester blir dyrare och svårare att felsöka ju fler komponenter som kopplas in samtidigt. Därför bör varje test ha en tydlig fråga.

End-to-end-tester följer ett användarflöde genom flera tjänster. Handläggaren registrerar ett ärende, bifogar ett dokument och öppnar dokumentet igen. Testet kan upptäcka att helheten brister trots att varje enskild tjänst verkar korrekt. Men ett stort antal sådana tester kan bli långsamma och instabila, särskilt när delade miljöer och asynkrona processer är inblandade. Ett mindre urval kritiska flöden ger ofta större nytta än att försöka testa alla variationer via användargränssnittet.

Icke-funktionella tester prövar andra egenskaper: kapacitet, svarstider, återhämtning, behörigheter, loggning och beteende vid bortfall. De är särskilt viktiga när en nätverksanslutning eller extern tjänst blir ett nytt felställe. Belastningstester ska även beakta gemensamma resurser, exempelvis databaser och köer, som flera tjänster kan överbelasta samtidigt.

Fel som uppstår på grund av fördröjningar, timeout och otillgängliga beroenden bör också testas avsiktligt. För asynkrona flöden behöver testen kunna vänta på ett definierat resultat med en rimlig tidsgräns, i stället för att förutsätta att händelsen hanteras omedelbart.

Testnivåerna kompletterar varandra. De är inte en universell pyramid med fasta procentsatser; sammansättningen bör styras av systemets risker, förändringstakt och kostnaden för att upptäcka fel sent. R10

Hur vet vi att olika versioner fungerar tillsammans?

Anta att dokumenttjänstens API har version A i produktion. Teamet vill införa version B med en ny egenskap. Ärendetjänsten behöver fortfarande kunna använda de uppgifter den tidigare fått. Ett praktiskt mål är att kunna driftsätta B utan att samtidigt behöva ändra och driftsätta ärendetjänsten.

Det kan uppnås genom bakåtkompatibel förändring: lägg till ett frivilligt fält i stället för att omedelbart kräva det, behåll tidigare beteenden under en övergångsperiod och använd kontraktstester för att se att befintliga konsumenter fortfarande fungerar. I andra fall behövs parallella gränssnitt eller planerade migrationer. Versionering är ett verktyg, inte en ersättning för att förstå konsumenternas verkliga beroenden.

En integrationsmiljö med enbart de *senaste* tjänsteversionerna fångar inte alla problem. Kontraktsverifieringen behöver dessutom använda de

kontrakt som faktiskt representerar konsumenter i drift och under den planerade utrullningen; annars kan ett nytt kontrakt felaktigt se kompatibelt ut när en äldre konsument fortfarande använder gränssnittet. Det kan behövas tester mot kombinationer som faktiskt kan förekomma, exempelvis den gamla ärendetjänsten tillsammans med den nya dokumenttjänsten. Det är sällan rimligt att testa varje matematisk kombination av alla tjänsteversioner. Välj i stället kombinationer utifrån uttalade kompatibilitetslöften, beroendekartor, planerad utrullning och identifierade risker.

Samma resonemang gäller händelser i meddelandeköer. Ett nytt fält eller en ändrad tolkning kan påverka konsumenter som uppdateras långt senare. En producent behöver därför känna till vilka kontrakt som måste fortsätta fungera och hur gamla meddelanden hanteras under en övergång.

Vad är en testmiljö i ett distribuerat system?

Tidigare kunde ett testmiljönamn peka på en avgränsad uppsättning VM:ar, applikationer och databaser. Med mikrotjänster kan samma namn avse en logisk miljö som delar fysisk kapacitet med andra miljöer. Man behöver skilja mellan *vilka tjänster och data som hör till miljön*, *vilken infrastruktur som kör dem* och *vilka säkerhetsgränser som faktiskt finns*.

Tre modeller kan användas, var för sig eller tillsammans:

- **Delad integrationsmiljö.** Flera team provar samspel i en gemensam uppsättning tjänster. Den kan ge realistiska beroenden och underlätta gemensam felsökning, men kräver styrning av ändringar, versionsspårbarhet, testdata och konflikter mellan tester.
- **Tillfällig miljö för en förändring.** En ändring får en kortlivad uppsättning relevanta tjänster eller resurser, ibland med simulerade beroenden. Det kan minska köer till en gemensam miljö, men kostar kapacitet och måste städas bort. En tillfällig miljö är inte automatiskt en fullständig kopia av produktion.
- **Separat miljö för kritisk helhetsverifiering.** Särskilda prestanda-, säkerhets- eller acceptanstester kan kräva mer stabil konfiguration, bestämda versioner eller starkare isolering. En sådan miljö kan vara dyr men motiverad av riskerna.

En miljö som delar kluster med andra får inte beskrivas som isolerad ba-

ra för att den har ett eget namn eller en egen namnrymd. Behörigheter, nätverkspolicyer, resurstilldelning, åtkomst till hemligheter och separation från produktion måste utformas och kontrolleras utifrån kraven.

Testdata måste fungera över tjänstegränser

När den traditionella databasen delas upp kan ett ärende finnas i ärendetjänstens datalager samtidigt som dokumentets metadata och filinnehåll finns någon annanstans. Ett enkelt utdrag ur en databas är då inte säkert ett användbart testfall. Testdata behöver representera ett sammanhängande verksamhetstillstånd även om informationen ägs av flera tjänster.

I dokumentexemplet måste testmiljön kunna skapa ett ärende, koppla det till ett dokument och kontrollera vad som händer om uppladdningen misslyckas efter att ärendet registrerats. Testerna bör omfatta både lyckade flöden och situationer med fördröjda händelser, dubletter, ogiltiga referenser och avbrott.

För testdata som bygger på produktionsinformation gäller särskilda krav på integritet, sekretess, åtkomst och gallring. Anonymisering eller maskning behöver bedömas utifrån om uppgifterna faktiskt går att återidentifiera. Syntetiska data är ofta lämpliga, men måste täcka tillräckligt realistiska variationer. Vid flera datalager behövs dessutom kontroller för hur data skapas och städas bort utan att lämna inkonsekventa rester i andra tjänster. Det finns ingen allmän regel om att en kopia av produktionsdatabasen är det bästa testunderlaget.

Simulerade beroenden och verkliga integrationer

Kapitel 8 beskrev hur utvecklare kan arbeta lokalt med en simulerad dokumenttjänst. För kvalitetssäkringen uppstår en ny fråga: hur vet vi att simuleringen fortfarande beskriver verkligheten? Om testdubbeln alltid svarar snabbt och aldrig returnerar fel kan den dölja precis de problem som är viktigast att förstå.

En bra strategi är att använda simuleringar för snabba och avgränsade tester, samtidigt som kontrakt och vissa integrationer verifieras mot verkliga implementationer. Testdubblar bör täcka överenskomna felbeteenden och uppdateras när kontrakten ändras. Den övergripande test-

strategin behöver också bestämma var verkliga nätverkspolicyer, autentisering och köer ska ingå; annars kan en tjänst se felfri ut i laboratoriet men stoppas i produktion.

När ett test misslyckas – vem agerar?

Ett end-to-end-test som inte kan öppna ett dokument ger inte automatiskt svar på vilket team som orsakat felet. Felet kan ligga i ärendetjänstens anrop, dokumenttjänstens kontrakt, nätverkspolicyn, identitetsinställningen eller testmiljöns data. Det bör därför finnas gemensam spårbarhet från testets körning till tjänsteversioner, anropsidentifierare och miljökonfiguration.

En möjlig ansvarsfördelning är att varje **applikationsteam** ansvarar för tester av den egna tjänstens regler och gränssnitt, inklusive relevanta kontrakt. **Plattforms- och driftfunktioner** tillhandahåller testmiljöernas exekvering, isolering, gemensam diagnostik och resurshantering enligt överenskomna förutsättningar. **Test- och verksamhetsfunktioner** bidrar med kritiska verksamhetsflöden, acceptanskriterier och bedömning av kvarstående risker. I delade miljöer behöver någon även ha mandat att prioritera miljöstabilitet och hantera tester som påverkar andra team. Vem som felsöker gemensamma tester, godkänner undantag eller beslutar om produktionssättning behöver anges uttryckligen; ingen av dessa arbetsfördelningar följer automatiskt av mikrotjänstarkitekturen.

För applikationsläsaren innebär förändringen att kontrakt, felbeteenden och testbarhet måste byggas in i tjänsterna, inte läggas till vid en gemensam sluttestvecka. För driftläsaren innebär den att miljön inte bara ska kunna etableras på ett reproducerbart sätt, som i kapitel 8, utan också **ge tillförlitliga och spårbara testresultat**. Det kräver kända tjänsteversioner, avgränsad testdata, gemensam diagnostik och tydliga regler när olika teams tester delar resurser.

Vad vårt ärendehanteringssystem behöver verifiera

Inför nästa förändring av dokumenttjänsten väljer teamen ett begränsat antal tydliga kontroller. De provar dokumenttjänstens egna regler, kör kontraktstester mot ärendetjänstens förväntningar och testar verklig kommunikation via plattformens identitets- och nätverksfunktioner.

Ett litet antal helhetstester verifierar att handläggaren kan lägga till och öppna ett dokument. Testerna dokumenterar också vilka delar av produktionsmiljön som inte efterliknas, så att kvarstående risker inte döljs av ett grönt testresultat. Ett feltest låter dokumenttjänsten bli otillgänglig och kontrollerar att gränssnittet visar ett begripligt fel i stället för en falskt tom dokumentlista.

Man testar inte alla tänkbara versionskombinationer, utan de kombinationer som kan uppstå under den planerade förändringen. I testresultatet anges vilken dokumenttjänst, ärendetjänst, konfiguration och testdatamängd som användes. När ett test misslyckas kan teamen därför börja med gemensamma fakta i stället för att gissa vem som ska ta ärendet.

I nästa kapitel följer vi själva leveransen. Hur kan en godkänd förändring driftsättas utan att kräva en samtidig gemensam release, och hur kan organisationen återgå eller gå vidare när något ändå går fel?

Källor och vidare läsning

- **R10** Pact, When to use Pact (kontrollerad 2026-10-17).

10. Hur levererar vi förändringar utan att störa helheten?

En ny version av dokumenttjänsten är klar. Den kan lagra fler uppgifter om dokumenten, men den nuvarande ärendetjänsten förväntar sig det gamla svaret. I ett traditionellt IT-stöd hade verksamheten kanske planerat en gemensam release där backend, databas och användargränssnitt uppdaterades samtidigt. Nu vill dokumentteamet gå vidare utan att invänta alla andra team. Går det att göra utan att riskera ärendehandläggarnas arbete?

Svaret beror inte i första hand på hur snabbt vi kan starta en ny container. Det beror på om vi har utformat **förändringen, kontrakten, driftsättningen och återhämtningsförmågan** så att flera versioner kan fungera samtidigt. Oberoende leveranser är en möjlighet, inte ett löfte som automatiskt följer av att en applikation delats upp.

Från gemensam release till flera förändringstakter

I den traditionella miljön har ett ändringsärende ofta omfattat hela systemet. Utveckling levererar ett samlat paket, test verifierar en bestämd kombination och drift genomför installationen vid ett avtalat tillfälle. Modellen ger en tydlig helhetskontroll, men kan innebära att små ändringar får vänta på stora.

I mikrotjänstmiljön kan ett team leverera dokumenttjänsten medan andra tjänster förblir oförändrade. Det gör det möjligt att minska förändringens storlek och snabbare rätta fel. Samtidigt är den driftsatta helheten en kombination av flera versioner, ofta utan ett enda versionsnummer som beskriver allt. Drift, förvaltning och incidentledning måste därför kunna se **vilken version av vilken tjänst som körs var**, vilka kontrakt den använder och vilka förändringar som nyligen genomförts.

Målet är inte att alla team måste driftsätta varje dag. Målet är att en förändring ska kunna genomföras när den är tillräckligt säker, utan omotiverade beroenden till andra förändringar. R11

En leveranskedja som kan upprepas och granskas

En hållbar leverans börjar innan någon trycker på knappen för driftsättning. Källkod och konfiguration versionshanteras. Bygget ger en identifierbar artefakt som testas, kontrolleras och helst förs vidare oförändrad genom miljöerna. Om samma källa byggs om med andra beroenden vid produktionstillfället är det svårare att veta om man verkligen driftsätter det som testats.

En automatisk leveranskedja (CI/CD) kan exempelvis kontrollera kod, beroenden och sårbarheter, köra relevanta tester, paketera artefakten, dokumentera resultatet och tillämpa de godkännanden som krävs för produktion. Det betyder inte att alla kontroller ska vara automatiska. En verksamhetskritisk ändring kan behöva särskild riskbedömning eller ett tidsatt beslut. Skillnaden är att kontrollerna kan göras **spårbara och återkommande**, i stället för att bero på manuellt återkommande installationssteg.

Även leveranskedjans egna behörigheter, byggberoenden och skyddsmekanismer behöver granskas. Ett godkänt automatiserat flöde innebär inte att varje förändring är riskfri.

Det ska gå att besvara: Vem begärde förändringen? Vad har byggts? Vilka verifieringar passerade? Vilken konfiguration används? Vem godkände produktionssättningen? Vilka tjänster kan påverkas? Hur följs resultatet upp? Dessa uppgifter bör kunna knytas till både ändringen och den faktiska körmiljön.

Bakåtkompatibilitet är förutsättningen för självständighet

Vi återvänder till dokumenttjänsten. Den nya versionen inför ett nytt fält för dokumentklassificering. Ett säkert första steg kan vara att **lägga till** fältet utan att ta bort något som ärendetjänsten behöver. Dokumenttjänsten börjar stödja både gamla och nya konsumenter. Därefter kan ärendetjänsten uppdateras. Först när ingen längre använder det gamla kontraktet är det lämpligt att avveckla det.

Detta brukar beskrivas som *expandera – migrera – dra tillbaka*. Samma tanke behövs vid databasförändringar: ett nytt schema kan behöva stödja både tidigare och ny applikationsversion under en övergångstid. En ändring som direkt tar bort en kolumn eller byter betydelse på ett fält kan bryta äldre instanser, även om den nya versionen har passerat sina egna tester.

Kompatibilitet måste bedömas utifrån beteende, inte bara datatyper. Ett API som returnerar samma fält men med ändrad betydelse kan vara lika problematiskt som ett borttaget fält. För asynkrona händelser gäller motsvarande krav för producenter och konsumenter, även när meddelanden ligger kvar i köer efter en driftsättning.

Ibland går en förändring inte att göra bakåtkompatibel. Då behöver beroendena kartläggas och en samordnad migrering beslutas. Självständig driftsättning innebär inte att samordning är förbjuden, bara att den används när den faktiskt behövs.

Att rulla ut utan att byta allt på en gång

En ny version kan ersätta den gamla vid ett enda tillfälle, men det kan innebära ett avbrott. Det finns också gradvisa strategier. Vid **rullande uppdatering** byts instanser stegvis, förutsatt att kapacitet och kompatibilitet räcker under övergången. Med **blue–green** finns två uppsättningar och trafiken kan flyttas mellan dem. Med **canary** får en mindre del av trafiken först gå till den nya versionen medan man följer relevanta signaler.

Strategierna löser olika problem. Blue–green kan ge en tydlig trafikväxling men kräver extra kapacitet och hantering av gemensamma data. Canary ger möjlighet att observera effekter på begränsad trafik, men en andel HTTP-anrop styr inte automatiskt vilka instanser som behandlar

meddelanden i köer. För asynkrona konsumenter och gemensamma data krävs därför en särskild strategi för införande och återgång. En rullande uppdatering kräver att gammal och ny version verkligen kan samexistera.

Ingen utrullningsmodell garanterar att verksamheten slipper påverkan. För dokumenttjänsten behöver man exempelvis följa både tekniska fel och om dokument faktiskt går att registrera och hämta. En låg felfrekvens i webbservern är inte tillräcklig om dokumenten inte visas i ärendena.

Rollback är inte alltid att backa bandet

Om den nya dokumenttjänsten ger fel vill vi snabbt kunna återgå till den tidigare programversionen. Men vad händer om den nya versionen redan har ändrat databasens struktur eller skickat nya händelser? Att återställa en äldre container gör inte dessa förändringar ogjorda. En tidigare version kanske inte ens kan läsa de nya uppgifterna.

Därför behöver beslut om **rollback eller roll-forward** ingå i förändringens design. Teamen behöver i förväg bestämma vad som kan återställas, vilka mätvärden som ska utlösa ett stopp och vem som får fatta beslut om att avbryta införandet. Rollback kan vara lämpligt när gränssnitt och data fortfarande är kompatibla. Roll-forward innebär att felet rättas genom en ny framåtriktad ändring. Vid vissa datamigreringar behövs särskilda återställnings- eller kompensationsåtgärder. Att återläsa en databas är ett annat, större ingrepp som även kan påverka andra tjänster och dataägare.

Förberedda skyddsmekanismer, som funktionsflaggor eller möjlighet att stoppa en ny funktion, kan ibland ge snabbare riskbegränsning än att byta hela programversionen. De kräver i sin tur ansvar för flaggornas livscykel och säkra grundvärden. R11

Förändringsstyrning när leveranserna är många

Traditionell förändringshantering har ofta haft fokus på att granska en samlad systemrelease. I en distribuerad miljö kan antalet mindre driftsättningar bli betydligt större. Om varje rutinändring måste hanteras

med samma manuella process riskerar styrningen att bli ett hinder utan att öka säkerheten.

Ett alternativ är att skilja **förhandsgranskade standardförändringar** med låg och välkänd risk från förändringar som behöver särskilt godkännande. Automatisk dokumentation av byggen, tester, konfiguration och produktionssättning kan ge bättre revisionsspår än manuellt ifyllda checklistor. Men förutsättningen är att det finns tydliga kriterier för vilka förändringar som får gå genom vilken väg, och att avvikelser verkligen fångas upp.

Här möts applikations- och driftperspektivet. Applikationsteamet kan bedöma beteende, kompatibilitet och funktionella risker. Drift- och plattformsteam kan bedöma kapacitet, utrullningsmetod, teknisk återhämtning och påverkan på gemensamma beroenden. Säkerhetsfunktionen anger nödvändiga kontroller. Verksamhetsansvariga behöver kunna avgöra när en funktionell förändring innebär särskild verksamhetsrisk.

Vem får göra vad i produktion?

Att en teknisk plattform erbjuder självbetjänad driftsättning betyder inte att varje utvecklare automatiskt ska ha obegränsad produktionsbehörighet. Åtkomst, godkännanden och ansvar behöver utformas efter risk och organisationens styrmodell.

En möjlig uppdelning är att applikationsteamet ansvarar för leveransbarhet, kontrakt, tester och tydliga driftkrav. Plattformsteamet ansvarar för en säker och reproducerbar leveransväg, åtkomstmekanismer och övervakning av plattformens funktion. Driftorganisationen kan ansvara för övergripande produktionsberedskap, förändringssamordning och incidentledning. I en annan organisation kan samma produktteam bära en större del av det operativa ansvaret. Det viktiga är att **mandat och skyldigheter motsvarar varandra**.

Produktionsbehörigheter bör begränsas till det som uppgiften kräver. Automatiska leveransidentiteter behöver egna kontroller och spårbarhet, liksom undantagsåtkomst vid incidenter.

Före driftsättning behöver det exempelvis vara bestämt vem som kan avbryta en utrullning, vem som fattar beslut vid försämrade verksamhetssignaler och vem som ansvarar för kommunikationen om användarna

påverkas. Efter driftsättning behövs en överenskommen period eller metod för uppföljning, inte bara ett grönt statusmeddelande från leveransverktyget.

En förändring i ärendehanteringssystemet

Dokumentteamet vill driftsätta klassificeringsfältet innan ärendeteamet har hunnit uppdatera sin vy. Teamen kommer överens om att dokumenttjänsten först accepterar och returnerar den gamla representationen, samtidigt som den nya representationen kan aktiveras separat. Databasändringen görs additivt. Det finns en plan för när det äldre fältet senare får avvecklas.

En ny artefakt byggs och verifieras. Teamen har i förväg bestämt när den standardiserade förändringsvägen är tillräcklig och när ett särskilt godkännande krävs. Plattformens leveransflöde rullar ut den till en begränsad del av trafiken. Under införandet följer teamen både tekniska fel och om handläggare kan lägga till och öppna dokument. Om något försämras kan den nya funktionen stängas av utan att den gemensamma dokumenthanteringen behöver tas ur drift. Det är ett valt scenario: i verkligheten behöver även händelseflöden, last och datamigrering analyseras innan en liknande metod väljs.

Den viktiga vinsten är inte att uppdateringen sker snabbare, utan att **risken är begränsad, förändringen spårbar och ansvaret tydligt**. Dokumentteamet kan leverera utan att invänta en fullständig systemrelease, samtidigt som driftsidan har möjlighet att förstå, följa och ingripa i vad som händer i produktion.

Det som faktiskt förändras

I en monolit är det ofta möjligt att pröva och driftsätta en samlad version. I ett distribuerat IT-stöd är leveransförmågan en del av arkitekturen: varje tjänst måste tåla att andra tjänster under en period kör andra versioner, och driftmodellen måste kunna hantera många mindre förändringar utan att tappa helhetskontrollen.

Oberoende förändring är därför ett gemensamt resultat av kompatibel applikationsdesign, reproducerbar teknisk leverans och tydlig verksamhetsstyrning. Nästa kapitel behandlar de gemensamma plattformsförmågor som gör detta möjligt i praktiken.

Källor och vidare läsning

- **R11** Martin Fowler, Continuous Delivery (kontrollerad 2026-10-17).

11. Vem tillhandahåller miljöer och gemensamma förmågor?

I det traditionella ärendehanteringssystemet går det kanske att beskriva vägen till en ny miljö med några få beställningar: en virtuell maskin från servergruppen, en databas från DBA-funktionen, ett DNS-namn och en brandväggsöppning. Så småningom installerar applikationsgruppen sin programvara. Varje leverans kan fungera bra var för sig, men tiden mellan idé och körbar miljö styrs av flera manuella överlämningar.

När dokumenthanteringen blir en självständig tjänst förändras förutsättningarna. Fler tjänster kan behöva utvecklas, provas, uppdateras och driftsättas vid olika tillfällen. Om varje tjänst behöver en lång kedja av specialbeställningar förlorar arkitekturen en del av den självständighet den var avsedd att skapa. Lösningen behöver inte vara att alla utvecklare blir serveradministratörer. Den kan vara att organisationen bygger gemensamma **plattformsförmågor** som gör beprövade driftlösningar enkla att använda utan att ansvar och säkerhet går förlorade. R12

Plattformen är ett erbjudande, inte bara ett kluster

En plattform är mer än servrar, containrar eller en orkestrator. Den blir användbar först när den erbjuder dokumenterade och förutsägbara sätt att få något gjort: skapa en tjänstemiljö, driftsätta en version, ansluta en databas, hantera hemligheter, se loggar och mäta om tjänsten fungerar.

Organisationen behöver veta vilka förmågor som tillhandahålls, vilka krav de uppfyller och vilka begränsningar användarna måste acceptera.

Det kan beskrivas som ett **plattformskontrakt**. Det bör även ange kontaktvägar, ansvar vid incidenter, stödda versioner och hur en plattformsförändring meddelas. Kontraktet anger till exempel vad ett applikationsteam behöver deklarera, vilka kontroller som görs automatiskt, vem som godkänner undantag, vilken tillgänglighet plattformen erbjuder och hur stöd och incidenter hanteras. Kontraktet behöver inte vara ett juridiskt dokument. Poängen är att beroendet mellan utveckling och plattform blir begripligt och styrbart.

En väl vald plattformsförmåga minskar det arbete som varje team annars måste uppfinna på nytt. Samtidigt är en förmåga som döljer för mycket farlig: om applikationsteamet inte kan förstå hur tjänstens data lagras, hur den återställs eller hur nätverkstrafiken tillåts uppstår nya ansvarsglapp.

Från serverbeställning till deklarerat behov

I vårt exempel behöver dokumenttjänsten en körmiljö med två möjliga instanser, lagring för uppladdade dokument, ett sätt att nå ärendetjänsten och ett sätt att få sina loggar och mätvärden insamlade. Tidigare kunde vart och ett av dessa behov leda till en separat beställning. I en mer automatiserad modell beskriver applikationsteamet sina behov i versionsstyrd konfiguration, varefter plattformen skapar eller uppdaterar de resurser som krävs.

Detta brukar kallas **infrastruktur som kod** (Infrastructure as Code, IaC). Det betyder inte att all infrastruktur blir applikationskod, utan att dess önskade tillstånd kan beskrivas, granskas och ändras reproducerbart. En deklaration anger önskat läge men garanterar inte att verkligheten alltid överensstämmer med det; avvikelser mellan deklarerat och faktiskt tillstånd behöver upptäckas och hanteras. Automatisering kan minska manuella skillnader mellan miljöer, men en felaktig mall eller för omfattande behörighet kan sprida ett fel mycket snabbt. Därför behövs granskning, versionskontroll, ändringsspår och kontroll av vad som faktiskt har blivit installerat.

Automatisering betyder heller inte att varje team får beställa obegränsade resurser. Organisationen behöver standardiserade resursklasser, kvoter, säkerhetsregler, kostnadsgränser och rutiner för undantag.

Den självbetjäning som fungerar bäst är ofta **självbetjäning inom överenskomna ramar**.

Vilka gemensamma förmågor behövs?

Det är frestande att bygga en stor plattform innan de första tjänsterna har några tydliga behov. Ett bättre tillvägagångssätt är att utgå från återkommande uppgifter och göra dem lätta att utföra på ett säkert sätt.

För dokumenttjänsten kan ett första gemensamt erbjudande omfatta:

- **Körning och driftsättning:** en godkänd väg från byggd artefakt till körande instans, med versioner, resursgränser och återstartsmekanismer.
- **Konfiguration och hemligheter:** separering av kod och miljöspecifika värden, skydd för lösenord och nycklar samt kontrollerad rotation.
- **Nätverk och trafik:** namnuppslagning, trafikvägar, nätverkspolicyer och certifikathantering enligt beslutad säkerhetsmodell.
- **Databas och lagring:** standardiserad provisionering, övervakning, säkerhetskopiering och återställningsförutsättningar för godkända datatjänster.
- **Observerbarhet:** central insamling av loggar, mätvärden och spår, med gemensamma begrepp och åtkomstregler.
- **Policy och efterlevnad:** kontroller som upptäcker otillåtna inställningar eller osäkra beroenden innan förändringen når produktion.
- **Miljöernas livscykel:** möjlighet att skapa, uppdatera och avveckla utvecklings- och testmiljöer utan att kvarlämnade resurser växer okontrollerat.

Att flera funktioner finns i samma katalog innebär inte att de drivs av ett enda team. En databastjänst kan exempelvis levereras av en DBA-funktion, medan en gemensam plattform gör beställning, behörigheter och övervakning enhetliga. Det behöver vara tydligt om ett löfte om tillgänglighet gäller databasfunktionen, hela dokumenttjänsten eller den verksamhetsprocess som använder dem.

Databastjänst utan förlorat dataansvar

Anta att dokumenttjänsten får sin databas genom en gemensam databastjänst. DBA- eller plattformsteamet kan ansvara för motorns patchning, kapacitet, krypteringsinställningar, backupfunktioner och teknisk övervakning. Dokumenttjänstens team ansvarar fortfarande för datamodellen, schemaförändringar, rimliga frågor och att applikationens data går att tolka vid återställning. Verksamhetens ansvar för informationens riktighet och hantering försvinner inte.

Här räcker det inte att säga att plattformen tar backup. Det måste vara dokumenterat vad som säkerhetskopieras, hur ofta, vilka återställningspunkter som är möjliga och om dokumentfiler och metadata kan återställas så att de passar ihop. En tekniskt fungerande databasåterläsning är inte automatiskt en återställd verksamhetsprocess. Den större frågan om samordnad kontinuitet behandlas längre fram i boken.

Hemligheter, certifikat och identiteter

I en traditionell installation kan en drifttekniker manuellt lägga in ett certifikat eller ett lösenord på en server. När många tjänster startas om och byter instanser blir den modellen svår att skala och lätt att glömma bort. Plattformen kan i stället tillhandahålla tjänsteidentiteter, säker lagring och distribution av hemligheter samt automatiserad certifikathantering.

Men automatisering av förvaring och rotation är inte samma sak som beslut om åtkomst. Applikationsteamet behöver beskriva vilka tjänster och data det faktiskt behöver nå. Säkerhetsfunktionen kan fastställa krav på separation, minsta behörighet och spårbarhet. Plattformen genomdriver delar av dessa krav. Ansvarsfördelningen behöver vara tydlig även när en central identitetsleverantör används. Frågor om autentisering och auktorisation utvecklas i kapitel 15. R13

Vem får ändra vad?

En modell där utvecklare kan driftsätta självständigt kan skapa oro för att drift tappar kontroll. Automatiserad självbetjäning är dock inte liktydig med att varje tjänsteteam får administrativa rättigheter till den underliggande infrastrukturen. En modell där varje resursändring kräver ett manuellt ärende kan å andra sidan omöjliggöra den förändringstakt

arkitekturen bygger på. Det är inte nödvändigt att välja mellan gränslös frihet och total centralisering.

En möjlig uppdelning är att applikationsteamet får ändra sin tjänst inom en godkänd resursprofil och ett publicerat plattformskontrakt. Plattformsteamet bestämmer hur profilen omsätts till resurser och hur plattformen uppgraderas. Säkerhetsfunktionen bestämmer krav och vilka avvikelser som behöver godkännas. En funktion med helhetsansvar följer upp att verksamhetens krav på tillgänglighet och kontinuitet uppnås.

Denna uppdelning behöver kompletteras med beslut om vem som får öka kvoter, byta databasprodukt, ändra nätverkspolicyer och stoppa driftsättningar under störningar. Mandat ska finnas före incidenten, inte uppfinnas under den.

Det är också möjligt att samma team har flera roller i en mindre organisation. Poängen är att ansvaren ska vara identifierade, inte att varje ansvar kräver en egen avdelning.

Plattformen måste själv förvaltas som en produkt

En gemensam plattform ger standardisering men blir också ett beroende för många tjänster. En felaktig uppdatering av ett gemensamt certifikatflöde eller en ändrad nätverkspolicy kan få större spridning än ett fel i en enskild mikrotjänst. Plattformens egna ändringar behöver därför testas, kommuniceras och driftsättas kontrollerat. Tjänsteteam behöver veta vilka delar som är stabila, vilka som är på väg att avvecklas och hur länge kompatibilitet upprätthålls.

För att plattformen ska bli användbar över tid behövs ett uttalat ägarskap, en väg för användarstöd, kapacitet att hantera förbättringsförslag och en plan för avveckling av äldre plattformsgränssnitt. Plattformens kvalitet bör inte bara mätas i upptid. Den bör också bedömas utifrån hur lätt det är att använda säkra standardlösningar, hur snabbt team kan få en fungerande miljö, hur ofta manuella undantag krävs och hur tydligt incidenter kan felsökas. Om självbetjäningen kräver specialistkunskap och långa handböcker har plattformen kanske bara flyttat komplexiteten till utvecklaren.

För driftsidan innebär detta en förskjutning från att utföra samma manuella moment upprepade gånger till att utveckla och förvalta gemensam-

ma automatiserade förmågor. Det är ett kvalificerat driftuppdrag, inte en avveckling av driftkompetensen. För applikationssidan innebär det att kunna formulera sina behov i standardiserade termer och ta ansvar för hur den egna tjänsten använder plattformens erbjudande.

När en ny dokumentmiljö ska skapas

Vi återvänder till ärendehanteringssystemet. Dokumentteamet vill starta en tillfällig utvecklingsmiljö för en ny förhandsvisningsfunktion. Teamet väljer en godkänd miljötyp och beskriver vilken tjänsteversion, databasprofil och resursnivå som behövs. Plattformen kontrollerar kvot, identitet och nätverkspolicy, skapar resurser och kopplar på standardövervakning. Miljön får ett ägarskap, en kostnadsmarkering och en regel för automatisk avveckling.

Om dokumenttjänsten behöver kommunicera med ärendetjänsten över en otillåten zon ska plattformen inte tyst öppna trafiken. Den ska ge en begriplig felrapport och hänvisa till ett beslutat undantagsflöde. På samma sätt ska dokumentteamet inte försöka kringgå resursgränser genom dolda manuella ändringar. Det är just samspelet mellan standardiserad frihet och synliga ansvar som gör lösningen förvaltningsbar.

Kapitlets viktigaste slutsats är att självständiga mikrotjänster inte kräver att varje utvecklingsteam driver sin egen infrastruktur. De kräver en **tydlig relation mellan tjänstens krav och plattformens erbjudande**. När den relationen fungerar kan både utveckling och drift arbeta mer självständigt utan att verksamhetens helhet tappas bort.

Källor och vidare läsning

- **R12** Microsoft Learn, Platform engineering documentation (kontrollerad 2026-10-17).
- **R13** Kubernetes, Multi-tenancy (kontrollerad 2026-10-17).

12. Vad händer när en tjänst slutar fungera?

Handläggaren öppnar ett ärende och väntar på att tillhörande dokument ska visas. Ärendeinformationen finns redan på skärmen, men dokumentlistan laddas aldrig färdigt. Efter en stund visar hela sidan ett felmeddelande. Samtidigt rapporterar driftplattformen att både ärendetjänsten och dokumenttjänsten körs. Hur kan IT-stödet ändå upplevas som otillgängligt?

I ett distribuerat system är en grön markering för en process inte samma sak som att ett verksamhetsflöde fungerar. Flera tjänster kan var för sig vara igång, men samspelet mellan dem kan misslyckas. Därför behöver både applikations- och driftorganisationen förstå **partiella fel**: situationer där en del av helheten fungerar medan en annan del är långsam, otillgänglig eller ger ett osäkert resultat. Frågan är inte bara hur felet kan åtgärdas, utan hur systemet ska bete sig medan felet pågår.

När den lokala felmodellen inte längre räcker

I ett traditionellt ärendehanteringssystem anropar backend ofta sina egna moduler inom samma process. Metodanrop kan misslyckas, men det finns vanligtvis inget separat nätverk mellan två delar av samma program. När en funktion bryts ut till en dokumenttjänst förändras felbilden. Anropet mellan ärendetjänsten och dokumenttjänsten kan misslyckas även om båda programmen fungerar korrekt.

Ett nätverksanrop kan stoppas av brandväggsregler, gå till en instans som håller på att avslutas, fördröjas av överbelastning eller avbrytas när

en mellanliggande komponent får problem. Ett svar kan också försvinna *efter* att dokumenttjänsten har utfört begäran. Då kan anroparen inte säkert veta om operationen genomfördes. Detta är en avgörande skillnad mot ett vanligt internt funktionsanrop: osäkerheten gäller ibland inte bara felorsaken utan även resultatet.

Den nya felmodellen innebär inte att monoliter saknar nätverksberoenden. Även en monolit talar med databaser, lagring och andra system. Skillnaden är att mikrotjänstarkitekturen medvetet inför fler anropsgränser mellan delar som tillsammans utför verksamhetsfunktionen. Ju fler beroenden som krävs för ett användarflöde, desto fler platser kan det flödet brytas på.

Att vänta är också ett beslut

Anta att ärendetjänsten begär en dokumentlista och dokumenttjänsten inte svarar. Om anropet får vänta obegränsat kan användaren, applikationens anropstrådar och anslutningspooler bli upptagna. Vid hög trafik kan ett långsamt beroende på det sättet sprida belastning bakåt i kedjan.

En **timeout** sätter en gräns för hur länge en anropare väntar. Men en alltför kort gräns avbryter även legitima anrop vid tillfälliga toppar; en alltför lång kan låsa resurser och ge användaren oacceptabla väntetider. Tidsgränser behöver därför utgå från tjänstens normala svarstider, verksamhetens krav och hur mycket tid hela anropskedjan får använda. Ett gemensamt tidsutrymme för hela begäran kan behöva fördelas mellan flera underanrop.

Timeout betyder inte att arbete automatiskt avbryts hos mottagaren. Dokumenttjänsten kan fortsätta en operation efter att ärendetjänsten slutat vänta. För ändrande operationer är detta viktigt: ett nytt försök kan annars utföra samma förändring igen. R14

Återförsök kan hjälpa – eller förvärra

Om en begäran misslyckas vid ett kort nätverksavbrott kan ett nytt försök vara rimligt. Men om dokumenttjänsten redan är överbelastad och varje anropare försöker flera gånger, kan trafiken öka just när tjänsten minst klarar det. Om flera tjänster i kedjan dessutom gör egna återförsök kan belastningen multipliceras.

Återförsök bör därför användas selektivt, med begränsat antal försök, ökande väntetider och viss slumpmässig variation så att anropare inte försöker samtidigt. Fel som tydligt är permanenta, såsom bristande behörighet eller felaktigt format, ska normalt inte upprepas. Operationer som förändrar data behöver dessutom stöd för **idempotens**, exempelvis en unik operationsidentifierare som mottagaren kan känna igen, eller annan mekanism som förhindrar oönskad dubbel effekt vid upprepning. Detta skydd måste omfatta även fall där ett svar försvinner efter genomförd operation.

Det centrala är att anropande tjänst behöver avgöra *vilken sorts fel* som inträffat och *vilken konsekvens* ett nytt försök kan få. Ett generellt återförsök på varje fel är ingen robusthetsstrategi.

Att förhindra att ett fel sprider sig

Feltolerans kan beskrivas som flera kompletterande skydd, inte som en enskild teknisk produkt.

En **circuit breaker** kan tillfälligt sluta skicka anrop till ett beroende som upprepade gånger misslyckas. Det skyddar anroparens resurser och kan ge det felande beroendet utrymme att återhämta sig. Funktionen måste samtidigt kunna pröva när kommunikationen åter kan öppnas. Den reparerar inte det underliggande felet. Trösklar och återöppning behöver anpassas så att skyddet inte blir kvar i ett onödigt blockerande läge.

Resursisolering kan begränsa hur många samtidiga anrop eller arbeten ett beroende får belasta. Om dokumentlistan tar lång tid ska den helst inte kunna förbruka alla resurser som behövs för att läsa ärendeinformation eller spara andra uppgifter. Isoleringen kan finnas i applikationens anslutningspooler, arbetsköer och samtidighetsgränser, men också i plattformens resursgränser och kapacitetsregler.

Belastningsbegränsning och regler för att avvisa anrop vid överlast kan vara bättre än att acceptera mer arbete än tjänsten kan slutföra. Det förutsätter att klienter och användargränssnitt får begripliga signaler om vad som hände och om ett nytt försök är lämpligt.

Gemensamt för skydden är att de behövs *före* en incident. Ett team som upptäcker behovet av timeout, resursisolering och kontrollerade återförsök först under pågående avbrott får svårare att undvika följdproblem.

Allt behöver inte sluta fungera samtidigt

I vårt exempel kan ärendeinformation fortfarande visas när dokumenttjänsten inte kan nås. Då kan användargränssnittet visa ärendet och tydligt ange: **”Dokument kan inte hämtas just nu.”** Det är något annat än att visa en tom dokumentlista. En tom lista säger i praktiken att ärendet saknar dokument, vilket kan vara fel och få verksamhetsmässiga följder.

Detta kallas ofta **kontrollerad degradering**: den del av funktionen som är tillgänglig fortsätter att fungera, samtidigt som användaren får korrekt information om begränsningen. I vissa fall kan en tidigare hämtad uppgift visas, men då måste dess ålder och tillförlitlighet vara tydliga. För ett rättsligt eller säkerhetskritiskt beslut kan det vara olämpligt att visa föråldrade data som om de vore aktuella.

Andra funktioner bör däremot stoppa helt när ett beroende fallerar. Om verksamhetsreglerna kräver att dokument verkligen har lagrats och registrerats innan ett ärende skickas vidare, får systemet inte rapportera genomförd registrering på grundval av enbart ett skickat anrop. Valet mellan degradering, köad behandling och stopp beror på verksamhetens krav – inte på att den ena tekniken alltid är bättre.

När ett meddelande blir liggande

För asynkron kommunikation ser felet annorlunda ut. Ärendetjänsten kan ha skickat en händelse om att ett dokument ska bearbetas, samtidigt som dokumenttjänsten är otillgänglig. Meddelandet kan då ligga kvar i en kö eller ström, under förutsättning att infrastrukturen är byggd för det. Användaren behöver veta om dokumentet väntar på behandling, misslyckats eller är färdigt.

Köer kopplar loss två tjänsters exekveringstidpunkter men tar inte bort fel. En växande kö kan signalera att mottagaren inte hinner med. Vissa meddelanden kan behöva hanteras flera gånger, komma i annan ordning än väntat eller hamna i en särskild hantering för upprepade fel. Systemet behöver bestämda regler för fördröjningar, återförsök, idempotens, felaktiga meddelanden och hur verksamheten kan rätta ett fastnat flöde.

En kö kan alltså förbättra tåligheten vid tillfälliga avbrott, men den introducerar ett annat slags ansvar: att upptäcka och hantera arbete som inte blivit klart. Även meddelandets lagringstid, kökapacitet och

återstart efter längre avbrott behöver vara avtalade; annars kan arbete gå förlorat eller ansamlas snabbare än det kan behandlas.

Hälsa är inte detsamma som verksamhetsförmåga

En plattform kan övervaka om dokumenttjänstens instanser körs och om de kan ta emot trafik. Sådana hälsokontroller är viktiga, men de räcker inte för att avgöra om handläggaren faktiskt kan öppna ett dokument. En instans kan svara på hälsokontroller samtidigt som dess datalager är mycket långsamt eller ett beroende är felkonfigurerat.

Hälsokontroller för **om instansen lever** och **om den bör få trafik** behöver utformas med eftertanke. Om beredskapskontrollen gör varje instans oanvändbar så snart ett gemensamt externt beroende fallerar, kan hela tjänsten tas ur trafik utan att någon annan instans kan hjälpa. Samtidigt får en instans som faktiskt inte kan ta emot begäran inte felaktigt markeras redo.

Utöver tekniska kontroller behöver organisationen följa verkliga verksamhetsflöden: går det att öppna ett ärende, läsa dess dokument och slutföra en registrering? Metoderna för hur detta mäts och visualiseras behandlas i nästa kapitel om observerbarhet.

Vem ansvarar för att tjänsten klarar störningar?

I en traditionell miljö kan driftorganisationen vara van att hantera tillgänglighet genom redundanta VM:ar, lastbalanserare och omstarter. Dessa åtgärder behövs ofta fortfarande. Men de kan inte på egen hand avgöra hur ett användargränssnitt ska reagera när ett beroende svarar för sent eller när en ändring kanske redan har utförts.

Applikationsteamet behöver därför beskriva vilka beroenden som är kritiska, välja tidsgränser och återförsök, utforma ett säkert användarbeteende samt se till att operationer fungerar när svar försvinner eller levereras flera gånger. **Drift- och plattformsteam** behöver bidra med kunskap om verkliga felmoder, kapacitetsgränser, redundans, trafikstyrning, resursisolering och återhämtningsmekanismer. **Verksamhetsansvariga** behöver ange vilka funktioner som måste fungera, hur länge

avbrott kan tolereras och när en osäker status är oacceptabel.

Ingen av dessa roller kan ensam garantera helheten. Kraven måste omsättas i gemensamt beslutade scenarier och verifieras genom feltester, exempelvis att stoppa en tjänsteinstans, blockera en anslutning eller begränsa kapaciteten i en kontrollerad miljö. Feltester i produktion kan också vara relevanta, men bara under överenskomna säkerhets- och riskvillkor. Vem som får initiera sådana tester och vem som leder en verklig incident behöver vara explicit bestämt. Det behöver också finnas överenskomna mål för hur mycket funktionalitet som får försämras och hur snabbt normal funktion ska återupptas.

Tillbaka till ärendehanteringen

När dokumenttjänsten i vårt exempel blir långsam ska ärendevyn inte vänta obegränsat. Ärendetjänsten använder en rimlig tidsgräns och begränsar hur många samtidiga anrop den skickar. Vid tillfälliga fel kan vissa läsanrop göras om enligt en begränsad policy. Om problemen fortsätter visas ärendeuppgifterna, men dokumentavsnittet ersätts av ett tydligt felmeddelande som inte påstår att ärendet saknar dokument.

När dokument ska registreras gäller strängare regler. Ärendehanteringen får inte visa att registreringen är avslutad om resultatet är okänt. I stället kan status vara *behandlas* eller *behöver kontrolleras*, med möjlighet till uppföljning och säker återupptagning. Plattformen bidrar med kapacitets- och nätverksmekanismer; tjänsterna bidrar med korrekt felbeteende. Verksamhetsansvariga avgör vilken begränsad funktionalitet som är acceptabel.

Den stora förändringen är inte att fel plötsligt uppstår – det gjorde de även i den traditionella miljön. Förändringen är att **fel i en del av systemet i större utsträckning måste hanteras som en normal förutsättning för hur hela tjänsten byggs och drivs**. Här har vi behandlat **hur tjänster utformas för att begränsa konsekvenserna**. Nästa kapitel handlar om hur organisationen **upptäcker och mäter påverkan**; först därefter går vi vidare till hur en verklig incident leds och åtgärdas.

Källor och vidare läsning

- **R14** AWS Prescriptive Guidance, Retry with backoff pattern (kontrollerad 2026-10-17).

13. Hur vet vi att hela IT-stödet fungerar?

Alla servrar visar grönt. Ändå kan användarna inte bifoga dokument till sina ärenden. Det är ett scenario som visar varför teknisk övervakning behöver kompletteras med förståelse för verksamhetsflöden. I en distribuerad arkitektur kan varje tjänst fungera tekniskt medan samspelet mellan dem misslyckas.

Från komponentstatus till verksamhetsfunktion

I ett traditionellt system övervakas ofta webbserver, backend, databas och nätverk. Sådana signaler behövs fortfarande. Men när ärendehantering, dokumenthantering och notifiering blir separata tjänster måste organisationen kunna skilja mellan att en process körs, att en tjänst behandlar anrop korrekt och att användaren faktiskt kan fullfölja sin arbetsuppgift.

I vårt ärendehanteringssystem vill användaren bifoga en fil. Ärendetjänsten registrerar begäran, dokumenttjänsten lagrar filen och en senare händelse uppdaterar ärendets dokumentlista. Alla komponenterna kan rapportera normal status även om uppdateringen är fördröjd och användaren inte kan se sitt dokument. Ett relevant mått på funktionen behöver därför omfatta hela förloppet – inte bara om dess enskilda delar svarar. Det ska dessutom skilja mellan en uppladdning som fortfarande behandlas, en som misslyckats och en som slutförts.

Det är viktigt att inte överdriva skillnaden mot monoliter. Även ett tradi-

tionellt system kan behöva övervakning av användarflöden. Mikrotjänster gör behovet mer påtagligt genom fler nätverksberoenden, fler körande instanser och självständiga förändringar.

Loggar, mätvärden och distribuerade spår

Tre typer av signaler kompletterar varandra. **Mätvärden** visar exempelvis antal begäranden, felandel, svarstid och ködjup över tid. **Loggar** beskriver enskilda händelser, såsom avvisad uppladdning eller misslyckad databasoperation. **Distribuerade spår** följer delar av en begäran när den passerar mellan olika tjänster.

Ett mätvärde kan visa att registreringstiden ökar. Medelvärden kan samtidigt dölja långa väntetider för en mindre grupp användare; därför kan också fördelningar och höga percentiler behövas. ett spår visa att merparten av tiden tillbringas i dokumenttjänstens lagringsanrop och en logg att ett enskilt försök tidsbegränsades. Inget av verktygen ger ensamt hela förklaringen.

För applikationssidan innebär detta att instrumenteringen behöver byggas in i tjänsterna. Strukturerade loggar och konsekventa felkoder ska förklara vad tjänsten försökte göra. För driftsidan innebär det att gemensamma system måste kunna samla in, söka och relatera signaler från tjänster som byter instans och adress över tid. R15

Korrelation mellan tjänster

När en begäran passerar från ärendetjänsten till dokumenttjänsten behöver det finnas ett sätt att koppla samman händelserna. Ett korrelations-ID eller en distribuerad spårkontext kan föras vidare i kommunikationen. Vid asynkrona meddelanden behövs ibland samband mellan ursprunglig begäran och senare arbete, snarare än en enda sammanhängande kedja. Om ett meddelande behandlas flera gånger ska detta kunna följas utan att varje försök felaktigt räknas som en ny verksamhetsoperation.

Detta måste utformas gemensamt. Om varje team hittar på egna fältnamn och identifierare blir felsökning över tjänstegränser besvärlig. Samtidigt ska spårbarhet inte uppnås genom att personuppgifter, dokumentinnehåll eller autentiseringstoken rutinmässigt lagras i loggar.

Vad är ett fungerande ärendeflöde?

En tjänstenivåindikator (SLI) beskriver något som går att mäta, exempelvis andelen lyckade dokumentuppladdningar eller tiden från uppladdning till synlig dokumentpost. Ett tjänstenivåmål (SLO) anger vilken nivå man eftersträvar under en bestämd period. Ett SLO är inte automatiskt ett avtalsmässigt åtagande (SLA). För ett asynkront flöde kan ett mottagningskvitto vara tekniskt lyckat trots att verksamhetsåtgärden ännu inte är genomförd; båda tillstånden behöver kunna observeras.

Definitionen behöver utgå från verksamheten. Räknas en uppladdning som lyckad när dokumenttjänsten tar emot filen, när den lagrats eller först när användaren ser den i ärendet? Om vi väljer fel definition kan övervakningen visa ett gott resultat samtidigt som användarna upplever återkommande problem.

Alla funktioner behöver inte ha samma mål. För vissa ärendeflöden måste resultat synas omedelbart. För andra kan en fördröjd notifiering vara acceptabel. Verksamhetsansvariga behöver bidra med prioriteringar, medan applikations- och driftansvariga översätter dem till mätbara indikatorer och genomförbara kontroller.

Syntetiska tester och verkliga användarsignaler

En syntetisk kontroll kan med jämna mellanrum prova ett viktigt arbetsflöde: skapa ett testärende, bifoga en ofarlig fil och kontrollera att den visas. Det gör det möjligt att upptäcka fel som serverövervakningen missar. Testdata behöver dock vara tydligt märkta, begränsade och rensas bort så att kontrollerna inte förorenar den verkliga verksamheten.

Signaler från verkliga användare kan visa hur ofta ett flöde misslyckas och hur länge de får vänta. De måste tolkas med hänsyn till klienter och nätverksförhållanden och hanteras med lämpligt integritetsskydd. Syntetiska och verkliga signaler kompletterar därför varandra.

När gröna tjänster ger en röd helhet

Anta att dokumenttjänsten tar emot uppladdningar men att kön för efterföljande registrering växer. Ärendetjänsten visar att den får svar och

dokumenttjänsten rapporterar att den är igång. Användarna ser ändå inte de uppladdade filerna inom utlovad tid.

Ett övergripande mått på genomförda dokumentregistreringar visar problemet. Kömått visar eftersläpningen, medan spår och loggar hjälper teamen att förstå varför. Applikationsteamet behöver analysera arbetsflödets logik och hur fel eller fördröjningar hanteras. Plattform och drift behöver undersöka resurser, lagring, kommunikation och köernas beteende. Ingen av grupperna kan ersätta den andras kunskap.

Observerbarheten löser inte störningen i sig, men kan göra den upptäckbar och begriplig innan den växer till en större incident.

Färre men bättre larm

Fler mikrotjänster kan innebära mängder av mätvärden och larm. Om varje tillfällig omstart utlöser ett akut larm riskerar driftorganisationen att drunkna i signaler. Larm bör därför prioriteras efter faktisk eller förväntad påverkan på verksamhetsflöden och efter om någon behöver agera.

Ett användbart larm anger vad som försämrats, vilket mål som hotas, vem som ska ta emot det och vilka första kontroller som är relevanta. Förebyggande tekniska larm kan vara motiverade innan användare hunnit påverkas, men bör ha begripliga åtgärdströsklar och en tydlig mottagare. Tekniska larm är fortfarande nödvändiga, men de behöver relateras till tjänstens och verksamhetens betydelse. Hur larm eskaleras och hur incidenter leds behandlas i nästa kapitel.

Observerbarhet har också en kostnad och en risk

Att samla allt är inte ett mål i sig. Hög loggvolym, långt bevarande och omfattande spårning kan bli kostsamt. Det kan också skapa säkerhets- och integritetsproblem. Samplingsbeslut bör beakta att sällsynta men allvarliga fel annars riskerar att missas. För kritiska händelser kan separata och mer fullständiga signaler behövas. Organisationen behöver därför besluta vad som är nödvändigt, hur länge det lagras, vilka som får läsa uppgifterna och vad som kan reduceras eller samplas utan att viktiga fel blir osynliga.

Även observabilitetsplattformen kan få störningar. Dess fel bör så långt möjligt inte stoppa verksamhetsfunktionerna, och kritiska larm kan behöva alternativa vägar.

Vem ansvarar för helhetsbilden?

En möjlig ansvarsfördelning är att tjänsteteamen ansvarar för verksamhetsrelevanta signaler och korrekt instrumentering, att plattformsteamet erbjuder insamling, sökning, lagring och behörighetsstyrning, och att ansvarig för det samlade IT-stödet säkerställer att kritiska användarflöden faktiskt går att följa upp. Driftorganisationen tillför kunskap om underliggande resurser och gemensamma beroenden. Säkerhets- och informationsansvariga anger ramar för känsliga uppgifter.

Det viktiga är inte att placera samtliga uppgifter i en viss organisation, utan att göra varje gränssnitt explicit: vem definierar ett mått, vem implementerar det, vem larmas, vem får ändra larmregler, vem hanterar insamlingsplattformens egna avbrott och vem följer upp att verksamhetsmålet nås?

För **applikationsläsaren** är lärdomen att driftbarhet och synligt beteende måste vara en del av tjänstens design. För **driftläsaren** är lärdomen att status för VM:ar och processer behöver kompletteras med signaler från de faktiska verksamhetsflödena. För båda är den avgörande frågan densamma: kan vi tillsammans avgöra om användarna får den funktion de behöver? **Observerbarheten hjälper oss att upptäcka och avgränsa påverkan**, men är inte samma sak som de felbegränsande mekanismerna i kapitel 12 eller den samordnade incidenthanteringen i kapitel 14. Den övergången är viktig: ett larm blir värdefullt först när någon kan bedöma det och vid behov agera.

Källor och vidare läsning

- **R15** OpenTelemetry, Context propagation (kontrollerad 2026-10-17).

14. Hur felsöker och hanterar vi incidenter?

En morgon går det inte att öppna dokument från ärendehanteringen. Användarna ser meddelandet *Dokumentet kan inte hämtas*. Ärendetjänsten svarar på sina hälsokontroller. Dokumenttjänsten gör det också. Driftcentralens sammanställning är huvudsakligen grön, men arbetet på handläggningsenheten står delvis stilla.

I en traditionell applikation finns ofta en gemensam förvaltnings- och driftkedja som kan samla in loggar från ett begränsat antal servrar och undersöka en gemensam release. I vårt distribuerade system måste en incident i stället undersökas över flera tjänster, nätverksförbindelser och organisatoriska ansvarsgränser. Det betyder inte att incidentledning är något nytt, utan att det blir viktigare att kunna samordna den oavsett vilken del som visar sig orsaka felet.

Börja med verksamhetspåverkan, inte med den misstänkta komponenten

Den som tar emot larmet vet sällan vilken tjänst som orsakat störningen. Ett alltför tidigt antagande om att det är en applikationsbugg eller ett nätverksfel riskerar att styra felsökningen fel. Börja därför med frågorna: Vilka användare och arbetsmoment påverkas? När började det? Är felet totalt, intermittent eller begränsat till vissa ärenden? Finns risk för informationsförlust eller felaktiga beslut?

Allvarlighetsgrad och första prioritet bör grundas i verksamhetspåverkan, tidskritikalitet och risk för felaktig eller förlorad information, inte bara i

antal larm. En dokumenterad tidslinje med klockslag, gjorda ändringar, hypoteser och beslut underlättar både samordningen och efteranalysen.

Den första lägesbilden ska kunna delas mellan applikationsteam, drift, plattform, informationssäkerhet och verksamhetsansvariga. Den behöver vara tillräckligt begriplig för att stödja prioritering, utan att redan innehålla en obestyrkt rotorsak. En incident med få tekniska felmeddelanden kan vara kritisk om ett centralt verksamhetsflöde är blockerat.

För ärendehanteringen fastställer incidentledaren exempelvis att dokument inte kan öppnas för en viss grupp användare, medan registrering av nya ärenden fungerar. Det ger en konkret avgränsning: funktionell åtkomst till dokument är problemet, inte allmän serverdrift. R16

Incidentledning är ett eget ansvar

Flera team kan arbeta samtidigt utan att någon av dem ensam har mandat att prioritera helheten. Därför behöver en utsedd incidentledare hålla samman lägesbild, beslut, resursfördelning, eskalering och kommunikation. Rollen kan ligga hos en central driftfunktion, en tjänsteorganisation eller en särskilt utsedd beredskapsfunktion. Det avgörande är att mandatet är tydligt innan störningen inträffar. Incidentledningen samordnar prioritering och riskbeslut men ska inte ge sig själv obegränsad rätt att ändra produktionssystem; behörigheter och tekniskt verkställande ansvar behöver vara uttryckligen delegerade.

Incidentledaren måste inte vara den tekniskt skickligaste felsökaren. Tvärtom är det ofta en fördel om tekniska experter kan koncentrera sig på diagnos och återställning medan incidentledningen dokumenterar beslut, säkerställer kommunikation och avgör när ytterligare kompetens ska kallas in.

Ett möjligt upplägg skiljer på tre roller: incidentledare, tekniska felsökare och ansvarig för kommunikation till användare och verksamhet. I mindre organisationer kan samma person fylla flera roller, men uppgifterna bör fortfarande vara uttalade. Vid en säkerhetsincident kan ytterligare besluts- och rapporteringsvägar behövas; detta ersätter inte organisationens säkerhetsrutiner.

Felsökning över tjänste- och plattformsgränser

I vårt exempel rapporterar ärendetjänsten att anrop till dokumenttjänsten misslyckas. Dokumenttjänstens egna loggar visar emellertid inte motsvarande begäranden. En sådan skillnad är värdefull information: felet kan ligga före dokumenttjänstens applikationskod, exempelvis i DNS, trafikstyrning, certifikathantering, en nätverkspolicy eller ett felaktigt tjänstenamn.

Teamet bör kontrollera händelsekedjan systematiskt. Har användarens begäran nått ärendetjänsten? Vilken adress försökte ärendetjänsten använda? Blev namnet uppslaget och anslutningen etablerad? Har en lastbalanserare skickat trafiken vidare? Har målinstansen tagit emot anropet och returnerat ett svar? Observerbarheten från föregående kapitel hjälper här, men är ingen ersättning för kunnande om applikationsprotokoll och infrastruktur.

Anta att en nyligen genomförd ändring har pekat ärendetjänsten mot ett äldre tjänstenamn som inte längre finns i den aktuella miljön. Felet visar sig då vara en felkonfiguration i tjänstens kommunikationsadress – inte en bugg i dokumenthanteringen och inte nödvändigtvis ett fel i DNS-tjänstens tillgänglighet. Vilket team som genomför rättningen beror på vem som äger konfigurationen och förändringsflödet. Ansvar för att användarna får tillbaka funktionen får däremot inte försvinna under diskussionen om felorsaken.

Samma princip gäller andra typer av incidenter. En överfull meddelandekö kan vara följden av långsam applikationsbehandling eller resursbrist på plattformen. Ett högt antal databasanrop kan orsakas av en ny applikationsversion trots att databasservern fungerar enligt sina egna driftmått. Felsökningen behöver därför pröva hypoteser mot fakta snarare än följa organisatoriska gränser. Den senaste driftsättningen är en viktig ledtråd, men tidsmässigt samband är inte bevis på orsak. Kontrollera också miljöförändringar, certifikat, routing och beroenden som kan ha ändrats oberoende av applikationsreleasen.

Återställ tjänsten innan alla orsaker är kända

Under en pågående incident är målet först att begränsa skadan och återställa acceptabel verksamhetsfunktion. En fullständig rotorsaksanalys

kan komma efteråt. Möjliga åtgärder är att återställa en tidigare fungerande konfiguration, styra om trafik, minska belastning, skala en resurs eller tillfälligt stänga av ett icke-kritiskt flöde.

Men varje åtgärd har risker. Att starta om alla instanser samtidigt kan öka avbrottet. Att automatiskt försöka om ett misslyckat kommando kan ge dubbel behandling om operationen redan utförts. Att återställa en äldre programversion kan vara olämpligt efter en icke-bakåtkompatibel datamigrering. En säker återställningsstrategi måste därför vara förberedd och prövad i förväg, inte uppfinnas under press. Om flera team arbetar parallellt bör varje planerad ändring ha en ansvarig utförare, en tydlig förväntad effekt och ett sätt att verifiera utfallet innan nästa riskfyllda åtgärd görs.

För dokumentincidenten väljer teamen att återställa den tidigare fungerande tjänsteadressen via det ordinarie konfigurationsflödet. De kontrollerar därefter både den tekniska anslutningen och att en användare faktiskt kan öppna dokument från ett ärende. Ett grönt hälsotest räcker inte för att avsluta incidenten. Kontrollera också om misslyckade eller uppskjutna dokumentoperationer behöver återköras, stämmas av eller hanteras manuellt så att informationsläget blir korrekt. Återställd åtkomst och avvecklad incident betyder inte alltid att allt efterarbete är avslutat.

Beredskap, eskalering och vägar in till rätt kompetens

När tjänster utvecklas av olika team kan ett fel inträffa utanför deras ordinarie arbetstid. Organisationen måste därför välja en tydlig beredskapsmodell. Ett tjänsteteam kan ha egen jour, en central funktion kan vara första linjen med eskalering till tjänsteexperter, eller flera team kan dela en beredskapstjänst. Ingen modell följer automatiskt av att applikationen består av mikrotjänster.

Vilken modell som är rimlig påverkas av tillgänglighetskrav, kompetens, personberoende, kostnad och hur väl systemet kan återhämta sig automatiskt. Om applikationsteamet förväntas svara på produktionsincidenter måste detta stödjas av avtalade arbetstider, åtkomst till relevant telemetri, utbildning, fungerande beredskapsrutiner och mandat att genomföra säkra åtgärder. Det räcker inte att skriva *teamet äger tjänsten*.

Eskaleringen bör bygga på vilket problem som behöver lösas, inte enbart på vilket team ett larm först träffar. Säkra kontaktvägar, tjänstekatalog, beroendekarta och aktuell information om ansvariga behöver vara åtkomliga även när delar av den ordinarie IT-miljön är nere.

Kommunikation och ansvar gentemot användarna

Användaren bryr sig inte om huruvida avbrottet orsakas av dokumenttjänsten, nätverket eller plattformen. Den behöver veta vad som inte fungerar, vilka tillfälliga arbetssätt som finns, vilken risk som föreligger och när en ny lägesrapport kommer. Därför behöver verksamhetskommunikationen vara samlad även om den tekniska felsökningen är fördelad.

Incidentledningen ska undvika tvärsäkra orsaksbeskrivningar innan de är verifierade. Det är bättre att säga att dokumentåtkomst är störd och att berörda team undersöker sambandet mellan tjänsterna än att tidigt peka ut en viss leverantör eller teknik. Om data kan ha gått förlorade eller behandlats felaktigt behöver dessa risker hanteras separat, med rätt verksamhets- och informationsansvariga.

Lärande efter incidenten

När dokumentåtkomsten åter fungerar behöver organisationen förstå varför felet kunde uppstå, varför det inte upptäcktes tidigare och varför återställningen tog den tid den gjorde. En efteranalys bör omfatta både tekniska och organisatoriska faktorer: en saknad kontroll av tjänsteadresser, ett otydligt ägarskap för konfigurationen, bristande test av miljöändringar eller svårigheter att nå rätt kompetens.

En bra analys söker förbättringar av systemet och arbetssättet framför att leta syndabockar. Det kan innebära ett automatiserat kontraktstest, en säkrare konfigurationsvalidering, en förbättrad larmregel eller en förändrad eskaleringsrutin. Efteranalysen bör skilja på säkerställda fakta, rimliga hypoteser och sådant som fortfarande är okänt. Varje åtgärd behöver en ansvarig och en uppföljning; annars riskerar slutsatserna att stanna i mötesanteckningarna.

Det bör också finnas en gräns mot förändringshantering. Ett akut återställande ska dokumenteras, men inte nödvändigtvis tvingas genom sam-

ma beslutsförfarande som en planerad större ombyggnad. Förhandsbeslutade nödrutiner kan förena handlingsförmåga med kontroll och efterföljande granskning.

Vad behöver applikations- och driftsidan göra annorlunda?

För **applikationssidan** innebär distribuerad incidenthantering att teamen måste kunna beskriva sina beroenden och deras felbeteenden, bidra med verksamhetsnära diagnostik och delta i återställning när den egna tjänsten eller dess kontrakt är inblandade. Det gäller även om infrastrukturen sköts av ett annat team.

För **driftsidan** innebär det att felsökningen inte kan stanna vid att varje server och plattformsdel är tekniskt frisk. Drift- och plattformsexperter behöver förstå användarflöden, tjänsteadresser, trafikvägar och förändringar som införts av olika leveransteam. Deras erfarenhet av strukturerad incidentledning och infrastrukturella beroenden blir därför fortsatt central.

Det **gemensamma ansvaret** är att utse vem som leder en incident, vem som kan ändra vad, vem som kommunicerar med verksamheten och hur man verifierar att hela funktionen återställts. Dessa beslut måste finnas före incidenten, även om den exakta tekniska felorsaken inte kan förutses.

En fungerande helhet även under störning

De skyddsmekanismer vi beskrev i kapitel 12 kan begränsa ett fel, och signalerna från kapitel 13 kan visa dess påverkan. Men **under en pågående incident behövs dessutom samordnade beslut**, eftersom flera team kan behöva agera samtidigt. Mikrotjänster kan göra felsökningen mer komplicerad även när de enskilda tjänsterna återhämtar sig självständigt. Det avgörande är inte om varje team har kontroll över sin egen komponent, utan om organisationen som helhet kan upptäcka påverkan, samordna åtgärder och återställa det IT-stöd som användaren är beroende av.

Nästa kapitel fördjupar en närliggande fråga: hur säkerställer vi att kommunikation och informationsåtkomst är säkra när identiteter och behö-

righeter behöver fungera över flera tjänstegränser?

Källor och vidare läsning

- **R16** Google SRE Workbook, Incident Response (kontrollerad 2026-10-17).

15. Hur skyddar vi systemet och dess information?

När det traditionella ärendehanteringssystemet ligger bakom en gemensam webbserver, i ett avgränsat nät och med en central databas, är det lätt att tänka på säkerhet som ett skydd runt systemet. Brandväggar, serverhärdning, användarinloggning och databasbehörigheter är fortfarande viktiga. Men när dokumenthanteringen och andra funktioner blir egna mikrotjänster uppstår fler kommunikationsvägar, fler tekniska identiteter och fler ställen där behörighetsbeslut fattas.

Det betyder inte att mikrotjänster per definition är osäkrare än en monolit. Skillnaden är att säkerhetsmodellen måste omfatta **relationerna mellan tjänsterna**, inte bara gränsen mellan användare och system. En intern anropare är inte automatiskt betrodd. En lyckad inloggning innebär inte heller att användaren får läsa varje dokument som en tjänst kan nå.

Från en skyddad systemgräns till flera tillitsgränser

I vårt exempel loggar en handläggare in i ärendestödet och öppnar ett ärende med tillhörande dokument. Det grafiska gränssnittet kontaktar ärendetjänsten, som i sin tur kan hämta dokumentmetadata från dokumenttjänsten. Själva filen kan ligga i en lagringstjänst som administreras av plattformen.

Varje övergång innebär frågor: Vem gör anropet? Vilket ärende gäller det? Vilket tillstånd har ärendet? Har användaren rätt att se just detta dokument? Vilka tjänster ska kunna begära eller leverera filens innehåll? Och hur upptäcker vi ett otillåtet försök?

En ändamålsenlig utgångspunkt är att identifiera tillitsgränser och hot utifrån informationens skyddsvärde. Nätverkssegmentering kan minska vilka komponenter som når varandra, men den ersätter inte identitetskontroll och behörighetsprövning. Det gäller även trafik som aldrig lämnar det interna nätet. R17

Autentisering är inte auktorisation

Autentisering handlar om att fastställa vem en användare eller tjänst är. **Auktorisation** handlar om vad den identifierade aktören får göra i det aktuella sammanhanget. De besluten behöver hållas isär.

Användaren kan autentiseras genom en gemensam identitetslösning. Det betyder inte att varje mikrotjänst behöver hantera användarens lösenord. I stället kan tjänsterna få verifierbara uppgifter om identitet och session genom en väl definierad säkerhetsarkitektur. En token måste kontrolleras för bland annat signatur eller annan föreskriven verifiering, giltighetstid, utfärdare, avsedd mottagare och relevanta rättigheter. Dessa kontroller måste göras på rätt plats i anropskedjan; att en tidigare tjänst redan har validerat användaren räcker inte utan ett uttryckligt, säkrat tillitskontrakt. Tjänster ska inte godta en token enbart därför att den verkar vara internt utfärdad.

Behörighetsreglerna måste dessutom ta hänsyn till verksamhetens objekt. Dokumenttjänsten får inte lämna ut en fil bara för att den som gör anropet känner till dokumentets identifierare. Åtkomsten kan bero på ärendetillhörighet, användarens roll, sekretessmarkeringar och ärendets aktuella tillstånd. Här behövs överenskommelser mellan verksamhetsansvariga och de team som äger ärende- respektive dokumentinformation.

Om ärendetjänsten gör anropet å en användares vägnar behöver dokumenttjänsten få tillräckligt underlag för ett korrekt behörighetsbeslut. Det kan ske genom en uttryckligt avgränsad delegering eller genom ett betrott behörighetsbeslut som dokumenttjänsten kan kontrollera. Att okritiskt vidarebefordra en token avsedd för ärendetjänsten till dokumenttjänsten riskerar att bryta mot tokenens mottagarbegränsning. Att vidarebefordra användarkontext är dock inte samma sak som att ge al-

la interna tjänster samma obegränsade befogenheter. Kontraktet måste ange vem som får anropa, för vilket ändamål och hur rättigheter begränsas. R18

Tjänster behöver egna identiteter

När en monolit delas upp ökar antalet anrop mellan programkomponenter. Dokumenttjänsten kan behöva läsa från lagring, publicera ett meddelande och anropa en separat skanningstjänst. Dessa anrop bör utföras med identifierbara tjänsteidentiteter och med **minsta nödvändiga behörighet**.

En tjänsteidentitet är inte detsamma som en användares identitet. Den beskriver vilken applikationskomponent som utför ett tekniskt anrop. För vissa flöden behövs båda perspektiven: dokumenttjänsten autentiserar ärendetjänsten som teknisk klient och kontrollerar samtidigt vilken användare eller verksamhetsprocess anropet gäller.

Tekniska identiteter kan bygga på flera olika mekanismer, till exempel kortlivade autentiseringsuppgifter, certifikat eller signerade token. Ömsesidig TLS kan autentisera tjänster och skydda kommunikationen, men löser inte ensam frågan om vilka dokument en handläggare får se. Kryptering, identitet och behörighet är kompletterande skydd, inte utbytbara lösningar.

Nätverk och kryptering i en distribuerad miljö

Drift- och nätverksorganisationen arbetar traditionellt med separata nät, brandväggsregler, lastbalanserare, DNS och certifikat. De förmågorna finns kvar. Skillnaden är att antalet tjänster och tillåtna kommunikationsrelationer kan förändras oftare. En statisk lista över serveradresser blir då svårare att underhålla än tydliga regler baserade på tjänsternas roller och behov.

Det behöver vara känt vilka anropsvägar som är avsedda: från webbläsare till ingångstjänst, från ärendetjänst till dokumenttjänst och från dokumenttjänst till lagringsresurs. En tjänst ska inte nå alla andra tjänster bara för att de delar samma kluster eller virtuella nät.

Kryptering under överföring skyddar mot insyn och manipulation på

vägen, medan skydd av lagrad information hanterar andra risker. Nycklar, certifikat och förtroendekedjor måste kunna förnyas, roteras och vid behov återkallas med planerad överlappning mellan gamla och nya uppgifter. Rotation ska provas mot beroende tjänster så att den inte oavsiktligt orsakar driftstopp. Valet av teknik och kontrollernas omfattning styrs av risk, miljö och organisationens krav, inte av att arkitekturen råkar vara uppdelad i mikrotjänster.

Hemligheter, konfiguration och leveranskedjor

Lösenord till databaser, API-nycklar och privata nycklar får inte hårdkodas i källkod eller skickas med i en containeravbild. De bör förmedlas genom en kontrollerad mekanism med spårbar åtkomst, definierad livslängd och rutiner för rotation. Att en hemlighet ligger i en miljövariabel eller ett konfigurationslager gör den inte automatiskt säker; åtkomst till körmiljö, loggar och felsökningsverktyg spelar också roll.

Säkerhet behöver också omfatta **hur programvaran byggs och levereras**. Beroenden, byggverktyg, artefaktregister och automatiserade driftsättningar kan annars bli vägar in i produktionsmiljön. Teamen bör kunna se vilken källkod och vilka beroenden en levererad version bygger på, och vilka kontroller den passerat. Kodgranskning, sårbarhetshantering och separerade behörigheter är exempel på kontroller som kan minska riskerna.

Detta är ett gemensamt ansvar. Applikationsteamet väljer och underhåller sina beroenden och sin kod. Plattformen erbjuder skyddade bygg- och körmiljöer. Säkerhetsfunktionen hjälper till att fastställa risknivåer och kontrollkrav. Driftorganisationen bidrar med kontroll över produktionsåtkomst och övervakning av infrastrukturen. Ingen av dessa delar kan ensam garantera en säker leverans.

Informationens skydd följer med över tjänstegränsen

I det genomgående exemplet kan ett dokument innehålla sekretessbelagda uppgifter även när dess metadata presenteras i ärendelistan. Därför behöver informationsklassning och åtkomstregler följas upp både i ären-

detjänstens API och i dokumenttjänstens lagring och nedladdningsväg.

Ett vanligt misstag är att kontrollera åtkomst när en dokumentlänk skapas men inte när filen faktiskt hämtas. Om nedladdning sker via en tidsbegränsad, signerad länk behöver länken ha snäv giltighet och omfattning, och organisationen måste ta ställning till hur återkallelse, vidarebefordran och loggning hanteras. En signerad länk ger inte i sig en ny prövning av användarens aktuella behörighet. En annan risk är att ett meddelande eller en loggrad innehåller mer information än mottagaren behöver. Genom att minimera uppgifter i meddelanden, loggar och tekniska spår minskar den sammanlagda exponeringen.

Även tillfälliga filer, säkerhetskopior och testmiljöer behöver ingå i skyddsmodellen. Det räcker inte att den primära databasen har rätt behörigheter om samma information finns i en oskyddad export eller i en felsökningslogg.

Hotmodellering och säkerhetstester

När ett nytt kommunikationsflöde införs bör teamen identifiera vad som kan gå fel. Hur skulle en obehörig tjänst försöka läsa ett dokument? Kan en handläggare byta ett dokument-ID i en begäran och därmed nå ett annat ärendes fil? Vad händer om en token stjäls eller en intern komponent får mer behörighet än den behöver?

Sådana frågor är exempel på hotmodellering. Arbetet kan genomföras i olika former, men behöver anknyta till verkliga dataflöden och tillitsgränser. Säkerhetstester bör omfatta både nekad åtkomst, otillåten åtkomst mellan verksamhetsobjekt och felaktiga tjänsteidentiteter. Det är viktigt att verifiera att systemet avvisar förbjudna åtgärder, inte bara att vanliga användarflöden fungerar.

Automatiska tester hjälper, men ersätter inte granskning av arkitektur och driftkonfiguration. En felaktig nätverkspolicy eller överdriven produktionsbehörighet kan ligga utanför applikationens testsvit. Därför behöver provning ske på flera nivåer.

Säkerhetshändelser och incidenthantering

Om dokumenttjänsten upptäcker upprepade obehöriga nedladdningsförsök behövs loggar som visar förloppet utan att själva exponera innehål-

let. Säkerhetsrelevanta loggar behöver skyddas mot obehörig ändring, få ändamålsenlig åtkomst- och gallringsstyrning och kunna korreleras över tjänstegränser. Säkerhetsfunktion och driftorganisation kan behöva samordna isolering av en identitet eller en tjänst, medan applikationsteamet analyserar om en behörighetsregel kan kringgås.

Säkerhetsincidenter kräver ibland andra prioriteringar än vanliga driftsstörningar: att omedelbart återställa tillgänglighet är inte alltid rätt första steg om det riskerar att förstöra bevis eller förvärra informationsläckage. Incidentmandat, rapporteringsvägar och ansvar för tekniska åtgärder behöver därför vara överenskomna i förväg. De generella arbetsformerna för samordnade incidenter behandlades i kapitel 14.

Vem ansvarar för säkerheten i helheten?

Säkerhet i ett distribuerat system blir svag om varje team antar att någon annan ansvarar för gränsen mellan tjänsterna. Ansvarsfördelningen bör därför beskriva både kravställning, genomförande och verifiering.

Verksamhets- och informationsansvariga avgör vilken information som är skyddsvärd och vilka åtkomstvillkor verksamheten kräver. **Applikationsteamen** inför kontroller i kod och API:er och säkerställer att behörighet prövas vid relevanta operationer. **Plattforms- och driftfunktioner** tillhandahåller och övervakar nät, identitetsmekanismer för tjänster, hemlighetshantering och säkra körmiljöer. **Säkerhetsfunktionen** stödjer riskanalys, gemensamma principer, oberoende uppföljning och hantering av avvikelser.

Organisationer kan fördela dessa uppgifter på olika sätt. Det avgörande är att beslut om exempelvis tokenvalidering, dokumentbehörighet, certifikatsrotation och loggåtkomst har uttryckliga ägare. För varje kritiskt skydd bör det också vara tydligt vem som verifierar att kontrollen faktiskt fungerar.

En gemensam säkerhetsmodell, inte en gemensam superbehörighet

När dokumenttjänsten bryts ut ur monoliten kan det vara lockande att ge ärendetjänsten obegränsad åtkomst till alla dokument för att förenkla integrationen. Det skulle minska friktionen på kort sikt men också utöka

konsekvensen av ett intrång eller ett programmeringsfel.

En hållbar lösning beskriver i stället vem som får begära vad, hur ett behörighetsbeslut fattas, hur kommunikationen skyddas och hur ett fel upptäcks. Gemensamma plattformsförmågor kan förenkla det arbetet, men ersätter inte verksamhetens regler för informationen.

Det är den centrala förändringen från monolit till mikrotjänster: säkerhet måste fortfarande fungera som en helhet, samtidigt som varje tjänst bär ett tydligt ansvar för sin del av skyddet.

Källor och vidare läsning

- **R17** OWASP, API1:2023 Broken Object Level Authorization (kontrollerad 2026-10-17).
- **R18** IETF, RFC 8725 JSON Web Token Best Current Practices (kontrollerad 2026-10-17).

16. Hur hanterar vi livscykler, underhåll och teknisk skuld?

När ärendehanteringssystemet var en sammanhållen applikation kunde organisationen ofta planera ett gemensamt versionsbyte: en ny applikationsversion, ett uppdaterat operativsystem och kanske en databasmigrering inom samma förändringsfönster. Det var inte alltid enkelt, men det fanns en tydlig leverans att förhålla sig till. Efter uppdelningen i ärendetjänst, dokumenttjänst och notifieringstjänst kan komponenterna förändras vid olika tidpunkter. Det är en viktig fördel, men även en förvaltningsutmaning.

Mikrotjänster gör inte underhåll överflödigt. De flyttar en del av ansvaret från en central releaseplan till flera samtidiga livscykler som måste fungera tillsammans. En tjänst kan vara nybyggd och ändå använda ett gammalt bibliotek. Om ingen längre ansvarar för dess underhåll kan dessutom själva tjänsten bli en säkerhetsrisk även om den sällan ändras. En stabil serverplattform kan fortfarande bära en tjänst vars API ingen längre vågar ändra. Frågan är därför inte bara *när* vi uppgraderar, utan **vem som ser helheten och hur vi upptäcker det som håller på att bli oförvaltningsbart**.

En tjänst har flera samtidiga livscykler

För dokumenttjänsten omfattar livscykeln åtminstone applikationskoden, dess bibliotek, språkets körmiljö, container- eller serveravbildning,

operativsystem, databasmotor, API-kontrakt och eventuella meddelandeformat. Den är dessutom beroende av certifikat, identiteter och de plattformstjänster som gör den körbar. Dessa delar kan ha olika leverantörer, supportvillkor och slutdatum för underhåll.

Ett ramverk som fortfarande fungerar kan ha slutat få säkerhetsuppdateringar. En databasversion som stöds av leverantören kanske inte längre stöds av ett drivrutinsbibliotek. Ett API kan vara tekniskt tillgängligt, men ha konsumenter som bygger på beteenden som aldrig dokumenterats. Att bara uppdatera en versionssiffra löser inte de sambanden.

En användbar livscykelöversikt behöver därför visa vilka komponenter som finns, vem som äger dem, vilka beroenden de har och vilka datum eller händelser som kräver beslut. Den måste omfatta både direkta och transitiva beroenden där sådana kan identifieras, och skilja mellan en komponent som finns i en byggartefakt och en som faktiskt körs i produktion. Den bör vara tillräckligt konkret för att driva åtgärder, inte bara en inventering som uppdateras vid en årlig revision.

Från central patchning till delat underhållsansvar

Tidigare kunde en driftorganisation patcha operativsystem och mellanprogramvara enligt en relativt enhetlig rutin. I en mikrotjänstmiljö kan plattformsteamet fortfarande ansvara för värdoperativsystem, kluster och förvaltade databastjänster. Men bibliotek och applikationsberoenden ligger ofta i varje tjänsts byggkedja. En containeravbildning har dessutom flera lager: en uppdaterad värd gör inte automatiskt en gammal basavbildning säker. Omvänt kan ett applikationsteam inte ensamt åtgärda en brist i den underliggande kluster- eller värdmiljön. Vem som patchar vad måste följa den verkliga driftsmodellen, inte en förenklad uppdelning mellan utveckling och drift.

Anta att tre tjänster använder samma föråldrade bibliotek. Säkerhetsfunktionen upptäcker en sårbarhet, men själva åtgärden kan behöva göras separat i tre kodbaser, testas mot tre uppsättningar beroenden och driftsättas vid tre olika tillfällen. Det gemensamma problemet kräver alltså både central överblick och lokalt genomförande.

En möjlig ansvarsfördelning är att säkerhetsfunktionen anger riskklassning och tidskrav, plattformsteamet erbjuder uppdaterade och godkän-

da baskomponenter, varje applikationsteam uppgraderar och testar sina beroenden och ansvarig för den samlade tjänsten följer upp att inga kritiska luckor återstår. Om ett team inte kan uppgradera direkt måste ett tidsbegränsat undantag ha en utsedd riskägare, kompletterande skyddsåtgärder och en beslutad plan. Ett undantag är inte samma sak som att risken försvunnit. Om en säkerhetsuppdatering innebär en kompatibilitetsrisk behöver riskägaren väga konsekvensen av att avvakta mot konsekvensen av en snabb ändring, med kontrollerade tester och en tydlig omprövningstidpunkt.

Beroendeförteckningar som faktiskt går att använda

Förvaltningen behöver kunna svara på vilka tjänster som påverkas när en sårbarhet eller en utfasning blir känd. Ett paketregister eller en programvaruförteckning kan visa vilka bibliotek som byggts in i en viss version. En tjänstekatalog kan visa ägare, driftmiljö, kontaktvägar, beroenden och kritikalitet. Den behöver även ange vem som tar vid när ett team upphör, eftersom ett tekniskt beroende ofta lever längre än den ursprungliga organisationen. En konfigurationsdatabas (CMDB) kan vara en del av denna helhetsbild, men den får inte förutsättas innehålla alla detaljer om varje byggartefakt.

Informationskällorna kompletterar varandra. Tjänstekatalogen svarar på *vem och vad*; bygg- och artefaktinformationen svarar på *vilken version*; driftens observationer svarar på *vad som faktiskt körs*. Om katalogen anger en uppdaterad version men produktionsmiljön fortfarande kör den gamla är förvaltningsproblemet inte löst. Därför behöver inventering och uppföljning knytas till verkliga driftsättningar och granskas vid avvikelser.

Även komponentförteckningar som en SBOM kan ge stöd, men de bevisar inte ensamma att programvaran är fri från sårbarheter. Resultaten behöver sättas i relation till hur komponenten används, om den exponerade funktionen faktiskt är nåbar och vilka skydd som finns runt tjänsten.

API:er och händelser lever längre än en enskild driftsättning

När dokumenttjänsten byter version måste ärendetjänsten fortfarande kunna anropa den. Den tekniska driftsättningen är bara ett steg i livscykeln för tjänstens gränssnitt. Ofta behöver en äldre och en nyare variant samexistera under en övergångsperiod.

Att lägga till ett valfritt fält kan ibland vara bakåtkompatibelt, men bara om konsumenterna hanterar okända fält och de faktiska valideringsreglerna tillåter ändringen. Att ta bort ett fält, ändra dess betydelse eller byta format på en händelse är ofta svårare. Asynkrona meddelanden kan dessutom ligga kvar i köer eller arkiv efter att producenten uppgraderats.

En praktisk modell är att först införa stöd för det nya beteendet, sedan migrera konsumenterna och först därefter ta bort den gamla varianten. För meddelandeflöden måste planen också omfatta meddelanden som producerats enligt äldre scheman men ännu inte behandlats. För databaser kan samma princip innebära att gammal och ny kod tillfälligt måste kunna använda samma datamodell. Under tiden behöver organisationen veta vilka versioner som används, vem som kontaktar beroende team och hur man ser att övergången är färdig. Att en äldre API-version är märkt som *utfasad* betyder inte att den kan stängas samma dag.

Versionshantering löser inte alla förändringsproblem: även samma version kan ändra svarstid, felmönster eller verksamhetsregler. Därför måste förändringsbedömningen omfatta både kontrakt och observerat beteende. Principerna för testning och kompatibilitet har behandlats i kapitel 9 och 10; här är frågan hur sådana gränssnitt **förvaltas över år**, även när ursprungsteamet ändras eller en leverantör lämnar organisationen. R19

Uppgraderingar som återkommande arbete

Om varje uppgradering skjuts upp tills något gått sönder byggs en skuld som blir svårare att hantera. Samtidigt kan helautomatiska uppdateringar av alla beroenden skapa en annan risk: kompatibilitetsbrott som upptäcks först vid körning. Automatisering behöver därför förenas med tester, stegvis införande och en möjlighet att stoppa en osäker förändring. Ett stort versionssprång kan kräva ändringar i kod, konfiguration, datamodell och driftsättningssätt samtidigt. Många mindre, kontrollera-

de uppgraderingar kan minska den risken, förutsatt att organisationen har fungerande automatiska tester, överblick över beroenden och tid avsatt för arbetet.

Det behövs därför en återkommande rytm för att granska uppdateringsbehov, planera underhåll och genomföra de förändringar som inte kan ske helt automatiskt. Vissa säkerhetsuppdateringar behöver hanteras snabbt; andra uppgraderingar kan samordnas med ordinarie utveckling. Risk, exponering, kompatibilitet och verksamhetens tolerans för avbrott avgör prioriteringen, inte en enda generell regel för alla tjänster.

Ett team som äger en mikrotjänst behöver ha utrymme att avsätta tid för denna sorts arbete. Annars kan självständiga team optimera för nya funktioner medan kostnaden för föråldrade komponenter hamnar hos framtida förvaltare eller hos driftorganisationen vid nästa incident.

Teknisk skuld är mer än gammal kod

Teknisk skuld kan bestå av otydliga tjänstegränser, manuella driftsättningar, svagt teststöd, odokumenterade beroenden och provisoriska undantag i nätverksregler eller behörigheter. En tjänst som kräver en unik och gammal körmiljö kan hindra en gemensam plattformsuppgradering. Ett nätverksundantag som skapades för en felsökning kan bli kvar tills ingen längre vet varför det finns.

Det betyder att teknisk skuld inte enbart kan prioriteras av applikationsutvecklare. Drift och plattform måste kunna synliggöra kostnaden för speciallösningar, blandade runtime-versioner, avvikande övervakning eller manuella åtgärder. Säkerhetsfunktionen behöver visa vilka avvikelser som höjer risken, och verksamheten behöver förstå hur skulden påverkar leveransförmåga och kontinuitet.

Ett bra beslutsunderlag beskriver *konsekvensen* av att låta skulden ligga kvar: längre ledtider för uppgraderingar, högre sårbarhet, större incidentrisk eller svårare återställning. Det är mer användbart än en lista som bara säger att något är ”gammalt”.

När något ska avvecklas

Självständiga tjänster bör också kunna tas bort. Men avveckling är inte samma sak som att stänga en server eller radera ett repository. Först

måste man veta vilka användare och tjänster som fortfarande använder gränssnitten, var data finns, om information ska bevaras och vilka drift- och säkerhetsfunktioner som kan tas bort.

För dokumenttjänstens gamla API-version kan avvecklingsplanen innebära att identifiera konsumenter, annonsera sista stöddatum, migrera ärendetjänsten, övervaka återstående trafik och först därefter stänga gränssnittet. Loggning och telemetri kan behövas under en övergångsperiod för att upptäcka användning som inte finns dokumenterad. Avveckling av en hel tjänst kan dessutom kräva export, arkivering eller gallring av information enligt verksamhetens beslut och gällande regler.

Om en plattformstjänst ska upphöra måste plattformsteamet erbjuda en fungerande migreringsväg. Om en applikation inte kan följa med behöver orsaken och det tidsbegränsade ansvaret dokumenteras. Annars skapas nya tekniska särlösningar i försöket att bli av med gamla.

Ett gemensamt förvaltningssamtal

Tänk dig ett kvartalsvis livscykelmöte för ärendehanteringsstödet. Dokumentteamet visar att ett bibliotek måste bytas innan stödet upphör. Driftteamet visar att den nuvarande körmiljön behöver uppgraderas. Ärendeteamet redovisar att det fortfarande använder ett äldre dokument-API. Säkerhetsfunktionen pekar på en känd sårbarhet och verksamheten informerar om en period när större förändringar bör undvikas.

Mötet behöver inte detaljstyra varje uppgradering. Syftet är att identifiera beroenden och beslut som inget team kan lösa ensamt. Vem äger åtgärden? Vilken sista tidpunkt gäller? Vad händer om den missas? Vilken åtgärd måste genomföras först? Vem finansierar arbetet och vem accepterar kvarstående risk? Vilket underlag visar att risken är avklarad? Sådana frågor skapar en gemensam plan utan att återinföra en enda stor release för allt.

För applikationssidan innebär livscykelansvaret att kod, bibliotek, kontrakt och datamigreringar måste kunna vidareutvecklas utan långa stopp. **För driftsidan** innebär det att operativsystem, baskomponenter, databaser och plattformstjänster behöver uppdateras med kända beroenden och stödperioder. **För den samlade systemförvaltningen** handlar det om att se när en åtgärd korsar dessa gränser och säkerställa ett beslut, en ägare och en verifierbar slutpunkt.

En arkitektur med många tjänster blir inte långsiktigt hållbar bara för att varje tjänst kan driftsättas självständigt. Den blir hållbar när organisationen också klarar att uppgradera, ersätta och avveckla delarna utan att tappa kontrollen över helheten. I nästa kapitel förflyttas perspektivet från underhåll till den särskilda frågan om återställning, kontinuitet och kostnader när flera tjänster och datalager samverkar.

Källor och vidare läsning

- **R19** NIST SP 800-218, Secure Software Development Framework (SSDF) (kontrollerad 2026-10-17).

17. Hur säkerställer vi återställning, kontinuitet och kostnadskontroll?

En brand i ett serverrum eller en trasig lagringsvolym kan få omfattande följder i ett traditionellt IT-system. Då finns ofta en etablerad rutin för att återställa servrar, databas och filer från säkerhetskopior. När systemet delas upp i mikrotjänster blir frågan svårare: **vilket tillstånd är det egentligen vi ska återställa?** Det räcker inte att få varje tjänst att starta. Ärendehanteringen och dokumenthanteringen måste också kunna förstå varandras information efteråt.

Kontinuitet handlar dessutom om mer än katastrofer. Långvariga nätverksproblem, korrupta meddelanden, felaktiga ändringar och bortfall hos en extern leverantör kan stoppa viktiga arbetsmoment. Samtidigt kostar redundans, säkerhetskopiering, datalagring och reservkapacitet pengar. En hållbar lösning kräver därför att verksamhetens krav, applikationernas beteende och driftens förutsättningar möts i samma planering.

Ett återställningsproblem som inte syns i serverlistan

Föreställ dig vårt ärendehanteringssystem. Ärendetjänsten hanterar ärendestatus och hänvisningar till dokument. Dokumenttjänsten lagrar dokumentens metadata och filer i egna resurser. På tisdagen uppstår ett fel som gör dokumentdatabasen oanvändbar. Den senaste användbara

säkerhetskopian är från måndag kväll, men ärendeinformationen är aktuell fram till tisdag eftermiddag.

Driftorganisationen kan kanske återläsa dokumentdatabasen utan större svårigheter. Ändå kan ärendetjänsten då hänvisa till dokument som inte längre finns i den återställda databasen. Om ett dokument fysiskt finns i objektlagringen men dess metadata saknas behöver systemet veta om det ska återskapas, markeras för manuell hantering eller lämnas orört tills händelserna har analyserats.

Detta illustrerar skillnaden mellan **teknisk återläsning** och **verksamhetsmässig återställning**. Den första handlar om att få databas, filer och tjänster tillgängliga. Den andra handlar om att användaren åter kan lita på att ärenden och dokument beskriver ett begripligt, korrekt tillstånd. R20

Två mått som måste översättas till verksamhetskrav

Två vanliga kontinuitetsmått är *Recovery Time Objective* (RTO), den längsta planerade tiden för återställning, och *Recovery Point Objective* (RPO), den maximalt accepterade förlusten av data uttryckt som tid. Måtten är mål, inte garantier. **RTO måste knytas till vilken verksamhetsförmåga som ska vara återställd**, inte bara till att en databas svarar. RPO behöver på motsvarande sätt avse det informationsflöde vars förlust verksamheten ska acceptera, inte slentrianmässigt enbart tidpunkten för senaste backup. De säger dessutom inte automatiskt att ett verksamhetsflöde blir korrekt efter återställningen.

Om verksamheten kräver att ärenden ska kunna öppnas inom fyra timmar, men accepterar att dokumentuppladdning är begränsad längre, kan funktionerna ha olika återställningsprioritet. Om inga nya dokument får gå förlorade räcker däremot inte en nattlig databaskopia, även om dokumenttjänsten kan startas snabbt. Lösningen kan kräva tätare skydd, loggbaserad återställning eller andra mekanismer för beständighet och återskapande.

Verksamheten behöver uttrycka **vilka arbetsuppgifter som inte får avbrytas, hur länge de får vara otillgängliga och vilka informationsförluster som kan accepteras**. Applikationsteamen översätter detta till regler för data, beroenden och kompensation. Drift och platt-

form bedömer vilka tekniska åtgärder som krävs och vilka kostnader de för med sig. Säkerhets- och informationsansvariga behöver delta där lagring, integritet eller regelkrav påverkas.

Säkerhetskopiering över flera tjänstegränser

Säkerhetskopior kan tas på olika nivåer: databas, lagringsobjekt, konfiguration och vissa plattformstillstånd. Ett komplett återställningsunderlag kan också behöva innehålla versionsuppgifter, schemahistorik, nycklar och information om hur infrastrukturen återskapas. Hemligheter ska skyddas särskilt; kopior och återställningsförfaranden får inte bli en genväg runt behörighetskontrollen.

När flera datalager samverkar finns inte nödvändigtvis någon gemensam tidpunkt då all information kan kopieras atomärt. Att läsa tillbaka två individuellt korrekta kopior kan därför ge en inkonsistent helhet. Lösningen är inte alltid att försöka frysa hela systemet. Det kan vara mer realistiskt att dokumentera vilka data som är auktoritativa, vilka händelser som kan återuppspelas och vilka avvikelser som behöver hanteras genom avstämning eller kompensation.

Säkerhetskopian behöver också kunna användas i en praktisk återställningskedja: rätt version av programvara, databas- och meddelandescheman, krypteringsnycklar och behörigheter måste gå att återskapa eller ersätta. En kopia som inte kan läsas med tillgängliga nycklar har begränsat återställningsvärde.

En **replika** kan förbättra tillgängligheten men är inte liktydig med en säkerhetskopia. Felaktig radering, korruption och vissa säkerhetsincidenter kan spridas även till repliker. Säkerhetskopior behöver därför ha en genomtänkt skyddsmodell, ofta med separata åtkomsträttigheter och möjlighet att bevara återställningspunkter som inte kan ändras av det ordinarie produktionsflödet. Hur länge kopior bevaras ska styras av fastställda krav, inte av antagandet att mer alltid är bättre.

Återuppspelning, idempotens och avstämning

Anta att dokumenttjänsten återställs till ett äldre tillstånd. Ärendetjänsten kan fortfarande innehålla händelser eller referenser som skapades

efter denna tidpunkt. Om händelserna finns bevarade och deras innebörd fortfarande är giltig kan delar av informationen ibland byggas upp genom återuppspelning. Det förutsätter att historiken är tillräckligt komplett och att återuppspelningen sker i ett kontrollerat läge; man kan inte utgå från att en vanlig meddelandekö fungerar som ett obegränsat arkiv. Men det måste finnas regler för dubbletter, ordningsföljd och operationer som inte får utföras två gånger.

En omspelad händelse om att ett dokument registrerats ska exempelvis inte orsaka en ny, oavsiktlig uppladdning eller skicka ytterligare en extern notifiering. Det kan kräva att externa sidoeffekter spärras under återläsningen och att separata rutiner används när behandlingen återupptas. Det kräver idempotenta operationer eller särskild deduplicering. Det kräver också att meddelandenas livslängd och historik faktiskt räcker till för det tidsintervall som ska återställas.

Inte alla fel går att reparera automatiskt. En avstämning kan jämföra ärendereferenser, dokumentmetadata och faktiska lagringsobjekt. Resultatet kan vara en lista med avvikelser: saknad metadata, förlorad fil, dubbelregistrering eller en referens som inte längre ska finnas. **Vem som får korrigera vad** behöver vara bestämt i förväg, särskilt när informationen har juridisk eller verksamhetsmässig betydelse. I vissa fall ska processen stoppas och en ansvarig människa besluta om fortsatt hantering.

Återställningsordning och beroenden

I en monolit är återstartordningen ofta känd för en samlad applikation. Ett distribuerat system kan ha fler beroenden: identitetstjänst, namnuppslagning, nätverk, databaser, meddelandeköer, nyckelhantering och själva driftsättningsplattformen. Om återställningsplanen förutsätter att en central tjänst redan fungerar kan återhämtningen fastna när just den tjänsten ligger nere.

En praktisk plan beskriver vilka komponenter som måste fungera först, vilka som kan starta i begränsat läge och hur applikationer ska reagera medan beroenden saknas. Den bör också hantera situationen att övervakning, administration eller reservmiljön inte är fullt tillgänglig.

För ärendehanteringen kan en rimlig prioritering vara att först säkra inloggning och läsning av befintliga ärenden, därefter få tillbaka dokumentåtkomst och sist behandla eftersläpande notifieringar. Men den prioriteringen är ett verksamhetsbeslut och kan se annorlunda ut om

dokumenten krävs för att fatta rättssäkra beslut. Det är därför inte teknikteamens uppgift att ensamma bestämma vad som är »tillräckligt återställt«.

Återställningsövningar avslöjar luckor

En säkerhetskopia som aldrig har lästs tillbaka är ett antagande om återställningsförmåga. Kontrollerade övningar behöver visa att kopior kan hittas, dekrypteras, läsas och användas för att återstarta tjänsterna i en isolerad miljö. Övningen bör också omfatta de verksamhetsmässiga kontrollerna: kan användaren hitta rätt dokument, stämmer ärendets status och kan nya ärenden skapas utan att gamla operationer dubbleras?

Övningar kan göras stegvis. En grupp kan först återställa en enskild databas, därefter en hel tjänst och slutligen ett scenario med förlorad dokumentmetadata men intakt ärendedatabas. På så sätt går det att observera var beroenden, behörigheter och otydliga rutiner hindrar återhämtningen. Testerna ska använda skyddade eller syntetiska data och får inte riskera att överskriva produktion.

Resultatet bör dokumenteras i faktisk återställningstid, uppnådd återställningspunkt, kvarvarande dataskillnader och åtgärder som måste prioriteras. Även åtkomst till reservkopior och nycklar ska provas med de roller och behörigheter som faktiskt finns vid en större störning. Om de faktiska resultaten inte når upp till målen behöver antingen lösningen förbättras eller kraven omprövas med verksamheten.

Kontinuitet utan fullständig återställning

All kontinuitet behöver inte bygga på att varje komponent finns i dubbel uppsättning. I vissa situationer kan verksamheten arbeta vidare med begränsad funktionalitet, kölagda registreringar eller en tillfällig manuell rutin. Men reservrutiner måste ha tydliga gränser. Om användare fortsätter att arbeta offline behöver man veta hur poster senare förs in, hur konflikter upptäcks och hur man undviker dubbelregistrering.

Kontinuitetsplaneringen behöver också ta hänsyn till personal, beslutsvägar, kommunikation och beroendet av externa leverantörer. Att ha en teknisk reservlösning hjälper föga om ingen har mandat eller åtkomst att aktivera den. Liksom vid en incident ska det vara tydligt vem som fattar beslut om att växla arbetssätt och vem som meddelar användarna.

Vad kostar förmågan att återhämta sig?

Mikrotjänster kan ge bättre möjligheter att dimensionera delar av systemet var för sig. Samtidigt tillkommer kostnader för fler tjänsteinstanser, databaser, lagringskopior, nätverkstrafik, övervakning och automation. Reservkapacitet och geografisk redundans kan ytterligare öka kostnaderna. Den totala kostnadsbilden kan därför inte förstås enbart genom att summera utvecklingsteamens applikationsbudgetar.

För kostnader bör man skilja **kostnad för ordinarie drift**, **kostnad för beredskap och skydd** samt **möjlig ekonomisk och verksamhetsmässig konsekvens av längre avbrott**. Kostnadskontroll börjar med synlighet. Det behöver gå att skilja mellan gemensamma plattformskostnader och tjänstespecifika resurser, följa lagringsmängder, trafik, databasstorlek och kapacitetsutnyttjande samt förstå vad en ny kravbild på tillgänglighet eller RPO/RTO innebär ekonomiskt. Gemensamma resurser kan fördelas med en överenskommen modell, men siffrorna bör inte ge sken av större precision än underlaget medger.

En kostnadsbesparing som tar bort återställningspunkter eller reservkapacitet är inte neutral: den förändrar systemets risk. Omvänt kan dubblerad kapacitet i flera tjänster vara en onödig kostnad om verksamheten accepterar kortare störningar. Organisationen behöver därför jämföra **kostnad, förväntad konsekvens vid avbrott och verifierad återhämtningsförmåga**, snarare än att optimera varje tjänst för sig.

Vem äger den samlade kontinuiteten?

Ett applikationsteam kan äga sin tjänsts dataformat, återstartsbeteende och möjligheter till avstämning. Drift och plattform kan tillhandahålla säkerhetskopiering, lagring, automatiserad återläsning och övningsmiljöer. Säkerhetsfunktionen kan ange krav för skydd och hantering av kopior. Verksamhetsansvarig behöver definiera tolerabelt avbrott och informationsförlust. Men någon måste också se till att dessa delar bildar en **gemensam och testad återställningsplan för IT-stödet**.

Det ansvaret kan ligga hos en utsedd tjänsteägare, förvaltningsledning eller tvärfunktionell kontinuitetsfunktion. Organisationsformen varierar, men mandatet får inte bli underförstått. Det behöver finnas en ägare till prioriteringsordningen, beslutet om när återställningen är tillräcklig,

samordningen av övningar och budgetavvägningen mellan driftkostnad och risk.

Från att starta servrar till att återfå verksamhetsförmåga

Vårt dokumentexempel visar varför distribuerad arkitektur förändrar återställningsfrågan. Målet är inte bara att ärendetjänsten och dokumenttjänsten åter ska svara på anrop. Målet är att ärenden och dokument åter bildar en tillförlitlig helhet och att användarna vet vilka moment som fungerar, vilka som ännu väntar och vilka uppgifter som kan kräva manuell kontroll.

Samtidigt måste denna förmåga finansieras och övas under hela systemets livscykel. Den verkliga kontinuitetsnivån är den organisationen kan visa i en återställningsövning, inte den som enbart står i en plan.

Källor och vidare läsning

- **R20** NIST SP 800-34 Revision 1, Contingency Planning Guide for Federal Information Systems (updated 2010) (kontrollerad 2026-10-17).

18. Hur organiserar vi ansvar och styrning över tid?

När ärendehanteringen kördes som ett sammanhållet system var det ofta möjligt att tala om ett systemförvaltningsobjekt, en samlad leveransplan och en etablerad driftorganisation. Den uppdelade lösningen består i stället av ärendetjänst, dokumenttjänst och flera gemensamma plattformsfunktioner. Var och en kan ändras, driftsättas och underhållas separat. Ändå möter användaren fortfarande **ett** IT-stöd.

Det är just detta som gör styrningen svår: självständiga tekniska delar behöver ha tydliga ägare, men ansvaret för en fungerande verksamhetsförmåga kan inte försvinna mellan dem. Mikrotjänster tar alltså inte bort behovet av systemansvar. De förändrar vad ansvaret måste omfatta och hur det utövas.

Tre nivåer av ansvar

Vi kan skilja mellan tre kompletterande ansvarsnivåer. För det första finns **verksamhetsansvaret**: vilka arbetsmoment ska fungera, vilken information är kritisk och vilka avbrott kan tolereras? För det andra finns **tjänsteansvaret**: vem bestämmer över en tjänsts beteende, kontrakt, kvalitet och vidareutveckling? För det tredje finns **plattform- och driftansvaret**: vem tillhandahåller och förvaltar körmiljö, nät, lagring, identitetsfunktioner, övervakning och säker drift?

Nivåerna sammanfaller ibland organisatoriskt. Ett team kan både ut-

veckla och driva sin tjänst. På andra håll är utveckling och drift skilda, eller så levereras en del av plattformen av en extern leverantör. Inget av alternativen är automatiskt rätt för alla verksamheter. Avgörande är att gränserna är begripliga och att beslut, incidenter och förändringar inte fastnar mellan dem.

Ett fjärde, sammanhållande uppdrag behövs ofta: någon måste ha mandat att följa upp **hela ärendehanteringsförmågan**, även när ett problem inte kan hänföras till en enda tjänst. Uppdraget kan ligga hos en produktägare, en systemansvarig eller en gemensam ledningsfunktion. Titeln är mindre viktig än ett tydligt mandat, finansiering och möjligheten att prioritera gemensamma förbättringar. Helhetsansvar innebär inte att personen ensam utför alla åtgärder eller kan överpröva säkerhetskrav; mandatets gränser måste vara definierade.

Vem bestämmer, vem utför och vem följer upp?

Ett vanligt misstag är att skriva att ”teamet ansvarar” utan att ange vilken sorts ansvar som avses. Att formulera behov, fatta ett riskbeslut, genomföra en ändring och verifiera resultatet är olika handlingar.

Anta att dokumenttjänsten behöver byta lagringslösning. Verksamheten behöver formulera krav på åtkomst, sekretess och återställningstid. Dokumentteamet behöver bedöma hur information och gränssnitt påverkas. Plattform och drift behöver beskriva tekniska alternativ och deras operativa konsekvenser. Säkerhetsfunktionen behöver bidra med krav och granskning. En utsedd beslutsfattare behöver godkänna kvarvarande risk och kostnad. Efter införandet behöver någon kontrollera att dokumentflödet fortfarande fungerar enligt de överenskomna målen.

En enkel ansvarsmatris, exempelvis inspirerad av RACI, kan hjälpa till. Men matrisen bör inte bli en katalog över alla tänkbara arbetsmoment. Välj i stället de beslut och händelser som verkligen kräver samordning: nya tjänster, kontraktsändringar, produktionssättning med förhöjd risk, större incidenter, säkerhetsundantag och avveckling. För varje sådan situation bör det framgå **vem som beslutar, vem som genomför, vem som måste rådfrågas och vem som följer upp**. Det behöver också finnas en väg framåt när parterna inte är överens. En ansvarsmatris beskriver uppgifter, men ersätter inte formella befogenheter, avtal eller den lagstiftning som styr verksamheten. Särskilt för säkerhetsrisker och

personuppgifter måste det vara klart vem som får acceptera vilken risk och när frågan måste lyftas vidare. R21

Självständiga team inom gemensamma ramar

En viktig fördel med mikrotjänster kan vara att olika team arbetar i sin egen takt. Om varje dokumentförändring måste passera samma centrala godkännande som en stor systemrelease försvinner en del av vinsten. Men om teamen får fatta alla beslut utan gemensamma ramar ökar risken för inkompatibla gränssnitt, otydlig informationsförvaltning och varierande säkerhetsnivåer.

Gemensamma ramar kan exempelvis ange hur identiteter hanteras, vilka minimikrav som gäller för loggar och spårning, hur sårbarheter rapporteras, hur API:er versioneras och hur tjänster registreras i en katalog. Ramarna bör i största möjliga mån byggas in i utvecklings- och plattformstjänsterna som **användbara standardvägar**, snarare än kontroller som kräver manuell handläggning vid varje liten ändring.

Undantag kommer ändå att behövas. Ett undantag från en teknisk eller säkerhetsmässig regel bör därför ha en namngiven beslutsfattare, dokumenterad motivering, tidpunkt för omprövning och plan för hur risken begränsas. ”Det har alltid fungerat så” är inte en hållbar undantagsprocess.

En gemensam plan utan en gemensam release

I monoliten kan den årliga förvaltningsplanen ofta organiseras kring en produkt och några större releaser. I ett mikrotjänstlandskap finns flera leveransplaner, men också beroenden som kräver samordning. Dokumenttjänstens ändrade kontrakt kan behöva förberedas innan ärendetjänsten börjar använda det. En plattformsuppgradering kan påverka alla tjänster. Gemensamma sårbarheter kan kräva åtgärder inom samma tidsfönster.

Förändringar i delade gränssnitt behöver dessutom en dokumenterad kommunikationsväg till konsumerande team, även när den levererande tjänsten kan driftsättas självständigt.

En sammanhållen plan behöver därför visa både **varje tjänsts prioriteringar** och **de beroenden som sträcker sig över teamgränserna**. Den behöver inte detaljstyra varje driftsättning. Däremot bör den synliggöra planerade större förändringar, tekniska livscykelbeslut, gemensamma risker, kapacitetsbehov och mål för hela IT-stödet.

Om den samlade verksamhetsförmågan mäts med tjänstenivåmål, exempelvis tiden från uppladdning till synligt dokument, måste även ägarskapet för måttet vara tydligt. Ett team kan ansvara för sin tjänsts indikatorer, men ett helhetsmått behöver en funktion som kan samla flera team kring förbättringar. Detta ersätter inte respektive tjänsts ansvar utan lägger en gemensam nivå ovanpå det.

Finansiering och kostnader som inte följer teamgränserna

En distribuerad lösning kan göra vissa kostnader mer synliga och andra svårare att fördela. Ärendetjänsten och dokumenttjänsten kan ha egna beräkningsresurser och databaser. Samtidigt delas kostnader för nätverk, övervakning, säkerhetsverktyg, reservkapacitet, utvecklingsmiljöer och incidentberedskap.

Det går att märka resurser och följa kostnader per tjänst, men alla gemensamma kostnader kan inte utan vidare fördelas rättvist efter processortid eller datamängd. En enkel och redovisad fördelningsprincip är ofta bättre än en komplicerad modell som ger falsk precision. Kostnadsfördelning bör också skiljas från beslutsrätt: den som får en intern kostnad påförd kan sakna möjlighet att påverka en gemensam plattformsutgift. Det viktiga är att synliggöra vad en tjänst faktiskt förbrukar, vilka gemensamma kostnader som behövs och vem som kan besluta om investeringar för att minska framtida risk.

Teknisk skuld hör hemma här. Ett team kan behöva uppgradera ett ramverk utan att någon ny funktion tillkommer. Plattformen kan behöva byta en komponent som snart saknar stöd. Om varje budget endast belönar synliga verksamhetsfunktioner blir nödvändigt underhåll ständigt uppskjutet. Långsiktig styrning behöver därför ge plats för både nytta, riskreducering och förvaltning.

När flera delar fallerar samtidigt

Föreställ dig att användarna inte kan öppna dokument i sina ärenden. Dokumenttjänsten rapporterar timeout. Samtidigt har en gemensam nätverksförändring påverkat kommunikationen mellan tjänsterna. Vem äger incidenten?

Dokumentteamet bör kunna undersöka sina anrop och avgöra hur tjänsten påverkas. Plattform och drift bör kunna analysera nätverksförändringen och återställa förbindelsen. Men någon måste också hålla ihop prioritering, lägesbild, användarinformation och beslut om när den samlade verksamhetsfunktionen kan betraktas som återställd. Det är inte rimligt att kräva att användaren ska veta vilket team som först borde kontaktas.

Kapitel 14 beskrev incidenthanteringens praktiska förlopp. Styrningsfrågan här är att **mandatet måste vara bestämt innan incidenten inträffar**. Det bör vara klart vem som leder en större incident, vem som får fatta vilka produktionsbeslut, hur leverantörer kontaktas och hur en konflikt mellan lokal tjänsteoptimering och återställning av helheten löses. Incidentledaren behöver inte själv ha administratörsbehörighet eller ensam besluta om varje teknisk åtgärd.

Efter en incident behöver åtgärder som berör flera tjänster få en utsedd samordningsansvarig, namngivna utförare och ett datum för uppföljning. Åtgärderna behöver prioriteras i förvaltningsplaneringen. Annars kan alla team avsluta sin lokala uppgift medan den grundläggande samverkansbristen består.

När utveckling och drift är olika organisationer

Det finns en förenklad bild av mikrotjänster där varje utvecklingsteam alltid sköter all drift. Så behöver det inte vara. En organisation med etablerad drift- eller infrastrukturkompetens kan välja att behålla separata funktioner. Men uppdelningen får inte betyda att applikationen lämnas över som en svart låda.

Applikationsteamet måste bidra med driftbar kod, diagnostik, tekniska beroenden, återställningsrutiner och förväntat beteende vid fel. Drift- och plattformsteamen måste tillhandahålla tydliga tjänster, förändringsinformation, incidentvägar och kunskap om infrastrukturen. De behöver

också kunna återföra erfarenheter från produktion till utvecklingen. Gemensamma övningar och granskningar av verkliga störningar bygger ofta bättre samverkan än ännu ett organisationsschema.

Där leverantörer ingår måste avtalen och gränssnitten stödja denna samverkan. En leverantörs åtagande för en databas eller en driftplattform är inte automatiskt ett åtagande för hela ärendeprocessen. Organisationen som använder IT-stödet behöver fortfarande kunna bedöma och styra de samlade riskerna.

Kompetens, dokumentation och kontinuitet i ansvaret

Självständighet kräver att teamen förstår sina tjänsters gränser och beroenden. En tjänstekatalog bör därför inte bara ange ett namn och en nätverksadress. Den behöver ange vem som förvaltar tjänsten, hur teamet nås vid incidenter, vilka kontrakt den erbjuder, vilka viktigare beroenden som finns och hur förändringar kommuniceras. Informationen måste hållas aktuell, helst i anslutning till det ordinarie leveransarbetet.

Tjänstekatalogen kan även behöva beskriva ansvar för data, integrationskonsumenter, versioner med återstående stödtid och var driftinstruktioner finns. Ägarskapsuppgifter som bara finns i en organisationsbild blir snabbt föråldrade.

Ett särskilt problem uppstår när team omorganiseras. Om dokumentteamet läggs ned eller byter uppdrag får dokumenttjänsten inte bli ägarlös. Det behövs regler för överlämning av kod, behörigheter, avtal, körmiljöer, larm och teknisk kunskap. Samma sak gäller om en extern leverantör byts ut.

Kompetensförsörjning omfattar därför mer än vilka tekniker organisationen kan rekrytera. Den handlar även om gemensam förståelse för hela verksamhetsflödet, övningar över organisationsgränser och tid avsatt för att lära av produktionen.

Styrning som möjliggör helhetsansvar

I kapitel 3 beskrev vi de praktiska ansvarsgränserna mellan tjänst, plattform och IT-stöd. Här har vi i stället sett hur de gränserna måste **förvaltas, finansieras, följas upp och överlämnas** över tid. Vi kan sam-

manfatta skillnaden så här: i den traditionella modellen samlas många tekniska beslut inom en gemensam system- och driftstruktur. I en mikrotjänstlösning flyttas vissa beslut närmare tjänsteteamen, medan andra behöver lyftas till en gemensam nivå. Det är inte fråga om mindre styrning utan om **styrning på rätt nivå**.

För applikationsläsaren blir slutsatsen att ett tydligt tjänsteägarskap innebär mer än ansvar för kod och funktioner. För driftläsaren innebär det att plattformens och infrastrukturens tjänster måste beskrivas som begripliga åtaganden med gränssnitt och mandat. För båda gäller att någon behöver kunna svara för att användarna faktiskt kan hantera sina ärenden, oavsett hur många tekniska delar som krävs.

I nästa kapitel går vi från den långsiktiga ansvarsfördelningen till övergången i sig: hur en organisation kan leva med gammal och ny arkitektur samtidigt utan att tillfälliga lösningar blir permanenta.

Källor och vidare läsning

- **R21** James Lewis och Martin Fowler, Microservices (kontrollerad 2026-10-17).

19. Hur lever vi med gamla och nya arkitekturer samtidigt?

Det är frestande att beskriva modernisering som ett före och ett efter. Först har organisationen en monolit, sedan har den ett antal mikrotjänster. I praktiken finns det ofta en lång mellanperiod när gammal och ny arkitektur används samtidigt. Ärendehanteringen körs fortfarande som en sammanhållen applikation på sina virtuella servrar, medan dokumenthanteringen har flyttats till en egen tjänst. Båda lösningarna är produktionskritiska. Båda behöver utvecklas, övervakas, skyddas och förvaltas.

Det svåra är inte bara att få det gamla och nya att tala med varandra. Det svåra är att se till att **övergången faktiskt leder mot en enklare och mer ändamålsenlig målbild**, i stället för att skapa två permanenta tekniska världar som måste hållas vid liv.

Mellanläget är ett riktigt driftläge

När dokumentfunktionen först bryts ut påverkas användaren kanske knappt alls. Hen öppnar fortfarande ett ärende i samma gränssnitt och klickar på samma knapp för att bifoga en fil. Men bakom knappen har ansvaret förändrats. Den äldre ärendeapplikationen anropar dokumenttjänsten över nätverket och kan behöva hantera tidsgränser, identiteter och tillfälliga störningar som tidigare inte fanns mellan funktionerna.

Samtidigt kan det äldre systemet fortsätta vara beroende av den gamla

databasen, schemalagda jobb och verktyg för serverdrift. Den nya dokumenttjänsten kan driftsättas via en automatiserad leveranskedja och använda andra mekanismer för loggning, nätverk och hemlighetshantering. Detta är inte nödvändigtvis fel. Olika livscykler är själva poängen med en stegvis förändring. Men olikheterna måste vara synliga och accepterade i organisationens förvaltningsplan.

Mellanläget behöver därför ha en uttrycklig ägare, en riskbild, avtalade integrationsgränser, mätbara kriterier för nästa etapp och en tidshorisont som kan omprövas. Varje omprövning bör redovisa varför gamla beroenden fortfarande behövs, i stället för att automatiskt förlänga övergången. Annars är risken att det beskrivs som tillfälligt år efter år utan att någon finansierar de sista stegen. R22

Flytta förmågor stegvis i stället för att skriva om allt

Ett vanligt angreppssätt är att gradvis låta en ny lösning ta över avgränsade funktioner från den gamla. Detta brukar kallas *strangler-mönstret*: nya gränssnitt eller tjänster ersätter delar av en äldre lösning tills de delar som inte längre behövs kan stängas av.

För ärendehanteringen kan första steget vara att en ny dokumenttjänst tar hand om nya uppladdningar. Nästa steg kan vara att också läsa äldre dokument genom samma gränssnitt. Det betyder inte att hela ärendehanteringen behöver delas upp. En lyckad modernisering kan bestå av en begränsad utbrytning där resten av monoliten lever vidare med tydligare interna gränser.

Det viktiga är att varje etapp ger en prövbar verksamhetsnytta och minskar ett identifierat problem. Utan sådana mål kan utbrytningen skapa nya nätverksanrop och mer driftarbete utan att användarna eller teamen vinner tillräckligt mycket.

Integrationslager som skydd mellan olika modeller

Det äldre systemets datamodell är sällan ett bra kontrakt för en ny mikrotjänst. I monoliten kan ett dokument vara en rad i samma databas

som ärendet, med direkt åtkomst till interna tabeller. Den nya dokumenttjänsten behöver i stället ett avgränsat API och egna regler för metadata, åtkomst och lagring.

Ett **översättande integrationslager**, ibland kallat *anti-corruption layer*, kan omvandla äldre begrepp och anrop till den nya tjänstens språk. Det ger ärendeapplikationen en stabil koppling utan att dokumenttjänsten behöver överta alla äldre antaganden. Exempelvis kan en gammal intern statuskod översättas till ett mer uttryckligt dokumenttillstånd. Översättningen bör ha tydlig ägare och egna tester. Det behöver också framgå om lagret är en del av monoliten, en separat adapter eller den nya tjänstens gräns; placeringen påverkar driftsättning, behörigheter och felsökning.

Lagret har samtidigt en kostnad. Om det blir platsen där all ny verksamhetslogik hamnar har man skapat ännu en komponent som behöver förvaltas. Syftet är att skydda gränsen, inte att dölja en permanent oklarhet om vilket system som bestämmer vad.

Vem äger informationen under flytten?

En av de riskfyllda övergångarna gäller data. Om både monoliten och dokumenttjänsten får skriva samma dokumentmetadata är det svårt att veta vilken version som är rätt vid en konflikt. Det kan fungera under mycket avgränsade kontrollerade former, men delat skrivansvar får inte bli en omedveten standardlösning.

Organisationen behöver fastställa **vilken komponent som är auktoritativ för varje uppgift under varje etapp**. Det räcker inte att ange en ägare för hela dokumentet: dokumentinnehåll, metadata, behörighetsbeslut och kopplingen till ärendet kan ha olika ansvariga. För varje uppgift bör läsare, skrivväg och ansvarig vid avvikelse vara dokumenterade. I ett första steg kan monoliten äga äldre metadata och dokumenttjänsten äga nya uppladdningar. I ett senare steg kan dokumenttjänsten ta över allt dokumentinnehåll och all dokumentmetadata medan ärendetjänsten behåller relationen mellan ärende och dokument. Det senare ansvaret kan uttryckas som ett dokument-ID och ett tillstånd, inte som direkt tillgång till dokumenttjänstens tabeller.

Ibland behöver data kopieras mellan miljöerna under en övergång. Kopiering kan ske som en engångsmigrering, i mindre omgångar eller genom synkronisering av förändringar. Varje metod kräver en verifierbar plan

för provmigrering, avstämning, felhantering och beslut om övergång till den nya källan. Är antalet poster rätt? Har filerna korrekt innehåll? Är behörigheterna bevarade? Går det att spåra vad som migrerats och vad som misslyckats? Det räcker inte att databastabellerna ser lika ut om dokument inte längre går att öppna i verksamhetsflödet.

Om data förs över löpande måste man också definiera fördröjning och avstämning. En användare får inte tro att en handling försvunnit bara för att uppdateringen ännu inte har nått lässidan. Därför behövs tydliga tillstånd och ett sätt att upptäcka och rätta avvikelser.

Undvik att skapa två system som båda bestämmer

Att låta två system parallellt genomföra samma affärsoperation, så kallade dubbla skrivningar, kan verka som en enkel migrationsväg. Men det går inte att anta att två separata skrivningar antingen lyckas eller misslyckas tillsammans. Det ena systemet kan uppdateras medan det andra får ett nätverksfel.

Ett säkrare utgångsläge är att styra varje skrivoperation till en bestämd ägare. Om den andra miljön behöver information kan den få en kopia via händelser eller en kontrollerad synkroniseringsprocess. Då behövs hantering av duplicerade meddelanden, fördröjningar, ordningsföljd och avstämning. Ett händelseflöde är inte i sig en garanti för att den mottagande sidan har en aktuell eller korrekt kopia. Om dubbla skrivningar ändå väljs krävs en medveten strategi för konflikter, återhämtning och beslut om vilket resultat som gäller.

När äldre dokument flyttas behöver cutover – bytet från gammal till ny källa – planeras i förväg. Ett återgångsbeslut är inte automatiskt möjligt efter att data börjat ändras i den nya lösningen. Teamen måste därför veta vilka steg som är reversibla, när återgången stängs och hur felaktiga eller ofullständiga poster hanteras.

Två driftsmodeller – ett sammanhållet säkerhetskrav

För driftsidan kan övergången innebära att virtuella servrar och traditionell förändringshantering samsas med en plattform för mer självständiga

tjänster. Nätverksvägar, namnuppslagning, brandväggsregler, certifikat, reservkopior och övervakning måste fungera i båda miljöerna. Den nya dokumenttjänsten kan behöva anropa en äldre identitetslösning, samtidigt som den gamla ärendeapplikationen behöver tillgång till det nya tjänste-API:t.

Sådana förbindelser bör dokumenteras som namngivna och ägda beroenden. Det är särskilt viktigt om en tillfällig brandväggsöppning eller en övergångsidentitet riskerar att bli kvar med alltför breda rättigheter. Säkerhetskraven ska gälla hela informationsflödet även om tekniken för att uppfylla dem skiljer sig åt. Särskilt behöver teamen verifiera att åtkomst som nekas i det gamla systemet inte av misstag tillåts genom det nya gränssnittet; nätverksåtkomst och tjänsteautentisering ersätter inte användarens behörighetsprövning.

En ny driftsmodell innebär inte heller att det äldre systemet kan sluta underhållas. Så länge det är produktionskritiskt behöver det patchning, kapacitet, backup, återläsningsförmåga och kompetens. Moderniseringsbudgeten måste rymma även detta arbete.

Testa övergången som en helhet

Ett test av dokumenttjänsten kan vara godkänt utan att användaren kan bifoga dokument från monolitens gränssnitt. Därför måste de kritiska flödena testas över gränsen. Relevanta scenarier är exempelvis uppladdning, läsning, borttagning enligt gällande regler, behörighetsändring och visning av dokument som ännu migreras.

Övergången ger också blandade versionslägen: en äldre ärendeapplikation kan använda en ny dokumenttjänst, och ett gammalt integrationsmeddelande kan behandlas efter att tjänsten har uppdaterats. Kontrakt och kompatibilitet behöver därför provas utifrån de kombinationer som faktiskt kan förekomma under migreringen, inte enbart den senaste versionen av varje komponent.

Drift- och applikationsteam behöver gemensamt genomföra åtminstone några felprov: Vad händer om kommunikationen mellan de två miljöerna bryts? Vad ser användaren? Hur larmas rätt grupp? Och finns dokument som lagrats men ännu inte kopplats till sitt ärende? Svar på de frågorna är en förutsättning för ett säkert etappbyte. Inför varje etapp bör teamen även ha gemensamma kriterier för start, stopp och godkännande, inklusive vilken verksamhetsfunktion som ska återtestas efter en återgång eller

korrigering.

Vem äger övergången?

En stegvis modernisering behöver mer än separata leveransplaner. Någon måste kunna prioritera när arbetet med den nya tjänsten konkurrerar med felrättningar i monoliten eller med plattformsuppgraderingar. Det sammanhållande ansvaret kan ligga hos en systemägare, en produktledning eller ett särskilt moderniseringsprogram. Organisationen kan variera, men mandatet måste täcka helheten.

Verksamheten avgör vilka arbetsflöden som inte får störas och vilken informationsförlust eller tillfällig begränsning som kan accepteras. **Applikationsteamen** utformar tjänstegränser, datamigrering, kontrakt och felhantering. **Drift- och plattformsteamen** ansvarar för de förutsättningar de tillhandahåller, inklusive nätverk, identitet, övervakning och återställning. **Säkerhetsfunktioner** bidrar med krav och verifiering utifrån informationsskyddet. För gemensamma förändringar måste det vara klart vem som beslutar om driftsättning, stopp och avveckling.

Det är viktigt att inte fastna i en modell där en projektgrupp lämnar över en halvfärdig integrationslösning till den löpande förvaltningen utan tydligt ägarskap. Övergångskomponenter är också produktionskomponenter och behöver samma grundläggande ansvar för support, incidenter och livscykel.

Planera för att tillfälliga lösningar ska försvinna

För varje tillfällig adapter, synkronisering eller brandväggsregel bör det finnas ett svar på fyra frågor: Varför behövs den? Vem ansvarar för den? Vilket villkor gör att den kan tas bort? Hur verifieras att borttagningen är säker?

Avveckling kan ske när den sista konsumenten lämnat ett gammalt API, när alla nödvändiga dokument har flyttats och kontrollerats eller när ingen verksamhetsfunktion längre behöver en viss server. Då återstår ändå att stänga konton, ta bort behörigheter och nätverksregler, hantera arkiv- och bevarandekrav, avsluta reservkopieringsrutiner på rätt sätt

och uppdatera dokumentation samt kostnadsuppföljning.

Det är lättare att finansiera nyutveckling än att avveckla det sista gamla beroendet. Därför bör avvecklingsarbete vara ett uttryckligt leveransresultat, inte en vag ambition. En planerad avveckling behöver också styrkas i drift: en komponent får inte tas bort enbart därför att inga anrop synts under en kort mätperiod. Sällan använda integrationsvägar, reservrutiner och beroenden vid årsvisa händelser kan annars missas. Mät både vad som tillkommit och vad som faktiskt har försvunnit: kvarvarande äldre integrationer, aktiva synkroniseringsflöden, driftsatta servrar och beroenden som ännu kräver dubbel kompetens.

Vad förändras för vem?

För **applikationsläsaren** innebär samlevnaden att tjänstegränser och informationsägarskap behöver införas gradvis och att gammal kod inte får styra den nya tjänstens interna modell. För **driftläsaren** innebär den att två tekniska världar måste hållas säkra, övervakade och återställningsbara samtidigt, ibland under flera år.

Den gemensamma lärdomen är att mellanläget inte är ett misslyckande – men det måste styras som ett medvetet och tidsbundet arkitekturtillstånd. När dokumenttjänsten väl är fristående är det inte antalet nya tjänster som visar om moderniseringen lyckats. Det är om verksamheten fått en hållbar lösning med tydliga ansvar, fungerande flöden och färre onödiga beroenden.

I det avslutande kapitlet återstår därför den viktigaste frågan: När motiverar nyttan med mikrotjänster den distribuerade komplexitet som organisationen nu har lärt sig att hantera?

Källor och vidare läsning

- **R22** Microsoft Azure Architecture Center, Strangler Fig Pattern (kontrollerad 2026-10-17).

20. När är mikrotjänster rätt väg?

Efter att ha följt ärendehanteringssystemet från en sammanhållen applikation till ett system med flera tjänster är det frestande att ställa frågan: Borde vi dela upp resten också? Men bokens viktigaste slutsats är inte att varje monolit bör ersättas. Den är att en uppdelning förändrar vad organisationen behöver kunna utveckla, driva och förvalta – och att dessa konsekvenser måste ingå i beslutet från början.

En mikrotjänstarkitektur kan lösa verkliga problem. Den kan också skapa nya problem som är svårare än dem den var tänkt att undanröja. Valet behöver därför bygga på ett konkret förändringsbehov, inte på vilken arkitektur som för tillfället uppfattas som modern.

Vilket problem försöker vi egentligen lösa?

I vårt ärendehanteringssystem uppstår kanske ett tryck att modernisera därför att en liten förändring i dokumenthanteringen kräver en stor, gemensam release. Kanske har dokumentfunktionen helt andra krav på lagring, skalning och informationssäkerhet än ärendehanteringen. Eller så hindrar en stor kodbas flera grupper från att arbeta självständigt.

Det är tre olika problem. De behöver inte samma lösning. Innan en tjänst bryts ut behöver vi också kontrollera om problemet främst kommer från arkitekturen eller från exempelvis bristande tester, gemensamma godkännandeflöden eller otillräckliga teammandat.

Om problemet främst är att ändringar i koden påverkar för många delar, kan tydliga moduler i samma applikation räcka. Om problemet är

att dokumentlagringen behöver andra resurser, kan en separat lagringskomponent vara relevant utan att hela domänlogiken bryts ut. Om flera självständiga team däremot behöver leverera förändringar i olika takt kan en separat dokumenttjänst ge större nytta.

En användbar första fråga är därför: **Vilken förmåga behöver bli självständig, från vad och varför?** Svaret bör kunna kopplas till mätbara eller åtminstone observerbara problem: ledtid för förändringar, releasekonflikter, återkommande fel, kapacitetsbegränsningar eller svårt delat ägarskap.

Alternativen är fler än monolit eller mikrotjänster

Det finns ingen skarp linje mellan en traditionell monolit och ett system bestående av många mikrotjänster. En **modulär monolit** kan ha tydliga interna gränser, separat testbar logik och väldefinierade ägarskap, men ändå byggas och driftsättas som en enhet. Den erbjuder ofta enklare lokala transaktioner, färre nätverksberoenden och lägre operativ kostnad än ett distribuerat system. Även en modulär monolit kräver disciplin kring modulgränser och information; annars återkommer den svårföränderliga sammanflätningen inom samma process.

En arkitektur med ett fåtal större tjänster kan vara ett annat rimligt val. Dokumenthanteringen kanske behöver brytas ut, medan handläggning, ärendestatus och arbetsflöde förblir tillsammans. Det behöver inte vara ett misslyckat mellansteg; det kan vara den arkitektur som bäst motsvarar verksamhetens krav under lång tid.

En mikrotjänst är dessutom inte bättre bara för att den är liten. Om varje verksamhetsförändring kräver samtidiga ändringar i fem tjänster har man skapat distribuerad samordning snarare än självständighet. Om tjänsterna delar skrivåtkomst till samma tabeller är gränserna i praktiken svagare än organisationsdiagrammet antyder.

Därför är **oberoende förändringsbarhet och tydligt ägarskap** mer intressanta än antalet tjänster eller deras kodstorlek.

När nyttan kan överväga kostnaden

Mikrotjänster kan vara väl motiverade när olika verksamhetsförmågor förändras i tydligt olika takt och det finns team som kan ta ett uthålligt ansvar för respektive tjänst. De kan också ge värde när en del av systemet behöver annan kapacitet, särskild säkerhetsisolering eller en egen tillgänglighetsprofil. Men en separat driftsenhet ger inte automatiskt isolering: delad databas, identitetsplattform, nätverk eller kö kan fortfarande skapa gemensamma fel och angreppsvägar.

I ärendehanteringen kan dokumenttjänsten vara ett sådant fall. Den hanterar filer, lagringslivscykler och kanske stora datamängder, medan ärendetjänsten främst hanterar ärendenas status och arbetsflöden. En väl vald gräns kan göra det möjligt att utveckla dokumentfunktionen separat och att skala lagringen utan att ändra hela ärendehanteringssystemet.

Men vinsten uppstår inte automatiskt när processen flyttas till en egen server eller container. Den uppstår först när API-kontrakt, informationsägarskap, driftsättning, testning, incidentberedskap och ansvar gör att teamen faktiskt kan arbeta med mindre inbördes beroenden.

Det är också möjligt att nyttan huvudsakligen finns för en enda funktion. Då är det rimligare att bryta ut just den än att dela upp allt på samma gång.

När den enklare arkitekturen ofta vinner

Om systemet är relativt litet, förändras långsamt och förvaltas av ett litet team kan en välstrukturerad monolit vara enklare att förstå och säkrare att förändra. Detsamma gäller om verksamhetsreglerna ofta behöver ändras tillsammans eller kräver många starkt sammanhållna transaktioner.

I en monolit är vissa beroenden synliga i kod och databasscheman. I ett distribuerat system blir en del av dem i stället nätverksanrop, tidsberoenden, meddelanden och versionskontrakt. De försvinner inte: de byter form. Om organisationen saknar fungerande leveransautomation, observerbarhet, återställningsövningar och gemensamt incidentarbete kan den operativa komplexiteten snabbt överstiga den arkitektoniska vinsten.

Det betyder inte att en organisation måste ha en fullskalig plattform innan den får bygga sin första tjänst. Det betyder att förmågorna måste

byggas i takt med den faktiska uppdelningen, och att det måste finnas ett finansierat ansvar för dem. R23

En jämförelse som omfattar hela livscykeln

Vid ett arkitekturval räcker det inte att uppskatta utvecklingsinsatsen. Utöver programmering behöver vi förstå testmiljöer, nätverks- och identitetslösningar, driftsättning, övervakning, beredskap, återställning, uppgraderingar och avveckling. Dessa aktiviteter pågår så länge tjänsten finns kvar.

För utvecklingsorganisationen kan självständiga releaser ge snabbare förändringar. För driftorganisationen betyder samma självständighet att fler förändringar, versioner och felbilder behöver hanteras. För verksamheten kan snabbare leveranser ge värde, men bara om helheten fortsätter vara tillförlitlig och någon kan ta ansvar när ett arbetsflöde brister.

Jämförelsen behöver därför omfatta tre alternativ: att förbättra den befintliga monoliten, att skapa tydliga moduler inom samma driftsenhet och att bryta ut utvalda tjänster. Bedöm både investering och återkommande kostnad. Räkna inte bara servrar eller molnresurser, utan även samordning, kompetens, säkerhet, jour, testning och förvaltningsarbete. Ta med övergångsperiodens dubbla lösningar, kostnaden för avveckling och riskerna med att under en tid behöva stödja både gamla och nya gränssnitt.

Ett kostnadsunderlag kan inte reducera alla dessa faktorer till ett exakt tal. Men det kan synliggöra var antagandena ligger och vilka osäkerheter som bör testas innan ett stort vägval görs.

Beslutet prövas mot konkreta förutsättningar

Ett beslutsunderlag kan byggas kring några återkommande frågor:

1. **Verksamhetsnytta:** Vilket konkret problem ska lösas, och hur vet vi om lösningen lyckas?
2. **Gränser och data:** Finns en stabil verksamhetsgräns med tydligt informationsägarskap, eller krävs täta tvärgående uppdateringar?
3. **Team och mandat:** Kan ett team ansvara för tjänstens utveckling, leverans och operativa beteende över tid, samtidigt som hel-

hetsansvaret är tydligt?
4. **Driftförmåga:** Har vi tillräcklig automatisering, övervakning, säkerhet, incidenthantering och återställning för den komplexitet vi inför?
5. **Ekonomi och livscykel:** Vem finansierar gemensamma förmågor, löpande underhåll och en framtida avveckling?
6. **Alternativ:** Kan samma mål nås med förbättrade moduler, ändrad arbetssätt eller en mindre omfattande uppdelning?

Beslutsunderlaget bör ange vilka antaganden som ännu inte är verifierade, vem som fattar beslutet, vilka alternativ som jämförts och när effekterna ska följas upp. Även en rekommendation att behålla monoliten är ett aktivt arkitekturbeslut som kan behöva omprövas.

Frågorna är inte en poängmodell som automatiskt ger rätt svar. De är ett sätt att få utveckling, drift, arkitektur, säkerhet och verksamhet att pröva samma beslut med sina olika kunskaper. Ett svar som inte kan förklaras för någon av dessa grupper är ofta ett tecken på att förutsättningarna ännu inte är tillräckligt genomarbetade.

Börja litet och kontrollera att nyttan uppstår

Anta att vi ändå väljer att bryta ut dokumenthanteringen. Ett klokt första steg är då att definiera vad som ska bli bättre: kortare ledtid för dokumentrelaterade förändringar, färre gemensamma driftsättningar eller bättre förmåga att hantera stora filer. Samtidigt identifierar vi risker: fel vid nätverksanrop, inkonsekvens mellan dokument och ärenden och oklarheter kring behörigheter.

Därefter kan en begränsad funktion flyttas, med tydliga API-kontrakt och en plan för hur gamla och nya delar ska samverka. Före flytten behöver man bestämma vilka mått som ska jämföras med nuläget och vilka signaler som innebär att arbetet bör stoppas eller omprövas. Uppföljningen behöver mäta både önskad nytta och tillkommande kostnad. Har releaserna verkligen blivit mer självständiga? Har användarnas arbetsflöden blivit stabilare? Har drift- och säkerhetsarbetet fått rimliga förutsättningar? Är ansvarsfördelningen begriplig när en incident inträffar?

Om svaret är nej bör nästa steg inte automatiskt vara att skapa fler tjänster. Det kan vara att förenkla kontrakt, återförena alltför finfördelade funktioner eller förbättra plattformsförmågorna. Arkitektur är ett

medel som får omprövas när verkligheten ger ny kunskap.

Vad förändras för vem?

För applikationsutveckling innebär det att arkitekturvalet inte bara handlar om kodens struktur. Teamet måste visa att en tänkt tjänstegräns faktiskt minskar samordning och att det finns ett tydligt ägarskap för data, kontrakt och livscykel.

För drift och infrastruktur innebär det att kapacitet, nätverk, identitet, övervakning, återställning och säkerhet behöver vara en del av beslutet, inte något som beställs efter att uppdelningen redan är genomförd. Driftens perspektiv kan också visa när en enklare arkitektur ger bättre robusthet till lägre kostnad.

För verksamhet och ledning innebär det att välja vilka problem som är värda att lösa, finansiera de gemensamma förmågorna och säkerställa ett sammanhållet ansvar för det IT-stöd som användarna faktiskt möter.

Tillsammans behöver parterna kunna beskriva varför varje gräns finns, vem som ansvarar för den och när beslutet ska omprövas.

Arkitekturens värde visar sig i vardagen

Bokens resa började med ett välbekant system: frontend, backend, databas och en hanterbar uppsättning servermiljöer. När dokumentfunktionen bröts ut blev flera tidigare interna frågor gemensamma frågor om trafik, information, fel, leverans, drift och ansvar. Ingen enskild teknik löste dem alla.

Det avgörande är därför inte om organisationen kan kalla sin lösning mikrotjänstbaserad. Det avgörande är om systemet går att förändra i den takt verksamheten behöver, om det går att förstå och återställa när något går fel och om ansvaret fungerar under hela dess livslängd.

Den bästa arkitekturen är inte den mest uppdelade, utan den som ger rätt självständighet till en komplexitet som organisationen kan bära.

Källor och vidare läsning

- **R23** Martin Fowler, Microservice Trade-Offs (kontrollerad 2026-10-17).